LEÇON PRATIQUE · LINUX DEBIAN / UBUNTU · 2E ÉDITION, NOVEMBRE 2026

Créer un projet Java de zéro

À la main avec `javac` et `jar`, puis avec Maven, puis avec Gradle. Chaque commande est complète, expliquée, et vérifiée sur une vraie machine.

Public	Développeuse / développeur débutant à intermédiaire, étudiant, ou toute personne qui a « déjà vu Java » mais ne sait pas monter un projet propre.
Prérequis	Un terminal Linux, un JDK 21 ou 25 installé (`java -version` et `javac -version` répondent), une connexion Internet. Aucune connaissance de Maven ou Gradle.
Durée	≈ 3 h 20 de mise en pratique : packages 20 min · parcours A (à la main) 60 min · parcours B (Maven) 45 min · parcours C (Gradle) 30 min · exercices 45 min.
Objectif final	Produire un projet Java complet — code, ressources, tests, JAR exécutable, livraison — et savoir exactement ce que fait chaque outil, y compris quand il n'y a pas d'outil.

javac · java · jar · jdeps · jlink | Maven 3.10.0 · 3.9.16 | Maven Wrapper (mvnw) | Gradle 9.0 | JUnit 5.12 | Maven Central

Comment lire ce document

Un rappel d'abord (chapitre 2), puis trois parcours et un seul projet : une petite application « **boutique** » qui calcule un prix TTC et une remise, utilise une bibliothèque externe (`commons-lang3`), lit un fichier de configuration et possède de vrais tests. Vous le construirez d'abord **à la main** (chapitre 3) — pour voir tout ce qu'un gestionnaire cache — puis avec **Maven** (chapitre 4) et enfin avec **Gradle** (chapitre 5). À la fin, vous saurez choisir, et surtout vous saurez ce qui se passe sous la couche de confort.

Le chapitre 2 reprend les **packages** de zéro (espace de noms, `import`, visibilité, erreurs classiques) : tout le reste en dépend. Les encadrés violets **ATELIER** sont des manipulations à faire immédiatement. Les encadrés rouges signalent les erreurs que tout le monde rencontre au moins une fois.

Sources principales : documentation Apache Maven (Getting Started, POM Reference, Introduction to the Build Lifecycle, Standard Directory Layout, Dependency Mechanism, Settings Reference, Maven Wrapper, Maven Archetype Plugin), documentation Gradle 9 (User Manual, Application Plugin, Build Init Plugin, Toolchains), pages de manuel Oracle des outils `javac`, `java`, `jar`, `jdeps`, `jlink`, `jpackage`, guide utilisateur JUnit 5. Toutes les commandes ont été exécutées sur Debian 13 avec Temurin 25.0.5, Maven 3.10.0 et Gradle 9.9.0 en novembre 2026.

Sommaire

1 Ce qu'est réellement un « projet Java » 4
- 1.1 Un projet = du code + des ressources + des tests + une façon de construire 4
- 1.2 Le problème que résolvent les gestionnaires : les dépendances 4
- 1.3 À la main, avec Maven, avec Gradle : qui fait quoi ? 5
- 1.4 La convention qui gouverne tout 5

2 Les packages : la notion à comprendre avant tout projet 9
- 2.1 Un package est un espace de noms, pas un dossier 9
- 2.2 La convention de nommage, et pourquoi elle est obligatoire en pratique 11
- 2.3 `import`, nom qualifié, `import static` : ce que fait vraiment le compilateur 11
- 2.4 Visibilité : le package comme frontière d'encapsulation 12
- 2.5 Package, dossier, classpath : les cinq erreurs classiques 14
- 2.6 Découper son code en packages : trois règles et un atelier 15

3 Étape 0 — Préparer le terrain et vérifier l'environnement 7

3 Parcours A — Créer le projet « à la main » (javac, java, jar) 17
- 3.1 A1 — L'arborescence, et pourquoi celle-là précisément 17
- 3.2 A2 — Écrire le code, compiler, exécuter 18
- 3.3 A3 — Les ressources : lire un fichier de configuration 20
- 3.4 A4 — Une dépendance externe, téléchargée et vérifiée à la main 21
- 3.5 A5 — Le classpath : la notion qui explique 80 % des erreurs 23
- 3.6 A6 — Les tests avec JUnit 5, sans build tool 24
- 3.7 A7 — Empaqueter : JAR, manifeste, Class-Path, Main-Class 26
- 3.8 A8 — Industrialiser à la main : un Makefile 27
- 3.9 A9 — Livrer : jdeps, jlink, jpackage 29

4 Parcours B — Créer le même projet avec Maven 31
- 4.1 B1 — Installer Maven (paquet, archive Apache, SDKMAN!) 31
- 4.2 B2 — Générer un projet : `archetype:generate`, pas à pas 34
- 4.3 B2 bis — Créer un projet Maven vide, sans archétype 36
- 4.4 B2 ter — Créer un package dans un projet Maven 38
- 4.5 B2 quater — Choisir ses coordonnées : `groupId`, `artifactId`, `version` 41
- 4.6 B3 — Le POM décortiqué, ligne par ligne 47
- 4.7 B4 — Les commandes Maven : cycle de vie, phases, goals 51
- 4.8 B5 — Dépendances, scopes, dépôt local et miroirs 53
- 4.9 B6 — Tester, empaqueter, lancer (surefire, jar, shade, exec) 56
- 4.10 B7 — Le Maven Wrapper : reproductibilité garantie 59

5 Parcours C — Le même projet avec Gradle 62
- 5.1 C1 — `gradle init` et la structure produite 62
- 5.2 C2 — build.gradle.kts expliqué, et les tâches essentielles 65
- 5.3 C3 — Toolchains : la bonne façon de cibler Java 21 68

6 Choisir : comparatif et critères de décision 70

7 Livrer et publier un projet 74

8 Dépannage : 12 erreurs réelles et leur correctif 78

9 Fiche de survie et aide-mémoire 85

10 Exercices notés (avec corrigés) 89

11 Sources et pour aller plus loin 98

Le projet fil rouge : « boutique »

Un seul projet, construit trois fois. Il est volontairement petit mais contient tout ce qui fait la différence entre un fichier `Hello.java` et un vrai projet :

Élément	Ce que fait le projet	Ce que cela vous apprend
`Prix.java`	Calcule un prix TTC (TVA 20 %) et une remise, avec arrondi au centime.	Écrire du code testable, avec des packages.
`Config.java` + `application.properties`	Lit la devise dans un fichier de configuration placé dans le classpath.	Les **ressources** : un concept que 90 % des débutants ignorent.
`App.java`	Affiche un prix calculé, en utilisant `StringUtils` de `commons-lang3`.	Les **dépendances externes** : téléchargement, classpath, packaging.
`PrixTest.java`	Trois tests JUnit 5 : TVA, remise, cas d'erreur.	Tester sans IDE, en ligne de commande.
Livrable	Un JAR exécutable par `java -jar`.	Manifeste, `Main-Class`, `Class-Path`, fat-jar.

Rien de spectaculaire, et c'est voulu. La difficulté d'un projet Java n'est presque jamais dans le code : elle est dans tout ce qui l'entoure — où mettre les fichiers, d'où viennent les dépendances, comment les rendre disponibles à l'exécution, ce qu'on met dans le livrable. Ce sont ces questions que les trois parcours traitent, sur un domaine assez simple pour qu'aucune ligne de métier ne vienne masquer le problème.

Ce que vous saurez faire à la fin	Vérifiable par	Chapitre
Compiler un ensemble de sources hétérogènes	`javac --release 21 -d out $(find …)`	3.2
Déclarer et obtenir une dépendance externe	`curl` + empreinte, ou une balise `<dependency>`	3.4 · 4.7
Expliquer un `ClassNotFoundException`	le message, lu sans hésitation	3.5 · 8
Produire un livrable exécutable	`java -jar` depuis un autre dossier	3.7 · 4.8 · 5.2
Figer la version d'un outil pour une équipe	`./mvnw -v`, `./gradlew -v`	4.9
Choisir un outil en connaissance de cause	les critères du chapitre 7	7

1 Ce qu'est réellement un « projet Java »

> Avant d'installer quoi que ce soit, il faut voir la chaîne complète : du fichier texte que vous écrivez au JAR que quelqu'un exécute. Ce chapitre n'a qu'une ambition : que plus aucune commande des chapitres suivants ne vous soit opaque.

1.1 Un projet = du code + des ressources + des tests + une façon de construire

```
src/main/java/fr/nice/shop/Prix.java          ← du TEXTE que vous écrivez
              │
              │  javac --release 21 -d out          (compilation)
              ▼
out/fr/nice/shop/Prix.class                   ← du BYTECODE (lisible par la JVM)

src/main/resources/application.properties     ← des RESSOURCES (non compilées)
              │
              │  cp -r src/main/resources/. out/    (copie telle quelle)
              ▼
out/application.properties                    ← accessible dans le classpath

src/test/java/fr/nice/shop/PrixTest.java      ← des TESTS (code compilé à part)
              │
              │  javac -cp "junit.jar:out" -d test-out
              ▼
test-out/fr/nice/shop/PrixTest.class          ← exécutés par le lanceur JUnit

TOUT CELA, PLUS LA FAÇON DE LE REFAIRE        ← c'est « le projet »
(commandes, versions, dépendances, manifeste)
```

La dernière ligne est la plus importante. Un dossier de `.java` n'est pas un projet : un projet, c'est l'ensemble **reproductible** — quelqu'un d'autre doit pouvoir obtenir le même résultat avec les mêmes commandes. C'est exactement ce que fait un gestionnaire de projet, et c'est ce que vous ferez à la main au chapitre 3.

Élément	Rôle	Sans gestionnaire, qui s'en occupe ?
Code source	Le programme lui-même	`javac`
Ressources	Configuration, images, messages — lues depuis le classpath, pas depuis le disque	une commande `cp`
Bibliothèques tierces	Code écrit par d'autres (commons-lang3, Jackson, JUnit…)	téléchargement manuel + `-cp`
Tests	Preuves que le code fait ce qu'il prétend	lanceur JUnit + `-cp`
Empaquetage	Livrer un seul fichier exécutable	`jar` + un manifeste écrit à la main
Automatisation	Refaire tout cela en une commande, pour tous	un `Makefile`, ou l'outil (Maven/Gradle)

1.2 Le problème que résolvent les gestionnaires : les dépendances

Une application réelle n'utilise pas seulement la bibliothèque standard : elle s'appuie sur des bibliothèques externes. Or une bibliothèque en appelle d'autres, qui en appellent d'autres encore. Ce phénomène s'appelle les **dépendances transitives**.

```
votre code
   └── commons-lang3 3.20.0
   └── junit-jupiter 5.12.2
          ├── junit-jupiter-api 5.12.2
          │      ├── opentest4j 1.3.0
          │      ├── junit-platform-commons 1.12.2
          │      └── apiguardian-api 1.1.2
          └── junit-jupiter-params 5.12.2

À la main   : il faut trouver, télécharger, vérifier et ordonner TOUS ces JAR.
Avec Maven  : vous déclarez 2 dépendances, l'outil résout les 9 autres.
```

Ce qu'un gestionnaire de projet fait vraiment

Un gestionnaire ne « compile pas mieux » que `javac` : c'est le même `javac` qui est appelé au final. Il automatise quatre choses pénibles : **(1)** télécharger et vérifier les dépendances, y compris transitives ; **(2)** construire le classpath exact ; **(3)** enchaîner les étapes dans le bon ordre (nettoyer, compiler, tester, empaqueter) ; **(4)** rendre le tout reproductible sur une autre machine. Tout le reste — plugins, cycles de vie, dépôts — n'est que du confort autour de ces quatre idées.

1.3 À la main, avec Maven, avec Gradle : qui fait quoi ?

Tâche	À la main	Maven	Gradle
Créer l'arborescence	`mkdir -p src/main/java/…`	`mvn archetype:generate`	`gradle init`
Décrire le projet	rien (ou un `Makefile`)	`pom.xml`	`build.gradle.kts`
Télécharger une dépendance	`curl` + `sha1sum`	déclarée dans le POM, résolue automatiquement	déclarée dans le build, résolue automatiquement
Compiler	`javac -cp …`	`mvn compile`	`./gradlew compileJava`
Tester	`java -jar junit-console…`	`mvn test`	`./gradlew test`
Empaqueter	`jar --create --manifest…`	`mvn package`	`./gradlew build`
Installer Maven/ Gradle	—	paquet, archive, SDKMAN!	wrapper fourni par le projet
Rejouer le build ailleurs	copier le `Makefile`	`./mvnw`	`./gradlew`

1.4 La convention qui gouverne tout

Maven a imposé en 2004 une disposition des dossiers devenue le standard de fait de tout l'écosystème Java — Gradle la reprend par défaut, et les IDE aussi. La connaître par cœur évite la moitié des erreurs de débutant :

```
mon-projet/
├── pom.xml  (ou build.gradle.kts)     ← la description du projet
└── src/
    ├── main/
    │   ├── java/fr/nice/shop/App.java ← le CODE (chemin = package)
    │   └── resources/                 ← les RESSOURCES (copiées telles quelles)
    └── test/
        ├── java/fr/nice/shop/AppTest.java   ← les TESTS
        └── resources/                       ← ressources réservées aux tests

produits par le build (jamais versionnés) :
target/   (Maven)      build/   (Gradle)      out/  (à la main)
```

Les trois erreurs de structure les plus fréquentes

1. **Le chemin ne reflète pas le package.** Un fichier `App.java` déclarant `package fr.nice.shop;` doit être dans `…/fr/nice/shop/`. Maven échoue avec `package … does not correspond to the file path` ; à la main, `javac` compile quand même, mais vous ne retrouverez plus rien.
2. **Un fichier `.java` à la racine du projet.** Il ne sera pas compilé (Maven ne regarde que `src/main/java`) et vous chercherez l'erreur pendant une heure.
3. **Les ressources placées dans `java/`.** Elles seraient traitées comme du code. Elles vont dans `src/main/resources`, à la racine du classpath.

2 Étape 0 — Préparer le terrain et vérifier l'environnement

> Trois minutes de vérification évitent trois heures de faux problèmes. On contrôle le JDK, les outils du JDK, et la présence éventuelle d'un Maven ou d'un Gradle déjà installé.

2.1 Le JDK, et les outils qu'il contient

COMMANDES — VÉRIFICATION DU JDK

```
java -version
javac -version
java -XshowSettings:properties -version 2>&1 | grep java.home

# La caisse à outils fournie par le JDK (compilateur, archiveur, dépendances, runtime)
for t in javac jar javadoc jshell jdeps jlink jpackage; do
  printf '%-10s %s\n' "$t" "$(command -v $t || echo ABSENT)"
done
```

SORTIE RÉELLE (TEMURIN 25.0.5 INSTALLÉ DANS /OPT)

```
openjdk version "25.0.5" 2026-10-20 LTS
javac 25.0.5
    java.home = /opt/java/jdk-25.0.5+8
javac      /opt/java/jdk-25.0.5+8/bin/javac
jar        /opt/java/jdk-25.0.5+8/bin/jar
javadoc    /opt/java/jdk-25.0.5+8/bin/javadoc
jshell     /opt/java/jdk-25.0.5+8/bin/jshell
jdeps      /opt/java/jdk-25.0.5+8/bin/jdeps
jlink      /opt/java/jdk-25.0.5+8/bin/jlink
jpackage   /opt/java/jdk-25.0.5+8/bin/jpackage
```

Chacun de ces outils interviendra dans la leçon : `javac` compile, `jar` empaquette, `jshell` teste une idée en trois secondes, `jdeps` analyse les dépendances, `jlink` fabrique un runtime minimal, `jpackage` produit un installateur natif. Aucun n'a besoin de Maven.

2.2 Maven ou Gradle sont-ils déjà là ?

COMMANDES

```
command -v mvn gradle; mvn -v 2>/dev/null | head -3; gradle --version 2>/dev/null | head -5

# Paquets éventuellement installés
dpkg -l | awk '/^ii/ && $2 ~ /^(maven|gradle)/ {print $2, $3}'

# Et du côté des gestionnaires de versions (si vous les utilisez)
ls -d ~/.sdkman/candidates/maven/current ~/.sdkman/candidates/gradle/current 2>/dev/null
```

SORTIE RÉELLE

```
Apache Maven 3.10.0 (c43a36b8d67be7e0805a411bc0898af1a51f5472)
Maven home: /opt/maven/current
Java version: 25.0.5, vendor: Eclipse Adoptium, runtime: /opt/java/jdk-25.0.5+8
```

Le piège du Gradle de la distribution

Sur Debian 13, le paquet `gradle` proposé par `apt` est en version **4.4.1** (sortie en 2017), alors que la version actuelle est 9.9.0. Cette version ne connaît ni Kotlin DSL moderne, ni `gradle init --type java-application`, ni les toolchains récentes. Autrement dit : inutilisable pour suivre cette leçon. **Pour Gradle, on passe par le wrapper** (fourni par le projet) ou par SDKMAN!. Pour Maven, le paquet Debian (3.9.9) est en revanche parfaitement utilisable.

2.3 Les outils annexes : `curl`, `make`, `unzip`

COMMANDES

```
# curl : télécharger ; make : automatiser à la main ; unzip/zip : archives
sudo apt update
sudo apt install -y curl make zip unzip ca-certificates

# Contrôle
curl --version | head -1; make --version | head -1
```

Faut-il un IDE ?

Non. Tout ce qui suit se fait au terminal, et c'est volontaire : un IDE masque exactement ce que cette leçon veut montrer. Une fois les trois parcours faits, ouvrez le même projet dans IntelliJ IDEA ou VS Code : vous reconnaîtrez le `pom.xml`, le `build.gradle.kts`, le dossier `target/`, et vous saurez pourquoi l'IDE vous demande un « SDK » et un « JVM for Gradle ».

ATELIER 0 — 5 minutes

Objectif : savoir ce que votre machine sait déjà faire.

1. Lancez les commandes de 3.1 et notez la version de votre JDK.
2. Vérifiez la présence de `mvn` et `gradle` (3.2). Si aucun n'est présent, c'est l'état normal : le chapitre 3 ne les utilise pas.
3. Installez `make` et `curl` si nécessaire, puis répondez : pourquoi *`make`* n'est-il pas un gestionnaire de projet Java ? (Indice : que sait-il faire des dépendances ?)

2 Les packages : la notion à comprendre avant tout projet

> Un projet Java commence toujours par une ligne que les débutants recopient sans la lire : `package fr.acme.boutique;`. Cette ligne décide de l'identité de vos classes, de leur visibilité, de l'emplacement des fichiers compilés et de la moitié des messages d'erreur que vous rencontrerez. On l'explique donc ici, avant d'installer quoi que ce soit : les chapitres suivants supposeront que c'est acquis.

La définition en une phrase

Un **package** (en français : un paquet, mais tout le monde dit « package ») est un **espace de noms** qui regroupe des classes apparentées et définit entre elles une **frontière de visibilité**. Le nom complet d'une classe — son nom qualifié — est la concaténation du package et du nom simple : `fr.acme.boutique.model.Produit`. Deux classes peuvent porter le même nom simple si elles sont dans deux packages différents ; c'est même tout l'intérêt.

2.1 Un package est un espace de noms, pas un dossier

La confusion la plus répandue consiste à croire qu'un package est un dossier. En réalité : un package est une notion du **langage** (elle existe dans le fichier `.java`, elle est inscrite dans le fichier `.class`), et le dossier n'est qu'une **convention de rangement** que les outils utilisent pour retrouver les fichiers. Les deux vont presque toujours ensemble, mais ils sont indépendants — et c'est précisément ce qui produit les erreurs de la section 2.5.

PROGRAMME — LE NOM QUALIFIÉ, ÉCRIT UNE FOIS POUR TOUTES

```
// fichier : src/main/java/fr/acme/boutique/model/Produit.java
package fr.acme.boutique.model;          // 1re instruction du fichier, avant tout import

public record Produit(String nom, long prixCentimes) {
    public long prixTTC(int tauxPourCent) {
        return Math.round(prixCentimes * (1 + tauxPourCent / 100.0));
    }
}
```

```
// fichier : src/main/java/fr/acme/boutique/App.java
package fr.acme.boutique;

import fr.acme.boutique.model.Produit;    // pour pouvoir écrire « Produit »

public class App {
    public static void main(String[] args) {
        Produit cafe = new Produit("Café", 450);
        System.out.println(cafe.nom() + " TTC : " + cafe.prixTTC(20) + " centimes");
        System.out.println(App.class.getPackageName());     // le package, à l'exécution
        System.out.println(Produit.class.getPackageName());
    }
}
```

COMPILATION ET EXÉCUTION RÉELLES

```
$ javac -d out -sourcepath src src/fr/acme/boutique/App.java

$ find out -name '*.class'
out/fr/acme/boutique/App.class
out/fr/acme/boutique/model/Produit.class

$ java -cp out fr.acme.boutique.App
Café TTC : 540 centimes
fr.acme.boutique
fr.acme.boutique.model
```

Trois observations, toutes vérifiées ci-dessus. **Un** : `javac` a rangé `Produit.class` dans `out/fr/acme/boutique/model/` — le chemin de sortie reproduit le package, il ne le décide pas ; et il a compilé `Produit` au passage, sans qu'on le nomme, grâce à `-sourcepath`. **Deux** : à l'exécution, une classe connaît son package (`getPackageName()`), et c'est cette information qui est écrite dans le `.class`. **Trois** : la classe s'exécute par son **nom qualifié** (`fr.acme.boutique.App`), jamais par son nom simple. Retenez cette dernière règle : elle explique à elle seule le message `Could not find or load main class App` que tout le monde a vu au moins une fois.

> **Ce que « le package est dans le fichier .class » implique**
>
> Renommer le dossier d'un fichier compilé ne change pas son package : la classe `fr.acme.boutique.App` placée dans un dossier `autre/` reste `fr.acme.boutique.App` et devient simplement introuvable. Inversement, un fichier source dont le `package` ne correspond pas à son dossier se compile quand même (section 2.5) — les outils le tolèrent plus ou moins bien, mais le résultat est toujours une source de confusion. Une seule règle simple : **le package, le dossier et le chemin de sortie avancent ensemble**.

2.2 La convention de nommage, et pourquoi elle est obligatoire en pratique

Le langage impose peu de choses : un nom de package est une suite d'identifiants séparés par des points (`fr.acme.boutique.model`), chaque identifiant obéissant aux règles des identifiants Java — lettres, chiffres, `_`, `$`, sans commencer par un chiffre, sans trait d'union, et sans être un mot-clé. La **convention**, elle, est universelle et non négociable :

Règle	Exemple correct	Ce qui casse sinon
Nom de domaine inversé en préfixe	`fr.acme.boutique`, `com.exemple.projet`	Sans préfixe (`boutique`, `model`), votre package entre en collision avec celui d'une bibliothèque tierce.
Tout en minuscules, pas de camelCase	`fr.acme.boutique.model`	`fr.acme.Boutique.Model` compile mais se distingue mal des classes ; les outils de lint le refusent.
Uniquement des identifiants Java	`fr.acme.boutique_v2`	`fr.acme.boutique-v2` → `error: illegal character: '-'` ; `fr.acme.new` → `new` est un mot-clé.
Un package = un propriétaire	toujours le même préfixe dans un projet	Deux préfixes dans le même livrable (`fr.acme` et `org.autre`) : impossible de savoir ce qui vous appartient.
Stable dans le temps	renommer un package = casser l'API	Le nom qualifié est un identifiant public : un client qui écrit `import fr.acme.boutique.Prix;` casse si vous renommez.

> **Le test des trois secondes**
>
> Un bon nom de package répond à la question « de quoi parlent ces classes ? » sans ouvrir le code. `fr.acme.boutique.model`, `fr.acme.boutique.service`, `fr.acme.boutique.persistance` répondent. `fr.acme.boutique.utils`, `fr.acme.boutique.common`, `fr.acme.boutique.misc` ne répondent à rien : ce sont des fourre-tout, et ils grossissent toujours jusqu'à devenir ingérables.

2.3 `import`, nom qualifié et `import static` : ce que fait vraiment le compilateur

Un `import` n'inclut pas un fichier, ne copie pas de code et n'a aucun effet à l'exécution : il ne sert qu'à **permettre d'écrire un nom court** à la place d'un nom qualifié. Le compilateur résout ensuite chaque nom simple vers un nom qualifié, et c'est celui-ci qui est inscrit dans le `.class`. Conséquence utile : un `import` inutilisé ne coûte rien, tandis qu'un `import` manquant coûte une erreur de compilation.

LES QUATRE FORMES D'IMPORT, SUR LE MÊME EXEMPLE

```
// 1. import explicite d'une classe (à préférer)
import fr.acme.boutique.model.Produit;

// 2. import à la demande : toutes les classes du package, mais un seul niveau
import fr.acme.boutique.model.*;

// 3. import statique : un membre statique utilisable sans son préfixe
import static java.lang.Math.round;      // puis : long x = round(4.6);

// 4. aucun import : le nom qualifié complet, à écrire à chaque fois
var p = new fr.acme.boutique.model.Produit("Café", 450);
```

Ce que fait le compilateur	Conséquence pratique
`import` = alias de nom	Aucun impact sur la taille du JAR, aucun impact sur les performances. Seul le nom qualifié existe dans le bytecode.
`java.lang.*` est importé d'office	`String`, `Math`, `System`, `Integer`… s'écrivent sans import ; tout le reste doit être importé.
`import p.*` ne couvre qu'**un** niveau	`import fr.acme.boutique.*` ne donne pas accès à `fr.acme.boutique.model.Produit` : il faut l'importer, ou écrire le nom qualifié.
Deux imports de même nom simple → ambiguïté	Le compilateur refuse ; il faut un nom qualifié pour l'un des deux usages (erreur exacte ci-dessous).
Le package par défaut ne s'importe pas	Une classe sans `package` ne peut pas être utilisée depuis un package nommé (erreur exacte ci-dessous).

LES DEUX ERREURS D'IMPORT, REPRODUITES

```
// A. deux classes Date de même nom simple
import java.util.*;
import java.sql.*;

public class D {
    public static void main(String[] x) { Date d = new Date(0); System.out.println(d); }
}
```

```
D.java:5: error: reference to Date is ambiguous
        public static void main(String[] x) { Date d = new Date(0); System.out.println(d); }
                                               ^
  both class java.sql.Date in java.sql and class java.util.Date in java.util match
1 error
```

```
// B. tenter d'importer une classe du package par défaut
import SansPaquet;

public class E {
    public static void main(String[] x) { System.out.println(SansPaquet.x); }
}
```

```
dp/E.java:1: error: '.' expected
import SansPaquet;
                ^
1 error
```

Le correctif du cas A est le nom qualifié, écrit à l'endroit de l'usage — pas un second import : `java.util.Date d = new java.util.Date(0);`. Celui du cas B est plus radical : donnez un package à la classe concernée. Une classe du package par défaut est, en pratique, inutilisable dès qu'elle doit dialoguer avec autre chose qu'elle-même.

2.4 Visibilité : le package comme frontière d'encapsulation

C'est la fonction la plus utile — et la moins connue — des packages : ils constituent une **quatrième** niveau de visibilité, celui qu'on obtient en n'écrivant aucun modificateur. Un membre sans modificateur est package-private : accessible à toutes les classes du même package, et à elles seules.

Modificateur	Accessible depuis	Usage typique
`public`	partout	L'API du package : ce que vous vous engagez à maintenir.
`protected`	le package + les sous-classes	Points d'extension pour l'héritage (rares sur une API applicative).
(aucun) — package-private	le package uniquement	Les classes et méthodes « de service » internes : le vrai outil d'encapsulation en Java.
`private`	la classe uniquement	L'état interne. Un membre privé n'est pas accessible depuis une autre classe du même package.

PROGRAMME — LA FRONTIÈRE, DÉMONTRÉE PAR UNE ERREUR DE COMPILATION

```
// fichier : src/fr/acme/util/Texte.java
package fr.acme.util;

public class Texte {
    static final String PREFIXE = "[boutique] ";        // package-private

    public static String nettoie(String s) {
        return PREFIXE + s.strip().toLowerCase();
    }

    static String interne() {                            // package-private
        return "visible seulement dans fr.acme.util";
    }
}
```

```
// fichier : src/fr/acme/boutique/Essai.java — autre package
package fr.acme.boutique;

public class Essai {
    public static void main(String[] a) {
        System.out.println(fr.acme.util.Texte.interne());
    }
}
```

ERREUR RÉELLE

```
src/fr/acme/boutique/Essai.java:5: error: interne() is not public in Texte; cannot be accessed from
outside package
        System.out.println(fr.acme.util.Texte.interne());
                                             ^
1 error
```

Retenez la formulation : « is not public in … ; cannot be accessed from outside package ». Derrière cette phrase se cache une technique de conception très utile : une classe `public` peut exposer une méthode `public` qui s'appuie sur trois classes package-private — invisibles du reste du monde, libres de changer à tout moment. C'est ainsi qu'on garde une API petite et un intérieur remaniable.

> **La règle qui découle de tout ça**
>
> Par défaut, écrivez **le moins visible possible** : pas de modificateur pour tout ce qui est interne au package, `private` pour les champs, `public` seulement pour ce que d'autres packages doivent appeler. Vous pourrez toujours élargir une visibilité plus tard ; la réduire casse du code.

2.5 paquet, dossier, classpath : les cinq erreurs classiques

Les cinq erreurs suivantes représentent l'essentiel des difficultés de « démarrage » en Java. Elles ont toutes la même origine : une incohérence entre le nom déclaré, le chemin du fichier et le chemin de recherche.

Erreur 1 — Le package ne correspond pas au dossier

REPRODUCTION : LA CLASSE OUTIL DÉCLARE FR.ACME.BOUTIQUE.SERVICES MAIS VIT DANS FR/ACME/BOUTIQUE/

```
mauvais/fr/acme/boutique/Outil.java    // package fr.acme.boutique.services;
$ javac -d mauvais/out mauvais/fr/acme/boutique/Outil.java
$ find mauvais/out -name '*.class'
mauvais/out/fr/acme/boutique/services/Outil.class
$ java -cp mauvais/out fr.acme.boutique.services.Outil
outil
```

Le fichier compile et s'exécute, certes — mais le `.class` est écrit dans un dossier qui ne correspond pas à l'emplacement du `.java`. Vous venez de créer un projet où l'emplacement d'un fichier ne dit plus rien de son identité. Les IDE et les gestionnaires de build, eux, se fient à l'un ou à l'autre et finissent tôt ou tard par ne plus trouver la classe. **Correctif** : déplacer le fichier dans un dossier `…/fr/acme/boutique/services/`, ou aligner le `package` sur le dossier — mais jamais les deux à moitié.

Erreur 2 — « Could not find or load main class »

REPRODUCTION : LANCER LA CLASSE PAR SON NOM SIMPLE

```
$ java -cp out fr.acme.boutique.App
[boutique] café
[boutique] thé

$ cd out && java fr.acme.boutique.App      # fonctionne aussi
$ java -cp out App                          # nom simple : échoue
```

```
Error: Could not find or load main class App
Caused by: java.lang.ClassNotFoundException: App
```

Le message dit exactement ce qui se passe : la machine virtuelle cherche une classe nommée `App` — c'est-à-dire `App.class` à la racine du classpath. Or le fichier s'appelle `out/fr/acme/boutique/App.class`, donc la classe s'appelle `fr.acme.boutique.App`. **Correctif** : toujours le nom qualifié. C'est aussi la cause de 90 % des `ClassNotFoundException` en production.

Erreur 3 — Deux classes de même nom dans le package par défaut

REPRODUCTION : DEUX FICHIERS PRIX.JAVA SANS PACKAGE, COMPILÉS ENSEMBLE

```
a/Prix.java    // public class Prix { … }  (aucun package)
b/Prix.java    // public class Prix { … }  (aucun package)
$ javac -d out2 a/Prix.java b/Prix.java
```

```
b/Prix.java:1: error: duplicate class: Prix
public class Prix {
       ^
1 error
```

Sans package, toutes les classes du monde partagent le même espace de noms : deux `Prix` ne peuvent pas coexister. Pire, compilées séparément dans le même dossier de sortie, la seconde écrase silencieusement la première (le programme affiche alors « Prix de b » alors qu'on croyait lancer celui de `a`). **Correctif** : un package par projet, systématiquement.

Erreur 4 — Un `import` qui ne trouve rien parce que le classpath est faux

Si le package existe mais que le compilateur répond `package fr.acme.boutique.services does not exist`, le problème n'est ni le nom ni la casse : c'est le **classpath**, c'est-à-dire la liste des endroits où l'on cherche les classes. Deux classes se trouvant dans deux arbres de répertoires différents exigent deux entrées de classpath, ou une compilation en une seule passe depuis la racine des sources :

LES DEUX FORMES CORRECTES

```
# 1. tout compiler d'un coup, depuis la racine des sources
javac -d out $(find src -name '*.java')

# 2. compiler par étapes, en pointant le classpath sur la sortie précédente
javac -d out -cp out src/fr/acme/boutique/App.java
```

Erreur 5 — Croire que `import p.*` suffit pour les sous-packages

`import fr.acme.boutique.*;` n'importe pas `fr.acme.boutique.model.Produit` : les packages ne sont pas hiérarchiques du point de vue du langage, `fr.acme.boutique.model` n'est pas « contenu dans » `fr.acme.boutique`, il commence juste par les mêmes caractères. Un import à la demande ne couvre qu'un seul niveau.

2.6 Découper son code en packages : trois règles et un atelier

Il n'existe pas de découpage « officiel », mais trois règles évitent 95 % des mauvais packages.

Règle	Pourquoi
Par fonctionnalité, pas par couche technique	`fr.acme.boutique.facturation` (qui contient `Facture`, `LigneFacture`, `ServiceFacturation`) vaut mieux que trois packages `model`, `service`, `dao` où chaque modification d'une fonctionnalité oblige à toucher trois dossiers. Sur un petit projet, l'inverse (par couche) est acceptable ; sur un gros, il devient un piège.
Pas de dépendances circulaires	Si `a` utilise une classe de `b` et `b` une classe de `a`, les deux ne forment qu'un seul package déguisé : fusionnez-les, ou extrayez l'élément commun. Les cycles se détectent mécaniquement (`jdeps`, ArchUnit).
Un package = une phrase	Si vous ne pouvez pas résumer son rôle en une phrase sans « et », il contient deux choses. Corollaire : un package nommé `utils` n'a pas de phrase, donc pas de raison d'exister.

Atelier — 15 min objectif : créer deux packages propres et vérifier la visibilité

☐ Dans le projet de la section 2.1, créez `src/main/java/fr/acme/boutique/service/` et écrivez-y `Caisse.java` (package `fr.acme.boutique.service`) qui expose `public long total()` et utilise `Produit`.

☐ Ajoutez dans `Caisse` une méthode sans modificateur, `long remiseInterne()`. Depuis `App` (autre package), essayez de l'appeler : notez le message exact du compilateur.

☐ Compilez tout en une passe : `javac -d out $(find src/main/java -name '*.java')`, puis exécutez par le nom qualifié `fr.acme.boutique.App`.

☐ Vérifiez l'arborescence produite dans `out` : elle doit reproduire exactement celle des sources, à partir du premier segment du package.

Ce que vous devez avoir compris à la fin : le dossier est une convention, le package est une identité ; le nom qualifié est la seule façon sûre de désigner une classe ; et la visibilité package-private est l'outil normal pour cacher l'intérieur d'un package.

Et dans Maven ? Rien de tout cela ne change : il ajoute une **structure imposée** (`src/main/java`, `src/test/java`) et une racine de sources. Un package `fr.acme.boutique.service` correspond alors au dossier `src/main/java/fr/acme/boutique/service/`, et les classes compilées atterrissent dans `target/classes/fr/acme/boutique/service/`. La création d'un projet vide et celle d'un package dans un projet Maven sont traitées au chapitre 4 (sections 4.3 et 4.4), avec les commandes exactes et leurs sorties réelles.

3 Parcours A — Créer le projet « à la main » (javac, java, jar)

> Neuf étapes, aucune dépendance à un outil de build. À la fin, vous aurez un projet complet : code, ressources, bibliothèque externe vérifiée, tests qui tournent, JAR exécutable et Makefile. Et vous saurez ce que Maven fera à votre place — donc ce que vous pourrez diagnostiquer quand il se taira ou mentira.

Étape	Ce qu'on fait	L'équivalent chez Maven
A1	Créer l'arborescence	`mvn archetype:generate`
A2	Écrire, compiler, exécuter	`mvn compile` + `exec:java`
A3	Ajouter des ressources	`src/main/resources` + plugin resources
A4	Télécharger et vérifier une dépendance	une balise `<dependency>`
A5	Maîtriser le classpath	résolution automatique
A6	Tester avec JUnit 5	`mvn test` (surefire)
A7	Empaqueter un JAR exécutable	`mvn package` (jar + shade)
A8	Automatiser avec un Makefile	le cycle de vie de Maven
A9	Livrer (runtime minimal, installateur)	`jlink` / `jpackage`

3.1 A1 — L'arborescence, et pourquoi celle-là précisément

COMMANDES

```
mkdir -p ~/projets/boutique/{lib,src/main/java/fr/nice/shop,src/main/resources,src/test/java/fr/nice/shop}
cd ~/projets/boutique
find . -type d | sort
```

SORTIE RÉELLE — LA CONVENTION MAVEN, CRÉÉE À LA MAIN

```
.
./lib
./src
./src/main
./src/main/java
./src/main/java/fr/nice
./src/main/java/fr/nice/shop
./src/main/resources
./src/test
./src/test/java
./src/test/java/fr/nice
./src/test/java/fr/nice/shop
```

Pourquoi `fr/nice/shop` et non un dossier `shop` ? Parce que le **package** Java (`package fr.nice.shop;`) doit correspondre à l'arborescence. C'est une convention que tous les outils — `javac`, Maven, Gradle, les IDE — supposent. La respecter dès le premier jour vous évitera, plus tard, des erreurs que vous ne comprendrez pas.

Comment choisir un `groupId` et un package

Le `groupId` est l'identité de l'organisation, en général le nom de domaine inversé : `fr.nice.shop`, `com.acme.billing`, `io.github.monpseudo`. Le package reprend le groupId et se termine par le nom du module. Évitez `com.example` (réservé aux exemples), les accents, les tirets, et les majuscules : les noms de packages sont en minuscules, sans exception.

3.2 A2 — Écrire le code, compiler, exécuter

COMMANDE — CRÉATION DU FICHIER SOURCE (HEREDOC, POUR ÉVITER TOUT PROBLÈME D'ENCODAGE)

```
cat > src/main/java/fr/nice/shop/Prix.java <<'EOF'
package fr.nice.shop;

import java.math.BigDecimal;
import java.math.RoundingMode;

/** Calculs de prix : TVA et remise. */
public final class Prix {

    private static final BigDecimal TAUX_TVA = new BigDecimal("0.20");

    private Prix() { }

    /** Prix TTC à partir d'un prix HT, arrondi au centime. */
    public static BigDecimal ttc(BigDecimal ht) {
        return ht.multiply(BigDecimal.ONE.add(TAUX_TVA)).setScale(2, RoundingMode.HALF_UP);
    }

    /** Applique une remise en pourcentage (0 à 100). */
    public static BigDecimal remise(BigDecimal prix, int pourcentage) {
        if (pourcentage < 0 || pourcentage > 100) {
            throw new IllegalArgumentException("Pourcentage hors bornes : " + pourcentage);
        }
        BigDecimal facteur = BigDecimal.ONE.subtract(
                BigDecimal.valueOf(pourcentage).movePointLeft(2));
        return prix.multiply(facteur).setScale(2, RoundingMode.HALF_UP);
    }
}
EOF
```

COMMANDES — COMPILER PUIS EXÉCUTER (VERSION MINIMALE, SANS DÉPENDANCE)

```
# Compiler : -d indique le dossier de sortie ; l'arborescence des packages y est recréée
javac -d out src/main/java/fr/nice/shop/Prix.java
find out -name '*.class'

# Tester la classe immédiatement, sans écrire de main (REPL jshell)
jshell --class-path out <<'EOF2'
import fr.nice.shop.Prix;
import java.math.BigDecimal;
System.out.println(Prix.ttc(new BigDecimal("49.90")));
System.out.println(Prix.remise(new BigDecimal("59.88"), 10));
/exit
EOF2
```

SORTIE RÉELLE

```
out/fr/nice/shop/Prix.class
|  Welcome to JShell -- Version 25.0.5
|  For an introduction type: /help intro

jshell> 59.88

jshell> 53.89

jshell> |  Goodbye
```

Deux options de `javac` à connaître absolument

`-d dossier` : où écrire les `.class`. Sans `-d`, ils sont écrits à côté du `.java` — ça marche, mais le code source est pollué et le nettoyage devient impossible.

`--release N` : compile pour la version `N`, en vérifiant aussi que vous n'utilisez pas d'API plus récentes. C'est l'option que tout projet sérieux utilise (voir la leçon « Installer Java » pour la démonstration complète). À défaut, `-encoding UTF-8` est indispensable dès que vos sources contiennent des accents.

Le point d'entrée : une classe avec `main`

COMMANDE

```
cat > src/main/java/fr/nice/shop/App.java <<'EOF'
package fr.nice.shop;

import java.math.BigDecimal;

public class App {
    public static void main(String[] args) {
        String nom = args.length > 0 ? args[0] : "client";
        System.out.println(nom + " : " + Prix.ttc(new BigDecimal("49.90")) + " EUR TTC");
    }
}
EOF

javac -d out $(find src/main/java -name '*.java')
java -cp out fr.nice.shop.App Alice
```

SORTIE RÉELLE

```
Alice : 59.88 EUR TTC
```

Notez le `$(find … -name '*.java')` : il liste tous les fichiers source. C'est le rôle que joue Maven en permanence — sauf que Maven le fait sans que vous ayez à y penser, et qu'il ne recompile que ce qui a changé.

3.3 A3 — Les ressources : lire un fichier de configuration

Une **ressource** est un fichier non compilé (propriétés, XML, JSON, images) que l'application lit depuis le classpath, c'est-à-dire depuis l'intérieur du JAR une fois livré. C'est un concept que beaucoup de débutants découvrent trop tard, quand leur application fonctionne en local et échoue en production parce qu'elle cherchait un fichier sur le disque.

COMMANDES

```
cat > src/main/resources/application.properties <<'EOF'
# Paramètres de l'application (lus depuis le classpath)
tva.taux=0.20
app.devise=EUR
EOF

cat > src/main/java/fr/nice/shop/Config.java <<'EOF'
package fr.nice.shop;

import java.io.IOException;
import java.io.InputStream;
import java.util.Properties;

/** Charge application.properties depuis le classpath. */
public final class Config {

    private static final Properties PROPS = new Properties();

    static {
        try (InputStream in = Config.class.getResourceAsStream("/application.properties")) {
            if (in == null) {
                throw new IllegalStateException("application.properties absent du classpath");
            }
            PROPS.load(in);
        } catch (IOException e) {
            throw new IllegalStateException("Lecture impossible", e);
        }
    }

    private Config() { }

    public static String devise() {
        return PROPS.getProperty("app.devise", "EUR");
    }
}
EOF
```

COMMANDES — COMPILER, PUIS COPIER LES RESSOURCES DANS `OUT/`

```
# Compiler…
javac -d out $(find src/main/java -name '*.java')
# …puis COPIER les ressources dans le dossier de sortie (l'étape que tout le monde oublie)
cp -r src/main/resources/. out/
find out -type f | sort
```

SORTIE RÉELLE

```
out/application.properties
out/fr/nice/shop/App.class
out/fr/nice/shop/Config.class
out/fr/nice/shop/Prix.class
```

Le `/` initial de `getResourceAsStream("/…")`

Avec une barre oblique au début, le chemin est absolu depuis la racine du classpath : `application.properties`. Sans barre, il est relatif au package de la classe appelante : `Config.class.getResourceAsStream("application.properties")` chercherait `fr/nice/shop/application.properties`. C'est une source d'erreur classique, et le message est muet : la méthode renvoie `null`, sans exception.

3.4 A4 — Une dépendance externe, téléchargée et vérifiée à la main

Notre application va utiliser `StringUtils` de la bibliothèque `commons-lang3`. Sans gestionnaire, il faut aller la chercher sur **Maven Central**, le dépôt public, dont l'URL suit une convention stricte.

```
   https://repo1.maven.org/maven2/ + groupId (en chemins) + artifactId + version + artifactId-
version.jar

   org.apache.commons : commons-lang3 : 3.20.0
        │
        └── https://repo1.maven.org/maven2/org/apache/commons/commons-lang3/3.20.0/commons-
lang3-3.20.0.jar

   À côté du JAR, le dépôt publie des fichiers de contrôle :
      …jar.sha1     empreinte SHA-1 (celle qu'utilise Maven)
      …jar.sha256   empreinte SHA-256
      …jar.md5      empreinte MD5 (obsolète, ne pas s'en contenter)
      …pom          la description de la bibliothèque et de SES dépendances
```

COMMANDES — TÉLÉCHARGEMENT ET VÉRIFICATION D'INTÉGRITÉ

```
cd ~/projets/boutique
BASE=https://repo1.maven.org/maven2

# 1. Télécharger le JAR…
curl -fsSL -o lib/commons-lang3-3.20.0.jar \
  $BASE/org/apache/commons/commons-lang3/3.20.0/commons-lang3-3.20.0.jar

# 2. …et l'empreinte publiée par le dépôt
curl -fsSL -o /tmp/commons.sha1 \
  $BASE/org/apache/commons/commons-lang3/3.20.0/commons-lang3-3.20.0.jar.sha1

# 3. Comparer
ATTENDU=$(cat /tmp/commons.sha1)
OBTENU=$(sha1sum lib/commons-lang3-3.20.0.jar | awk '{print $1}')
echo "attendu : $ATTENDU"
echo "obtenu  : $OBTENU"
[ "$ATTENDU" = "$OBTENU" ] && echo "OK : fichier intègre" || echo "KO : ne pas utiliser"
```

SORTIE RÉELLE

```
attendu : 65897b3e5731220962e659e001904af3c3cbeba9
obtenu  : 65897b3e5731220962e659e001904af3c3cbeba9
OK : fichier intègre
```

Ce que Maven ferait ici, et qu'il faut savoir quand même

Maven télécharge ce JAR dans son **dépôt local** (`~/.m2/repository`), vérifie l'empreinte, lit le fichier `.pom` de la bibliothèque pour découvrir ses dépendances, et recommence récursivement. C'est cette récursion qui justifie l'outil : `commons-lang3` n'a ici aucune dépendance, mais JUnit en a quatre.

Vous pouvez d'ailleurs inspecter ce que Maven a téléchargé : le dépôt local est une arborescence lisible à la main, exactement selon la même convention `groupId/artifactId/version/`.

COMMANDES — UTILISER LA DÉPENDANCE

```
cat > src/main/java/fr/nice/shop/App.java <<'EOF'
package fr.nice.shop;

import java.math.BigDecimal;
import org.apache.commons.lang3.StringUtils;

public class App {
    public static void main(String[] args) {
        String nom = args.length > 0 ? args[0] : "client";
        BigDecimal prixTtc = Prix.ttc(new BigDecimal("49.90"));
        BigDecimal remise = Prix.remise(prixTtc, 10);
        System.out.println(StringUtils.capitalize(nom) + " : " + prixTtc + " " + Config.devise() +
" TTC");
        System.out.println("Après remise de 10 % : " + remise + " " + Config.devise());
    }
}
EOF

# Compiler AVEC le JAR de la dépendance sur le classpath (-cp ou -classpath)
javac --release 21 -Xlint:all -cp "lib/commons-lang3-3.20.0.jar" -d out $(find src/main/java -name
'*.java')
cp -r src/main/resources/. out/

# Exécuter : le dossier de classes ET le JAR de la dépendance
LANG=C.UTF-8 java -cp "out:lib/commons-lang3-3.20.0.jar" fr.nice.shop.App Alice
```

SORTIE RÉELLE

```
Alice : 59.88 EUR TTC
Après remise de 10 % : 53.89 EUR
```

3.5 A5 — Le classpath : la notion qui explique 80 % des erreurs

Le classpath est la **liste des endroits où la JVM cherche les classes**. Le compilateur (`javac -cp`) et le runtime (`java -cp`) en ont chacun un : les deux doivent contenir les mêmes choses, sinon on obtient des erreurs différentes mais liées.

Symptôme	Où ça se passe	Ce que ça veut dire
`package … does not exist`	Compilation	Le JAR de la dépendance n'est pas dans le `-cp` de `javac`.
`cannot find symbol`	Compilation	Idem, ou alors la version du JAR est trop ancienne (la méthode n'existe pas encore).
`NoClassDefFoundError` / `ClassNotFoundException`	Exécution	La compilation a réussi, mais le JAR manque dans le `-cp` de `java`. **C'est l'erreur la plus fréquente des JAR « qui ne marchent pas ».**
`no main manifest attribute`	Exécution de `java -jar`	Le manifeste du JAR ne contient pas `Main-Class` (voir A7).

La démonstration, en trois commandes

COMMANDES — REPRODUIRE VOLONTAIREMENT LES DEUX ERREURS

```
# 1) dépendance absente au RUNTIME (compilation faite, exécution sans le JAR)
java -cp out fr.nice.shop.App X

# 2) classpath avec * NON protégé : c'est le SHELL qui développe le *
javac -cp lib/* -d out src/main/java/fr/nice/shop/App.java
```

SORTIES RÉELLES

```
Exception in thread "main" java.lang.NoClassDefFoundError: org/apache/commons/lang3/StringUtils
        at fr.nice.shop.App.main(App.java:11)
Caused by: java.lang.ClassNotFoundException: org.apache.commons.lang3.StringUtils

error: invalid flag: lib/junit-platform-console-standalone-1.12.2.jar
Usage: javac <options> <source files>
```

La règle des jokers : `"lib/*"` entre guillemets

Dans `java -cp "lib/*" …`, le joker `*` est interprété **par la JVM**, qui ajoute tous les JAR du dossier. Mais si vous ne mettez pas de guillemets, c'est d'abord le shell qui développe `*` en une liste de fichiers, et la JVM reçoit plusieurs arguments — d'où `invalid flag`. Deux notes importantes : `*` ne traverse pas les sous-dossiers, et il ne prend que les `.jar` (pas les `.zip` ni les dossiers de classes).

Autre piège : sur le **classpath**, le séparateur est `:` (deux-points) sous Linux/macOS, et `;` (point-virgule) sous Windows. C'est une cause de classpath silencieusement vide quand on copie une commande d'un système à l'autre.

3.6 A6 — Les tests avec JUnit 5, sans build tool

JUnit 5 (Jupiter) s'exécute en ligne de commande grâce à un artefact unique qui contient le moteur et ses dépendances : `junit-platform-console-standalone`. C'est la méthode officielle pour lancer des tests sans Maven, Gradle ni IDE.

COMMANDES — INSTALLER LE LANCEUR DE TESTS

```
BASE=https://repo1.maven.org/maven2
curl -fsSL -o lib/junit-platform-console-standalone-1.12.2.jar \
  $BASE/org/junit/platform/junit-platform-console-standalone/1.12.2/junit-platform-console-
standalone-1.12.2.jar
curl -fsSL -o /tmp/junit.sha1 \
  $BASE/org/junit/platform/junit-platform-console-standalone/1.12.2/junit-platform-console-
standalone-1.12.2.jar.sha1
sha1sum lib/junit-platform-console-standalone-1.12.2.jar; cat /tmp/junit.sha1

# Quelle version de JUnit Jupiter ce lanceur embarque-t-il ?
java -jar lib/junit-platform-console-standalone-1.12.2.jar --version
```

SORTIE RÉELLE

```
9eaf86bbd90d…  lib/junit-platform-console-standalone-1.12.2.jar
9eaf86bbd90d…
1.12.2
```

> **Versions : Platform 1.12.2 correspond à Jupiter 5.12.2**
>
> JUnit publie deux lignées : JUnit Platform (le moteur, versions 1.x) et JUnit Jupiter (les API de test, versions 5.x). Le lanceur « console-standalone » est numéroté en 1.x et embarque la version 5.x correspondante. Écrire `junit-jupiter:5.12.2` dans un POM et télécharger `platform 1.12.2` à la main revient exactement au même.

COMMANDES — ÉCRIRE PUIS EXÉCUTER LES TESTS

```
cat > src/test/java/fr/nice/shop/PrixTest.java <<'EOF'
package fr.nice.shop;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertThrows;

import java.math.BigDecimal;
import org.junit.jupiter.api.Test;

class PrixTest {

    @Test
    void ttc_ajoute_la_tva() {
        assertEquals(new BigDecimal("120.00"), Prix.ttc(new BigDecimal("100.00")));
    }

    @Test
    void remise_de_10_pourcent() {
        assertEquals(new BigDecimal("90.00"), Prix.remise(new BigDecimal("100.00"), 10));
    }

    @Test
    void remise_invalide_leve_une_exception() {
        assertThrows(IllegalArgumentException.class,
                () -> Prix.remise(new BigDecimal("10.00"), 120));
    }
}
EOF

# 1. Compiler les tests : JUnit + les classes de l'application
javac --release 21 -cp "lib/junit-platform-console-standalone-1.12.2.jar:out" \
      -d test-out $(find src/test/java -name '*.java')

# 2. Exécuter : --scan-class-path découvre les tests automatiquement
java -jar lib/junit-platform-console-standalone-1.12.2.jar execute \
     --class-path "out:test-out" --scan-class-path
```

SORTIE RÉELLE (LES COULEURS ONT ÉTÉ RETIRÉES)

```
├─ JUnit Platform Suite ✔
├─ JUnit Jupiter ✔
│  └─ PrixTest ✔
│     ├─ ttc_ajoute_la_tva() ✔
│     ├─ remise_invalide_leve_une_exception() ✔
│     └─ remise_de_10_pourcent() ✔
└─ JUnit Vintage ✔

Test run finished after 100 ms
[         3 tests found           ]
[         3 tests successful      ]
[         0 tests failed          ]
```

Et quand un test échoue, le message est parfaitement exploitable — c'est le même moteur que celui utilisé par Maven et Gradle :

SORTIE RÉELLE — UN TEST VOLONTAIREMENT FAUX (ATTENDU 119.00 AU LIEU DE 120.00)

```
org.opentest4j.AssertionFailedError: expected: <119.00> but was: <120.00>
    ├─ PrixTest ✘
    │  └─ ttc_ajoute_la_tva() ✘ expected: <119.00> but was: <120.00>
```

3.7 A7 — Empaqueter : JAR, manifeste, Class-Path, Main-Class

Un JAR est une archive ZIP avec une particularité : le fichier `META-INF/MANIFEST.MF`. C'est lui qui rend un JAR exécutable, à condition d'y déclarer la classe principale.

COMMANDES — CONSTRUIRE UN JAR EXÉCUTABLE, À LA MAIN

```
# 1. Écrire le manifeste (attention : la syntaxe est stricte)
cat > manifest.txt <<'EOF'
Manifest-Version: 1.0
Main-Class: fr.nice.shop.App
Class-Path: lib/commons-lang3-3.20.0.jar
EOF

# 2. Créer l'archive : -C out . signifie "tout le contenu de out/, à la racine"
jar --create --file boutique.jar --manifest manifest.txt -C out .

# 3. Vérifier le manifeste réellement embarqué
unzip -p boutique.jar META-INF/MANIFEST.MF

# 4. Lister le contenu
jar --list --file boutique.jar

# 5. Lancer : Class-Path est relatif AU DOSSIER DU JAR, donc on se place à côté de lib/
mkdir -p dist && cp boutique.jar dist/ && ln -sfn ../lib dist/lib
LANG=C.UTF-8 java -jar dist/boutique.jar Alice
```

SORTIE RÉELLE

```
Manifest-Version: 1.0
Main-Class: fr.nice.shop.App
Class-Path: lib/commons-lang3-3.20.0.jar
Created-By: 25.0.5 (Eclipse Adoptium)

META-INF/
META-INF/MANIFEST.MF
application.properties
fr/
fr/nice/
fr/nice/shop/
fr/nice/shop/App.class
fr/nice/shop/Config.class
fr/nice/shop/Prix.class

Alice : 59.88 EUR TTC
Après remise de 10 % : 53.89 EUR
```

Les quatre pièges du manifeste

1. **Une ligne vide à la fin.** Un manifeste doit se terminer par une ligne vide, sinon la dernière entrée est ignorée. L'outil `jar` l'ajoute, mais si vous éditez le fichier à la main, vérifiez.
2. **Le retour à la ligne à 72 octets.** Un manifeste est limité à 72 octets par ligne ; au-delà, il faut une continuation commençant par un espace. Critique pour les `Class-Path` très longs.
3. **`Class-Path` est relatif au JAR, pas au dossier courant.** Vous pouvez lancer `java -jar dist/boutique.jar` depuis n'importe où : les chemins sont résolus depuis `dist/`.
4. **`Class-Path` n'accepte pas les jokers.** Chaque JAR doit être listé explicitement, séparé par des espaces. C'est précisément pour cela que le fat-jar existe.

COMMANDE — L'ERREUR CLASSIQUE, REPRODUITE

```
# Construire un JAR SANS manifeste, puis essayer de le lancer
jar --create --file nomain.jar -C out .
java -jar nomain.jar
```

```
no main manifest attribute, in nomain.jar
```

Trois façons de livrer, et leur compromis

Approche	Comment	Compromis
JAR + `lib/`	`Class-Path` dans le manifeste	Léger (4,4 Ko ici), mais il faut livrer l'arborescence `lib/` à côté. Les chemins doivent rester valides.
Fat-jar (« uber-jar »)	Un JAR unique contenant les classes des dépendances	Un seul fichier, lancement trivial. Mais il faut gérer les collisions de fichiers (META-INF, services) — d'où les plugins dédiés (shade chez Maven).
Runtime embarqué	`jlink` + `jpackage` (A9)	Installateur natif, aucune JVM à installer côté client. Le plus lourd à produire.

3.8 A8 — Industrialiser à la main : un Makefile

Répéter huit commandes dans le bon ordre, c'est déjà un build tool — un mauvais. Un `Makefile` de 40 lignes donne exactement les mêmes fonctionnalités qu'un cycle de vie Maven, sans rien installer.

FICHIER MAKEFILE (EXTRAIT — VERSION COMPLÈTE FOURNIE AVEC LA LEÇON)

```
JAVA_HOME ?= $(shell dirname $(dirname $(readlink -f $(command -v javac))))
JAVA      := $(JAVA_HOME)/bin/java
JAVAC     := $(JAVA_HOME)/bin/javac
JAR       := $(JAVA_HOME)/bin/jar
RELEASE   := 21
LIB       := lib/commons-lang3-3.20.0.jar
JUNIT     := lib/junit-platform-console-standalone-1.12.2.jar
SRC       := $(shell find src/main/java -name '*.java')
TEST_SRC  := $(shell find src/test/java -name '*.java')
OUT       := out
TEST_OUT  := test-out

all: jar test

compile:
        @mkdir -p $(OUT)
        $(JAVAC) --release $(RELEASE) -Xlint:all -cp "$(LIB)" -d $(OUT) $(SRC)

test: compile
        @mkdir -p $(TEST_OUT)
        $(JAVAC) --release $(RELEASE) -cp "$(JUNIT):$(OUT)" -d $(TEST_OUT) $(TEST_SRC)
        $(JAVA) -jar $(JUNIT) execute --class-path "$(OUT):$(TEST_OUT)" --scan-class-path

jar: compile
        @printf 'Manifest-Version: 1.0\nMain-Class: fr.nice.shop.App\nClass-Path: $(LIB)\n' >
manifest.txt
        $(JAR) --create --file boutique.jar --manifest manifest.txt -C $(OUT) .
run: jar
        LANG=C.UTF-8 $(JAVA) -jar boutique.jar $(ARGS)
clean:
        rm -rf $(OUT) $(TEST_OUT) boutique.jar manifest.txt
```

COMMANDES

```
make clean && make            # compile, teste, empaquette
make run ARGS=Bob            # exécute avec un argument
make clean                   # ne laisse rien
```

SORTIE RÉELLE

```
[         3 tests successful      ]
[         0 tests failed          ]

LANG=C.UTF-8 /opt/java/jdk-25.0.5+8/bin/java -jar boutique.jar Bob
Bob : 59.88 EUR TTC
Après remise de 10 % : 53.89 EUR
```

Ce qu'un Makefile ne fera jamais (et qui justifie Maven)

Il ne résout pas les dépendances. Il faut toujours télécharger les JAR et écrire leurs chemins à la main : pour 2 dépendances, c'est supportable ; pour 40, avec des dépendances transitives et des conflits de version, c'est ingérable. Make ne gère pas non plus les scopes (telle bibliothèque uniquement pour les tests), les profils, ni la publication dans un dépôt. Si votre projet dépasse dix dépendances, arrêtez le Makefile et passez au chapitre 4.

3.9 A9 — Livrer : jdeps, jlink, jpackage

Analyser les dépendances de modules : `jdeps`

COMMANDES

```
# De quels modules du JDK mon application a-t-elle besoin ?
jdeps --print-module-deps --ignore-missing-deps --multi-release 21 boutique.jar
```

```
java.base
```

Fabriquer un runtime minimal : `jlink`

COMMANDES

```
# Un JRE de 43 Mo au lieu d'un JDK de 303 Mo, contenant uniquement java.base
jlink --add-modules java.base --strip-debug --no-header-files --no-man-pages \
      --compress zip-6 --output /tmp/minijre

du -sh /tmp/minijre /opt/java/jdk-25.0.5+8

# L'application tourne telle quelle avec ce runtime
LANG=C.UTF-8 /tmp/minijre/bin/java -jar boutique.jar Test
```

SORTIE RÉELLE

```
43M     /tmp/minijre
303M    /opt/java/jdk-25.0.5+8
Test : 59.88 EUR TTC
Après remise de 10 % : 53.89 EUR
```

> **Syndrome de la « classe mystérieuse » avec `jdeps`**
>
> Dans notre essai, `jdeps` a d'abord répondu `Error: commons-lang3-3.20.0.jar is a multi-release jar file but --multi-release option is not set`, puis `Missing dependencies: classes not found… org.apache.commons.lang3.StringUtils not found`. Deux causes cumulées : `commons-lang3` est un multi-release JAR (il faut `--multi-release 21`), et le JAR de la dépendance n'était pas fourni à l'analyse (`--class-path "lib/*"`). Ce n'est pas un bug : `jdeps` refuse de deviner.

Produire un installateur natif : `jpackage`

COMMANDE — GABARIT (NÉCESSITE UN RUNTIME ET LES OUTILS DE PACKAGING DE LA DISTRIBUTION)

```
jpackage --type deb \
  --name boutique \
  --input dist \
  --main-jar boutique.jar \
  --main-class fr.nice.shop.App \
  --runtime-image /tmp/minijre \
  --app-version 1.0.0 \
  --linux-shortcut

# Sur Debian/Ubuntu, jpackage a besoin de : fakeroot, dpkg-dev (paquet .deb)
sudo apt install -y fakeroot dpkg-dev
ls -l *.deb
```

ATELIER A — 45 minutes, le projet complet à la main

Objectif : produire un projet livrable sans Maven ni Gradle, et savoir diagnostiquer les erreurs de classpath.

1. **(5 min)** Créez l'arborescence (A1) et écrivez `Prix.java`. Compilez, puis testez la classe avec `jshell --class-path out`.
2. **(5 min)** Ajoutez `application.properties` et `Config.java`. **Oubliez volontairement** l'étape `cp -r src/main/resources/. out/` : que se passe-t-il à l'exécution ? Pourquoi le message est-il si peu explicite ?
3. **(8 min)** Téléchargez `commons-lang3` et vérifiez son SHA-1 (A4). Compilez `App.java` avec et sans `-cp`. Notez la différence entre les deux messages d'erreur.
4. **(10 min)** Écrivez les trois tests, compilez-les, lancez-les avec le lanceur JUnit. Cassez volontairement une assertion pour voir le message d'échec, puis réparez.
5. **(8 min)** Construisez `boutique.jar` avec `Main-Class` et `Class-Path`. Lancez-le depuis un autre dossier. Puis retirez la ligne `Class-Path` : quelle erreur obtient-on ?
6. **(9 min)** Recopiez le Makefile, lancez `make clean && make`, puis `make run ARGS=Alice`. Enfin, `jdeps` + `jlink` : combien de mégaoctets avez-vous gagnés ?

4 Parcours B — Créer le même projet avec Maven

> Maven fait exactement ce que vous venez de faire à la main : compiler, copier les ressources, lancer les tests, fabriquer un JAR. Simplement, il le fait toujours de la même façon, sur toutes les machines, et il télécharge les dépendances à votre place. Ce chapitre vous montre l'outil par l'intérieur, dans l'ordre où on l'utilise vraiment.

Étape	Ce qu'on fait	Ce que ça remplace du parcours A
B1	Installer Maven (3 méthodes, une seule recommandée)	—
B2	Générer l'arborescence avec un archétype	A1 (`mkdir -p`)
B3	Comprendre le `pom.xml` ligne par ligne	Le Makefile, en mieux
B4	Lancer les bonnes commandes au bon moment	`make`, `make test`
B5	Déclarer une dépendance, comprendre les scopes	A4 + A5 (curl, classpath)
B6	Tester, empaqueter, lancer	A6, A7
B7	Figer la version de Maven dans le projet	—

La bonne façon de penser Maven

Maven n'est pas un « script de build ». C'est un **modèle** : un projet a une identité (coordonnées), un cycle de vie (phases ordonnées), un répertoire de sortie (`target/`), et des plugins qui s'accrochent à ces phases. Vous ne dites pas comment compiler ; vous dites ce que vous voulez obtenir (`package`), et Maven en déduit les étapes. C'est plus rigide que Make, et c'est précisément pour cela que deux projets Maven se ressemblent.

4.1 B1 — Installer Maven (paquet, archive Apache, SDKMAN!)

Le prérequis est simple : un JDK doit être présent et `JAVA_HOME` doit pointer dessus. Maven est une application Java ; il ne contient pas de JDK. Vérifiez d'abord :

COMMANDE — LE PRÉREQUIS, À NE PAS SAUTER

```
echo "$JAVA_HOME"
java -version
javac -version
```

SORTIE RÉELLE

```
/opt/java/jdk-25.0.5+8
openjdk version "25.0.5" 2026-10-20 LTS
OpenJDK Runtime Environment Temurin-25.0.5+8 (build 25.0.5+8-LTS)
OpenJDK 64-Bit Server VM Temurin-25.0.5+8 (build 25.0.5+8-LTS, mixed mode, sharing)
javac 25.0.5
```

Méthode 1 — le paquet de la distribution (rapide, mais version datée)

COMMANDES

```
# Quelle version le dépôt Debian propose-t-il ?
apt-cache policy maven
sudo apt install -y maven
mvn -v
```

SORTIE RÉELLE SUR DEBIAN 13 (TRIXIE)

```
maven:
  Installed: (none)
  Candidate: 3.9.9-1
  Version table:
     3.9.9-1 500
        500 http://deb.debian.org/debian trixie/main amd64 Packages
```

C'est parfaitement utilisable : Maven 3.9.9 est une version maintenue. Deux réserves cependant : le paquet installe aussi un JDK par défaut (qui peut ne pas être celui que vous voulez utiliser), et il ne suivra pas les correctifs entre deux versions de Debian. Sur un poste de développement, préférez la méthode 2 ou 3.

> **Le paquet `maven` n'est pas le problème — mais regardez `gradle`**
>
> Sur la même Debian 13, `apt-cache policy gradle` répond **4.4.1**, sorti en 2017, qui ne compile même pas du Java 21. Pour Gradle, le paquet de la distribution est à écarter sans discussion (chapitre 5). Pour Maven, 3.9.9 reste correct. Retenez la règle : vérifiez toujours la version du paquet avant de l'installer.

Méthode 2 — l'archive officielle Apache, avec vérification cryptographique (recommandée)

C'est la méthode que la documentation Apache décrit comme référence : télécharger le binaire et son empreinte, vérifier l'empreinte, puis installer. Une empreinte non vérifiée ne protège de rien.

COMMANDES — TÉLÉCHARGEMENT ET VÉRIFICATION SHA-512

```
cd /tmp
curl -fLO https://dlcdn.apache.org/maven/maven-3/3.10.0/binaries/apache-maven-3.10.0-bin.tar.gz
curl -fLO https://dlcdn.apache.org/maven/maven-3/3.10.0/binaries/apache-maven-3.10.0-
bin.tar.gz.sha512

# Le fichier .sha512 contient l'empreinte, pas le nom du fichier : on la remet en forme
echo "$(cat apache-maven-3.10.0-bin.tar.gz.sha512)  apache-maven-3.10.0-bin.tar.gz" | sha512sum -c
-
```

SORTIE RÉELLE

```
apache-maven-3.10.0-bin.tar.gz: OK
```

Les trois empreintes de cette archive, relevées sur la machine, sont :

Algorithme	Empreinte de `apache-maven-3.10.0-bin.tar.gz`
SHA-512	`908b1501bfb420bf7c8affb855534a9c407fd6099367bfb9f2f2dcb8e9799102bffb84518cde74c679bd76870247c6528683abdd620581bffa90f95d92d175aa`
SHA-1	`46e21f3dd2c2524faccf5bd4a7c7eb8f03b92f17`
MD5	`c0ad4bb59dbd99bcdbde758c9afcc0ee` — utile seulement pour détecter une corruption, jamais pour la sécurité

COMMANDES — INSTALLATION DANS `/OPT` + LIEN SYMBOLIQUE STABLE

```
sudo mkdir -p /opt/maven
sudo tar -xzf apache-maven-3.10.0-bin.tar.gz -C /opt/maven
sudo ln -sfn /opt/maven/apache-maven-3.10.0 /opt/maven/current
ls -l /opt/maven
```

SORTIE RÉELLE

```
total 8
drwxr-xr-x 6 root root 4096 Nov 10 09:58 apache-maven-3.10.0
lrwxrwxrwx 1 root root   29 Nov 10 09:58 current -> /opt/maven/apache-maven-3.10.0
```

Le lien `current` est le détail qui compte : votre `PATH` pointe sur `/opt/maven/current/bin`, donc installer la 3.11 plus tard ne demandera qu'un nouveau lien symbolique, sans toucher à vos fichiers de configuration.

COMMANDES — DÉCLARER MAVEN DANS LE SHELL

```
cat >> ~/.bashrc <<'EOF'

# --- Java + Maven ---
export JAVA_HOME=/opt/java/jdk-25.0.5+8
export PATH="/opt/maven/current/bin:$JAVA_HOME/bin:$PATH"
EOF
source ~/.bashrc
mvn -v
```

SORTIE RÉELLE

```
Apache Maven 3.10.0 (c43a36b8d67be7e0805a411bc0898af1a51f5472)
Maven home: /opt/maven/current
Java version: 25.0.5, vendor: Eclipse Adoptium, runtime: /opt/java/jdk-25.0.5+8
Default locale: en_US, platform encoding: UTF-8, time zone: Etc/UTC
OS name: "linux", version: "6.1.189+", arch: "amd64", family: "unix"
```

Si le téléchargement répond 404 ou 403

`dlcdn.apache.org` ne conserve que les versions actuellement publiées : dès qu'une 3.10.1 sort, la 3.10.0 disparaît du CDN. Basculez alors sur l'archive perpétuelle `https://archive.apache.org/dist/maven/maven-3/3.10.0/binaries/` — même fichiers, mêmes empreintes, débit plus lent. C'est aussi là que vous trouverez les anciennes versions pour reproduire un build d'époque.

Méthode 3 — SDKMAN! (plusieurs versions côte à côte)

COMMANDES

```
sdk install maven 3.10.0
sdk use maven 3.10.0        # uniquement pour ce terminal
sdk default maven 3.10.0    # pour tous les nouveaux terminaux
mvn -v
```

C'est la méthode la plus confortable quand vous devez jongler entre plusieurs projets figés sur des versions différentes : `sdk list maven` montre tout l'historique, et `sdk current maven` vous rappelle ce que vous utilisez. Elle suppose d'avoir suivi la leçon « Installer Java » (SDKMAN! déjà en place).

Méthode	Version obtenue	Avantages	Inconvénients
`apt install maven`	3.9.9 (Debian 13)	Une commande, mises à jour de sécurité par l'OS	Version figée, JDK imposé, pas de choix
Archive Apache	Celle que vous voulez, exactement	Version maîtrisée, empreinte vérifiable, installation propre dans `/opt`	Mise à jour manuelle
SDKMAN!	Au choix, à la volée	Basculement instantané entre versions, idéal multi-projets	Dépend d'un outil tiers

Versions de Maven, et où se situe 3.10.0

Version	Statut	JDK requis	Quand l'utiliser
3.10.0	Version recommandée (documentation Apache)	8+	Votre choix par défaut en 2026. C'est celle de ce document.
3.9.16	Branche 3.9 maintenue	8+	Projets existants qui n'ont pas encore migré ; c'est aussi la version des dépôts Debian à venir.
4.0.0-rc-7	Release candidate (aperçu)	17+	Essais, veille technique. **Pas en production** : l'API des plugins et le modèle de build évoluent encore.

> **Ne mélangez pas « version de Maven » et « version du modèle de POM »**
>
> `<modelVersion>4.0.0</modelVersion>` ne veut pas dire « Maven 4 ». C'est la version du format de POM, inchangée depuis 2004. Un POM en `modelVersion 4.0.0` fonctionne avec Maven 3.10 comme avec Maven 3.0. C'est l'erreur de lecture la plus fréquente chez les débutants.

4.2 B2 — Générer un projet : `archetype:generate`, pas à pas

Un **archétype** est un gabarit de projet : une arborescence, des fichiers modèles et des paramètres. Maven en publie plusieurs ; le plus simple est `maven-archetype-quickstart`, qui produit exactement la structure « `src/main/java` + `src/test/java` » que vous avez créée à la main.

D'abord, regarder le catalogue (mode interactif)

COMMANDE

```
# Sans argument : Maven affiche le catalogue des archétypes connus, puis pose des questions
mvn archetype:generate

# Pour filtrer le catalogue (le format est [groupId:]artifactId) :
mvn archetype:generate -Dfilter=org.apache.maven.archetypes:
```

C'est le mode que décrit la documentation du Maven Archetype Plugin : une longue liste numérotée (plusieurs centaines de lignes, dominée par les archétypes obsolètes), puis les questions `groupId`, `artifactId`, `version`, `package`, et une confirmation. Utile pour découvrir, pénible pour travailler : en pratique on passe tout en ligne de commande.

La commande non interactive, avec des paramètres explicites

COMMANDE

```
cd ~/projets
mvn archetype:generate \
  -DgroupId=fr.nice.shop \
  -DartifactId=demo \
  -DarchetypeArtifactId=maven-archetype-quickstart \
  -DarchetypeVersion=1.5 \
  -DinteractiveMode=false
```

SORTIE RÉELLE

```
[INFO] --- archetype:3.4.1:generate (default-cli) @ standalone-pom ---
[INFO] Generating project in Batch mode
[INFO] ----------------------------------------------------------------------------
[INFO] Using following parameters for creating project from Archetype: maven-archetype-quickstart:1.5
[INFO] ----------------------------------------------------------------------------
[INFO] Parameter: groupId, Value: fr.nice.shop
[INFO] Parameter: artifactId, Value: demo
[INFO] Parameter: version, Value: 1.0-SNAPSHOT
[INFO] Parameter: package, Value: fr.nice.shop
[INFO] Parameter: packageInPathFormat, Value: fr/nice/shop
[INFO] Parameter: junitVersion, Value: 5.11.0
[INFO] Parameter: javaCompilerVersion, Value: 17
[INFO] Project created from Archetype in dir: /home/user/projets/demo
[INFO] BUILD SUCCESS
```

Trois détails de cette sortie méritent votre attention, parce qu'ils expliquent 90 % des surprises avec les archétypes.

Piège n°1 — sans `-DarchetypeVersion`, vous obtenez l'archétype 1.0 (de 2004)

La version de l'archétype n'est pas la version de votre projet : c'est celle du gabarit. Si vous omettez `-DarchetypeVersion`, Maven choisit la version la plus récente qu'il trouve mais, dans beaucoup de miroirs et de configurations, il retombe sur la **1.0**. Résultat : un `pom.xml` sans `<properties>`, sans verrouillage de plugins, et surtout un test écrit pour **JUnit 3.8.1** :

CE QUE PRODUIT L'ARCHÉTYPE 1.0 (EXTRAITS RÉELS)

```
<dependency>
  <groupId>junit</groupId>
  <artifactId>junit</artifactId>
  <version>3.8.1</version>
  <scope>test</scope>
</dependency>
```

et une classe `AppTest extends TestCase` avec un constructeur `AppTest(String)`. Ce n'est pas faux, c'est simplement un monde qui a vingt ans d'âge. Précisez toujours `-DarchetypeVersion=1.5`.

Piège n°2 — l'archétype 1.5 cible Java 17 et JUnit 5.11.0

Les paramètres `javaCompilerVersion, Value: 17` et `junitVersion, Value: 5.11.0` de la sortie ci-dessus ne sont pas décoratifs : ils se retrouvent dans le POM. Si votre JDK est plus récent et que vous voulez cibler Java 21, il faut éditer deux lignes. C'est normal : un archétype est un point de départ, pas une vérité. Vous pouvez aussi les forcer dès la génération avec `-DjavaCompilerVersion=21 -DjunitVersion=5.12.2`.

> **Piège n°3 — les fichiers `.mvn/` générés sont vides**
>
> L'archétype 1.5 crée `.mvn/maven.config` et `.mvn/jvm.config`, tous deux **vides**. Ils ne servent à rien tant que vous n'y mettez rien, mais leur présence surprend. Le premier reçoit des options Maven appliquées à chaque invocation (par exemple `-T 1C` pour un build parallèle), le second des options de la JVM de Maven (`-Xmx1g`).

COMMANDE — CE QUE L'ARCHÉTYPE A RÉELLEMENT PRODUIT

```
find demo -type f | sort
```

SORTIE RÉELLE

```
demo/.mvn/jvm.config
demo/.mvn/maven.config
demo/pom.xml
demo/src/main/java/fr/nice/shop/App.java
demo/src/test/java/fr/nice/shop/AppTest.java
```

COMMANDES — LE PROJET GÉNÉRÉ COMPILE ET SE TESTE IMMÉDIATEMENT

```
cd demo
mvn -q test
mvn -q package
ls -l target/*.jar
```

Reste la question que l'archétype pose sans y répondre : quel `groupId` choisir, et pourquoi ? C'est l'objet de la **section 4.5**, qui donne la méthode en cinq questions, les valeurs que Maven accepte et celles que `javac` refuse.

Vous pouvez aussi démarrer sans archétype du tout, en copiant l'arborescence du parcours A et en écrivant un `pom.xml` minimal : `groupId`, `artifactId`, `version`, et c'est tout. Maven complète le reste. L'archétype sert surtout à éviter d'oublier les bonnes pratiques (encodage, verrouillage des plugins).

> **D'autres générateurs, pour d'autres besoins**
>
> L'archétype `maven-archetype-quickstart` est minimaliste. Pour une vraie application Web, on utilise plutôt un générateur spécialisé : **Spring Initializr** (`start.spring.io`) produit un projet Maven complet prêt à lancer, et **JHipster** génère une application complète (back + front). Les deux produisent des POM Maven ou Gradle : ce sont des « archétypes » au sens large. Pour un projet Java sans framework, `quickstart` suffit.

4.3 B2 bis — Créer un projet Maven vide, sans archétype

L'archétype est un confort : il produit un projet qui compile, mais il produit aussi du code d'exemple qu'il faut ensuite effacer. Pour un projet dont vous maîtrisez la structure, la méthode la plus rapide et la plus propre tient en **cinq commandes et un fichier**. C'est celle que nous utiliserons pour le reste du livre. Elle a un autre mérite : vous saurez ce que contient un POM minimal, donc vous ne recopierez plus des POM de 300 lignes sans les comprendre.

Étape 1 — Créer le squelette

COMMANDES

```
mkdir -p boutique/src/main/java
cd boutique

# l'arborescence minimale, vérifiée juste après
find . -type d | sort
```

SORTIE RÉELLE

```
.
./src
./src/main
./src/main/java
```

Notez ce qui n'existe pas : pas de `src/test/java`, pas de `target/`, pas de `src/main/resources`. Maven ne les exige pas : il les utilise s'ils existent et les ignore sinon. Un projet vide est donc littéralement un dossier `src/main/java` vide et un `pom.xml`.

Étape 2 — Écrire le `pom.xml` minimal (et suffisant)

FICHIER — `POM.XML` D'UN PROJET VIDE, COMPLET

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/
maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <!-- l'identité du projet : groupId:artifactId:version = ses coordonnées -->
  <groupId>fr.acme.boutique</groupId>
  <artifactId>boutique</artifactId>
  <version>1.0.0</version>
  <packaging>jar</packaging>

  <!-- deux propriétés qui évitent deux catégories entières de bugs -->
  <properties>
    <maven.compiler.release>21</maven.compiler.release>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>
</project>
```

Élément	Ce qu'il fait (et ce qui arrive si on l'oublie)
`groupId` · `artifactId` · `version`	Les **coordonnées** du projet. Elles déterminent le nom du JAR produit (`boutique-1.0.0.jar`) et le chemin dans le dépôt local (`~/.m2/repository/fr/acme/boutique/boutique/1.0.0/`).
`maven.compiler.release`	La version de Java cible, passée au compilateur sous forme de `--release 21`. Sans elle, Maven utilise une valeur par défaut historiquement très basse (1.5, 1.6, 1.7 selon la version du plugin) : c'est **la** cause du message `Source option 7 is no longer supported` (reproduit ci-dessous).
`project.build.sourceEncoding`	L'encodage de lecture des sources. Sans elle, l'encodage de la plateforme s'applique : accents corrompus, warnings, tests qui échouent sur une machine et pas sur l'autre.
`packaging`	`jar` par défaut. Inutile de l'écrire, sauf pour la rendre explicite (on rencontre aussi `pom` pour un agrégateur et `war` pour une application web).

Étape 3 — Vérifier que le projet vide « construit »

COMMANDES

```
mvn -B validate           # le POM est-il syntaxiquement valide et cohérent ?
mvn -B package            # construire : mise à part une alerte, ça passe
unzip -l target/boutique-1.0.0.jar
```

SORTIE RÉELLE (EXTRAITS)

```
$ mvn -B validate
[INFO] --------------------------------[ jar ]---------------------------------
[INFO] BUILD SUCCESS
[INFO] Total time:  0.095 s

$ mvn -B package
...
[WARNING] JAR will be empty - no content was marked for inclusion!
[INFO] Building jar: /home/…/boutique/target/boutique-1.0.0.jar
[INFO] BUILD SUCCESS
[INFO] Total time:  3.803 s
```

Le warning « JAR will be empty » est normal et sans conséquence : un projet sans classe produit un JAR sans classe. Ce qui compte, c'est que la chaîne complète — validation, cycle de vie, fabrication du JAR — fonctionne **avant** l'écriture de la première ligne de code. À partir de là, tout échec ultérieur viendra de votre code, pas de votre installation.

La version par défaut du compilateur, reproduite pour que vous la reconnaissiez

Voici ce qui se passe si l'on génère un projet avec l'archétype `quickstart` 1.4 (qui fixe `maven.compiler.source/target` à 1.7) et qu'on lance `mvn package` sur un JDK 25 :

SORTIE RÉELLE

```
[ERROR] COMPILATION ERROR :
[ERROR] Source option 7 is no longer supported. Use 8 or later.
[ERROR] Target option 7 is no longer supported. Use 8 or later.
[INFO] BUILD FAILURE
[ERROR] Failed to execute goal org.apache.maven.plugins:maven-compiler-plugin:3.8.0:compile
        (default-compile) on project boutique: Compilation failure
```

Correctif : supprimez `maven.compiler.source` et `maven.compiler.target`, remplacez-les par `<maven.compiler.release>21</maven.compiler.release>`, et mettez à jour `maven-compiler-plugin` au moins en 3.13.0. Le POM minimal ci-dessus n'a pas ce problème par construction.

4.4 B2 ter — Créer un package dans un projet Maven

Toute la question tient en une phrase : dans un projet Maven, **on ne « crée » pas un package avec une commande, on crée un dossier sous `src/main/java` et on déclare le même nom en première ligne du fichier**. Maven ne connaît pas la notion de package : il compile toutes les sources sous `src/main/java` et laisse `javac` écrire les `.class` dans `target/classes` en respectant le package déclaré.

La correspondance exacte, en trois colonnes

Package déclaré	Fichier source (dans un projet Maven)	Classe compilée
`fr.acme.boutique`	`src/main/java/fr/acme/boutique/App.java`	`target/classes/fr/acme/boutique/App.class`
`fr.acme.boutique.model`	`src/main/java/fr/acme/boutique/model/Produit.java`	`target/classes/fr/acme/boutique/model/Produit.class`
`fr.acme.boutique.service` (test)	`src/test/java/fr/acme/boutique/service/CaisseTest.java`	`target/test-classes/fr/acme/boutique/service/CaisseTest.class`

> **Pourquoi `src/main/java` n'apparaît jamais dans le package**
>
> `src/main/java` est la **racine des sources** (source root) : le point de départ à partir duquel le chemin est le package. Un fichier placé dans `src/main/java/fr/acme/boutique/model/` doit donc déclarer `package fr.acme.boutique.model;` — et non `package src.main.java.fr.acme…`. C'est l'erreur n°1 des débutants qui « suivent le chemin ».

Les trois façons de le faire, de la plus concrète à la plus outillée

1. EN LIGNE DE COMMANDE (CELLE QUI MARCHE PARTOUT)

```
# créer le dossier du package et le fichier, puis écrire le package en 1re ligne
mkdir -p src/main/java/fr/acme/boutique/model
cat > src/main/java/fr/acme/boutique/model/Produit.java <<'EOF'
package fr.acme.boutique.model;

public record Produit(String nom, long prixCentimes) {
    public long prixTTC(int tauxPourCent) {
        return Math.round(prixCentimes * (1 + tauxPourCent / 100.0));
    }
}
EOF

# compiler : Maven trouve seul la nouvelle classe
mvn -B -q compile
# vérifier ce qui a été produit
find target/classes -name '*.class' | sort
```

SORTIE RÉELLE

```
$ find target/classes -name '*.class' | sort
target/classes/fr/acme/boutique/App.class
target/classes/fr/acme/boutique/model/Produit.class
```

2. DANS UN IDE (INTELLIJ, ECLIPSE, VS CODE)

```
IntelliJ IDEA    clic droit sur src/main/java → New → Package → fr.acme.boutique.model
                 puis clic droit sur le nouveau package → New → Java Class → Produit
                 (l'IDE pré-remplit la ligne package)

Eclipse          clic droit sur src/main/java → New → Package → Name : fr.acme.boutique.model
                 puis File → New → Class (le champ Package est déjà rempli)

VS Code          créez simplement le dossier dans l'explorateur, puis le fichier .java ;
                 l'extension Java complète la ligne package au premier enregistrement.
```

3. EN FORÇANT LE PACKAGE À LA GÉNÉRATION DU PROJET

```
# l'archétype accepte un package différent du groupId
mvn archetype:generate -DgroupId=fr.acme \
  -DartifactId=boutique \
  -DarchetypeArtifactId=maven-archetype-quickstart \
  -DarchetypeVersion=1.4 \
  -Dpackage=fr.acme.boutique \
  -DinteractiveMode=false
```

SORTIE RÉELLE (EXTRAIT)

```
[INFO] Parameter: package, Value: fr.acme.boutique
[INFO] Parameter: packageInPathFormat, Value: fr/acme/boutique
[INFO] Project created from Archetype in dir: /tmp/boutique
```

Vérifier que tout est cohérent (le contrôle en 10 secondes)

COMMANDE — COMPARER CE QUI EST DÉCLARÉ ET CE QUI EXISTE

```
# tous les packages déclarés dans les sources
grep -rh "^package " src/main/java | sort -u

# tous les dossiers sous la racine des sources
find src/main/java -type d -mindepth 1 | sed 's|src/main/java/||; s|/|.|g' | sort -u
```

SORTIE RÉELLE

```
$ grep -rh "^package " src/main/java | sort -u
package fr.acme.boutique;
package fr.acme.boutique.model;

$ find src/main/java -type d -mindepth 1 | sed 's|src/main/java/||; s|/|.|g' | sort -u
fr/acme/boutique
fr/acme/boutique/model
```

Les deux listes doivent se correspondre (la seconde s'affiche avec des `/` : convertissez mentalement, ou ajoutez `tr '/' '.'`). Un écart signale un package déclaré sans dossier, ou un dossier sans classe.

Le cas important : la classe de test, dans le même package

Voici la raison pour laquelle le découpage en packages compte autant pour les tests : une classe de test placée dans le **même package** que la classe testée accède à ses membres package-private. C'est ce qui permet de tester l'intérieur d'un package sans rendre public ce qui ne doit pas l'être.

FICHIER — SRC/TEST/JAVA/FR/ACME/BOUTIQUE/APPTEST.JAVA

```
package fr.acme.boutique;                // même package que App

import static org.junit.jupiter.api.Assertions.assertEquals;

import org.junit.jupiter.api.Test;

class AppTest {
    @Test
    void le_paquet_est_bien_fr_acme_boutique() {
        assertEquals("fr.acme.boutique", App.class.getPackageName());
    }
}
```

EXÉCUTION RÉELLE

```
$ mvn -B package
[INFO] --- surefire:3.5.5:test (default-test) @ boutique ---
[INFO] Running fr.acme.boutique.AppTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.084 s -- in
fr.acme.boutique.AppTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
```

La panne silencieuse : les tests qui ne s'exécutent pas

Maven ne cherche les tests que selon deux critères : ils doivent être sous `src/test/java`, et leur nom de classe doit correspondre aux motifs par défaut de Surefire (`*Test`, `Test*`, `*Tests`, `*TestCase`). Un fichier nommé `AppTest.java` placé par erreur dans `src/main/java`, ou une classe nommée `VerifierApp`, donne ceci — et le build est **vert** :

SORTIE RÉELLE, AVEC UN TEST INVISIBLE

```
[INFO] Tests run: 0, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
```

Règle : après chaque ajout de test, regardez le nombre de tests exécutés. « 0 test » n'est pas une réussite, c'est un test qui n'a pas eu lieu.

Le résumé à retenir

Projet vide = `pom.xml` + `src/main/java/`. Rien d'autre n'est obligatoire, et `mvn package` doit réussir dès ce stade. **Nouveau package** = un dossier sous `src/main/java` + une ligne `package` identique dans chaque fichier ; Maven le découvre seul, il n'y a rien à déclarer dans le POM. **Nouvelle classe de test** = sous `src/test/java`, dans le package de la classe testée, avec un nom en `…Test`. Ces trois phrases suffisent à créer n'importe quel projet Maven.

4.5 B2 quater — Choisir ses coordonnées : `groupId`, `artifactId`, `version`

La génération d'un projet ne pose qu'une question vraiment difficile, et l'assistant la pose en trois mots : Group Id ?. C'est le seul des trois champs dont le mauvais choix ne se corrige pas : l'`artifactId` se renomme avec un alias, la `version` avance d'elle-même, un `groupId` publié reste pour toujours. Cette section donne la réponse en cinq questions, puis les règles exactes — mesurées sur Maven 3.9.16 et le JDK 25.

Trois coordonnées, une adresse

Un artefact Maven est identifié par trois valeurs, ses **coordonnées**. La documentation Maven les décrit comme une adresse : « The three fields act much like an address and timestamp in one. »

Coordonnée	Ce qu'elle désigne	Dans le projet de ce livre
`groupId`	L'organisation ou le groupe de projets — la famille à laquelle l'artefact appartient	`fr.nice.shop`
`artifactId`	Le nom du projet lui-même, unique à l'intérieur du groupe	`boutique`
`version`	Laquelle des incarnations du projet on veut	`1.0-SNAPSHOT`

Le `groupId` est le **nom de famille, pas le prénom**. Deux projets de la même structure partagent le `groupId` et diffèrent par l'`artifactId` : `fr.nice.shop:boutique` et `fr.nice.shop:caisse`. La convention officielle part d'un nom de domaine inversé que vous contrôlez : « Each groupId should follow Java's package name rules. This means it starts with a reversed domain name you control. » Les groupes plus fins sont encouragés — « Each subgroup should append a new identifier to the parent's groupId » — d'où `fr.nice.shop.outils` à côté de `fr.nice.shop`.

La démonstration la plus concrète tient dans le chemin du dépôt local. Publiez le projet, puis regardez où Maven le range :

COMMANDES — PUBLIER ET REGARDER L'ADRESSE

```
mvn -B install
find ~/.m2/repository/fr/nice/shop -name '*.jar' -o -name '*.pom'
```

SORTIE RÉELLE

```
[INFO] Installing /home/user/projets/boutique/target/boutique-1.0-SNAPSHOT.jar to
       /home/user/.m2/repository/fr/nice/shop/boutique/1.0-SNAPSHOT/boutique-1.0-SNAPSHOT.jar
[INFO] Installing /home/user/projets/boutique/pom.xml to
       /home/user/.m2/repository/fr/nice/shop/boutique/1.0-SNAPSHOT/boutique-1.0-SNAPSHOT.pom
[INFO] BUILD SUCCESS

$ find ~/.m2/repository/fr/nice/shop \( -name '*.jar' -o -name '*.pom' \) | sort
/home/user/.m2/repository/fr/nice/shop/boutique/1.0-SNAPSHOT/boutique-1.0-SNAPSHOT.jar
/home/user/.m2/repository/fr/nice/shop/boutique/1.0-SNAPSHOT/boutique-1.0-SNAPSHOT.pom
```

Le chemin, `~/.m2/repository/groupId en répertoires/artifactId/version/`, est la réponse la plus concrète à la question « c'est quoi le `groupId` » : un **chemin de classement**, exactement comme un nom de package est un chemin de répertoires pour le compilateur.

Ce que Maven accepte, ce qu'il refuse (mesuré)

Le POM ne valide pas le `groupId` comme un identifiant Java : il applique une expression régulière plus permissive, dont le motif est `[A-Za-z0-9_\-.]+`. Résultat de treize valeurs réellement testées avec `mvn validate`, regroupées en neuf lignes.

`groupId` testé	Maven	Pourquoi
`fr.nice.shop`	accepté	Domaine inversé, minuscules : la forme recommandée
`Fr.Nice.Shop`	accepté	Les majuscules passent la validation — mais la convention Java les déconseille
`fr-nice-shop`	accepté	Le tiret est autorisé (la doc cite `commons-io`, groupe à mot unique historique)
`fr_nice`	accepté	Le souligné est autorisé
`fr.nice.2026`, `fr.nice.9x`, `123.abc`	acceptés	Les chiffres passent, y compris en tête de segment
`-fr.nice`, `.fr.nice`	acceptés	Maven ne vérifie pas la structure — ces formes sont pourtant invalides comme package Java
`fr.nice.aléa`	**refusé**	`does not match a valid id pattern` : l'accent sort du jeu ASCII
`fr nice`, `fr.nice.a+b`, `fr.nice.alea*`	**refusés**	Espace, `+`, `*` : hors du motif

Ce que Java accepte, lui (mesuré)

Un `groupId` valide n'est pas forcément un nom de package : ce sont deux validateurs différents. Huit fichiers, chacun avec une seule ligne `package`, compilés séparément avec `javac -d out` :

PROGRAMME — HUIT NOMS DE PACKAGE, UN FICHIER PAR NOM

```
// chacun déclaré dans son propre fichier, puis compilé séparément
package org.example.hyphenated_name;   // domaine hyphenated-name.example.org
package com.example._123name;          // domaine 123name.example.com
package int_.example;                  // domaine example.int
package fr.nice.shop.caisse;           // forme normale
package org.example.int;               // mot-clé réservé
package com.example.123name;           // segment commençant par un chiffre
package fr.nice.class;                 // mot-clé « class »
package fr.nice.9x;                    // segment commençant par un chiffre
```

SORTIE RÉELLE — `JAVAC -D OUT`

```
COMPILE   org.example.hyphenated_name
COMPILE   com.example._123name
COMPILE   int_.example
COMPILE   fr.nice.shop.caisse
REFUSÉ    org.example.int      -> error: <identifier> expected
REFUSÉ    com.example.123name  -> error: ';' expected
REFUSÉ    fr.nice.class        -> error: <identifier> expected
REFUSÉ    fr.nice.9x           -> error: ';' expected
```

Deux noms, deux destins : `fr-nice-shop` est un `groupId` parfaitement valide et un package impossible ; `fr.nice.shop` sert aux deux. C'est le meilleur argument pour choisir un `groupId` qui soit déjà un nom de package légal. Le tutoriel Java donne d'ailleurs la règle de conversion pour les domaines récalcitrants — un souligné : `hyphenated-name.example.org` donne le package `org.example.hyphenated_name`, `123name.example.com` donne `com.example._123name`.

La méthode en cinq questions

1 — Contrôlez-vous un nom de domaine ?

Si oui, la réponse est **le domaine à l'envers** : `nice.fr` devient `fr.nice`, `example.com` devient `com.example`. C'est la convention de la documentation Maven, celle du tutoriel Java (« Companies use their reversed Internet domain name to begin their package names ») et celle du registre Central, qui exige une preuve de propriété (enregistrement DNS `TXT`) avant d'accepter le namespace. Les sous-groupes sont ensuite libres, sans nouvelle autorisation.

2 — Sinon, avez-vous un compte de forge ?

Sans domaine, Central accepte un `groupId` rattaché à une plateforme de code, vérifiable par la création d'un dépôt temporaire. Les formes sont fixées :

Plateforme	`groupId` à utiliser
GitHub	`io.github.`*`votre-nom-utilisateur`*
GitLab	`io.gitlab.`*`votre-nom-utilisateur`*
Bitbucket	`io.bitbucket.`*`votre-nom-utilisateur`*
Gitee	`io.gitee.`*`votre-nom-utilisateur`*
SourceForge	`io.sourceforge.`*`votre-nom-utilisateur`*

C'est la solution la plus courante pour un projet personnel : `io.github.dupont.boutique` s'obtient en quelques minutes, sans rien acheter. Si vous vous connectez au portail Central avec votre compte GitHub, le namespace `io.github.`*`votre-nom`* est même accordé automatiquement.

3 — Ce nom restera-t-il disponible ?

Un domaine se perd, un compte de forge se renomme, une entreprise change de marque. Un `groupId` publié est **permanent** : Central ne supprime pas d'artefact, et des projets peuvent en dépendre. Préférez donc le nom de l'organisation à celui d'un produit qui sera renommé, et n'ajoutez un segment personnel que si ce rattachement est assumé durablement.

4 — Est-ce un identifiant Java légal ?

Maven accepte `fr-nice-shop` et `fr.nice.2026` ; `javac` refuse les deux (mesures ci-dessus). Choisissez la forme qui sert aux deux usages : minuscules, points, segments alphabétiques.

5 — Simple ou composé ?

Un mot unique reste techniquement possible, mais Central prévient : « it will be difficult to get a new single word group ID approved ». `commons-io` et `junit` sont des vestiges. Pour un projet neuf : au moins deux segments, dont un domaine ou une forge, et un troisième pour séparer les projets d'une même structure.

6 — Projet scolaire : aucun domaine, aucun compte de forge

C'est le cas le plus fréquent, et la réponse est simple : **vous n'avez besoin ni de domaine ni de compte**. Le `groupId` exigé par Maven est une chaîne de caractères, pas un titre de propriété ; la vérification de domaine n'existe que pour publier sur Maven Central, ce que vous ne ferez pas. Trois options, dans l'ordre de préférence.

`groupId` conseillé	Quand l'utiliser
`fr.nice.nom-projet`	Si l'établissement vous autorise à utiliser son domaine — c'est le plus propre : même forme que dans la vie professionnelle. Demandez au professeur : le domaine appartient à l'école, pas à vous.
`oge.nom-projet`	La convention des projets scolaires et étudiants : `oge` pour Oracle Groundbreaker Environment. Aucun domaine à posséder, aucun conflit possible avec un éditeur.
`etu.votre-nom.nom-projet`	Pour ranger plusieurs projets ou plusieurs membres d'un binôme : `etu.dupont.boutique`, `etu.martin.boutique`. Lisible par le correcteur, unique sur la machine.

Ce qui reste obligatoire, même sans domaine : **minuscules, ASCII, points**, et un nom qui est aussi un package Java légal. `oge.boutique` et `etu.dupont.boutique` passent les deux contrôles (Maven et `javac`) ; `OG3-Boutique`, `projet école` ou `oge.2026` échouent. Évitez aussi `fr.` ou `com.` inventés : ils suggèrent un domaine que vous ne contrôlez pas. Enfin, ne mettez pas de numéro d'étudiant dans le `groupId` — il finit dans tous les chemins de fichiers du rendu.

La question que le professeur peut poser

Question	Réponse à donner
« Pourquoi `oge` et pas votre nom ? »	`oge` est la convention des projets étudiants ; le nom du projet suffit à l'identifier, et cela évite de mêler identité personnelle et coordonnées techniques.
« Ce `groupId` pourrait-il être publié tel quel ? »	Non, et ce n'est pas le but : Central exigerait une preuve de propriété du domaine. Le `groupId` reste valide pour compiler, tester et empaqueter, ce qui est l'objectif du projet.
« Que se passe-t-il si je renomme le projet ? »	On change l'`artifactId`, jamais le `groupId` : le groupe désigne l'établissement ou le cours, pas le projet. Renommer le `groupId` après coup déplace le chemin dans le dépôt local.
« Et si je travaille en binôme ? »	Même `groupId` pour les deux (celui du cours ou de l'établissement), deux `artifactId` distincts, ou un segment par personne si les deux rendus sont séparés.

La grille de décision

Situation	`groupId` recommandé	Pourquoi
Entreprise avec domaine	`fr.nice.shop`	Domaine inversé + nom de la structure. Sous-groupes libres ensuite.
Projet scolaire ou étudiant	`oge.`*`nom-projet`* ou `etu.`*`nom.nom-projet`*	Aucun domaine ni compte à créer : convention `oge`, ou domaine de l'établissement s'il est autorisé (question 6).
Particulier sans domaine	`io.github.dupont`	Vérifiable immédiatement, gratuit, accepté par Central.
Projet libre sur GitLab	`io.gitlab.monprojet`	Même mécanisme, preuve par dépôt temporaire.
Bibliothèque destinée à être publiée	`io.github.dupont.boutique`	Le sous-groupe réserve la place des autres modules.
Projet interne, jamais publié	`local.dupont` ou le domaine de l'entreprise	Rien ne sort du dépôt local : l'essentiel est l'unicité sur la machine et la lisibilité.
Module d'un projet multi-modules	`fr.nice.shop.caisse`	Maven conseille un segment par module : `org.apache.maven`, `org.apache.maven.plugins`.

Les dix règles, et celles de l'`artifactId`

#	Règle	Concrètement
1	Domaine inversé, en minuscules	`example.com` → `com.example`. Jamais `com.Example`.
2	Pas de préfixe réservé	Jamais `java.`, `javax.`, `jdk.` : ces espaces appartiennent au JDK.
3	Pas un nom de projet	`boutique` comme `groupId` n'identifie personne — c'est un `artifactId`.
4	ASCII strict	`fr.nice.aléa` est refusé par Maven : `does not match a valid id pattern`.
5	Ni espace ni symbole	Espace, `+`, `*` : refusés (mesurés).
6	Pas de tiret si vous voulez le réutiliser en package	`fr-nice-shop` passe Maven, pas Java.
7	Pas de segment commençant par un chiffre	`fr.nice.9x` : accepté par Maven, refusé par `javac`.
8	Pas de mot-clé Java	`int`, `class`, `new`, `record`… ne sont pas des identifiants légaux.
9	Stable et impersonnel	Le nom de la structure, pas le pseudonyme du moment ni le nom du produit.
10	Cohérent avec le package	Le premier choix reste de réutiliser le même nom : moins de conversions, moins d'erreurs.

Champ	Dans ce livre	Règle de nommage
`artifactId`	`boutique`	Minuscules, chiffres et tirets ; pas de point. Sert de nom au fichier produit : `boutique-1.0-SNAPSHOT.jar`.
`version`	`1.0-SNAPSHOT`	SemVer (`majeur.mineur.correctif`) ; le suffixe `-SNAPSHOT` signale une version en cours, que les dépôts traitent différemment (section 7.2).

Ne changez jamais un `groupId` déjà publié

Si `fr.acme:boutique:1.0.0` est déjà chez vos consommateurs, passer à `fr.acme.shop:boutique:1.0.0` crée un second artefact : les dépendances existantes continuent de tirer l'ancien, que vous ne corrigez plus. Un renommage se fait avec une nouvelle version et une procédure de migration — jamais dans l'ombre d'un commit « nettoyage ».

Atelier — dix minutes pour figer vos coordonnées

À la fin, vous avez les trois valeurs à réutiliser dans tous vos projets, et la preuve qu'elles fonctionnent.

1. Répondez aux cinq questions et écrivez vos trois coordonnées : `groupId`, `artifactId`, `version`.
2. Vérifiez le `groupId` avec `mvn validate` dans un POM minimal : il doit passer sans avertissement.
3. Créez un fichier dont la première ligne est `package votre.groupId;` et compilez-le : il doit passer aussi. C'est le double contrôle.
4. Lancez `mvn install` et vérifiez que le chemin dans `~/.m2/repository` contient bien vos points transformés en répertoires.
5. Notez le tout dans le `README` du projet : dans six mois, c'est ce qui évitera le renommage accidentel.

4.6 B3 — Le POM décortiqué, ligne par ligne

Le `pom.xml` (Project Object Model) est le seul fichier de configuration d'un projet Maven. Voici celui de notre boutique, commenté. C'est le POM que les scripts fournis avec ce document écrivent, et celui qui a produit toutes les sorties du chapitre.

FICHIER — `POM.XML` COMPLET ET COMMENTÉ

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/
maven-4.0.0.xsd">

  <!-- 1. Version du modèle de POM : toujours 4.0.0 (ce n'est PAS la version de Maven) -->
  <modelVersion>4.0.0</modelVersion>

  <!-- 2. Identité du projet : groupId:artifactId:version (= "les coordonnées") -->
  <groupId>fr.nice.shop</groupId>
  <artifactId>boutique</artifactId>
  <version>1.0-SNAPSHOT</version>
  <packaging>jar</packaging>
  <name>boutique</name>

  <!-- 3. Propriétés : variables réutilisables, et réglages des plugins -->
  <properties>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <project.reporting.outputEncoding>UTF-8</project.reporting.outputEncoding>
    <maven.compiler.release>21</maven.compiler.release>
    <commons-lang3.version>3.20.0</commons-lang3.version>
  </properties>
```

(SUITE) — DÉPENDANCES ET PLUGINS

```
  <!-- 4. dependencyManagement : versions centralisées, déclarées une seule fois -->
  <dependencyManagement>
    <dependencies>
      <dependency>
        <groupId>org.junit</groupId>
        <artifactId>junit-bom</artifactId>
        <version>5.12.2</version>
        <type>pom</type>
        <scope>import</scope>
      </dependency>
    </dependencies>
  </dependencyManagement>

  <!-- 5. Dépendances réelles. scope = compile (défaut) | test | provided | runtime -->
  <dependencies>
    <dependency>
      <groupId>org.apache.commons</groupId>
      <artifactId>commons-lang3</artifactId>
      <version>${commons-lang3.version}</version>
    </dependency>

    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter</artifactId>
      <scope>test</scope>  <!-- présent à la compilation des tests, absent du JAR livré -->
    </dependency>
  </dependencies>
```

(SUITE) — LA SECTION `BUILD`

```
  <!-- 6. Build : version des plugins figée + manifeste du JAR -->
  <build>
    <pluginManagement>
      <plugins>
        <plugin><artifactId>maven-clean-plugin</artifactId><version>3.4.0</version></plugin>
```

```
        <plugin><artifactId>maven-resources-plugin</artifactId><version>3.3.1</version></plugin>
        <plugin><artifactId>maven-compiler-plugin</artifactId><version>3.13.0</version></plugin>
        <plugin><artifactId>maven-surefire-plugin</artifactId><version>3.3.0</version></plugin>
        <plugin><artifactId>maven-jar-plugin</artifactId><version>3.4.2</version></plugin>
      </plugins>
    </pluginManagement>

    <plugins>
```

(SUITE) — LES PLUGINS RÉELLEMENT EXÉCUTÉS

```
      <!-- manifeste : rend le JAR lançable par java -jar -->
      <plugin>
        <artifactId>maven-jar-plugin</artifactId>
        <configuration>
          <archive>
            <manifest>
              <mainClass>fr.nice.shop.App</mainClass>
            </manifest>
          </archive>
        </configuration>
      </plugin>
    </plugins>
  </build>
</project>
```

Trois remarques de lecture, avant le détail. **Un** : les numéros de version des plugins sont dans `pluginManagement`, mais les `<dependency>` de JUnit n'ont pas de version — c'est le BOM importé en section 4 qui les fournit. **Deux** : le plugin `maven-jar-plugin` apparaît deux fois, une fois pour figer sa version, une fois pour le configurer ; c'est normal et sans effet de bord. **Trois** : rien dans ce POM ne dit « comment compiler » — `<maven.compiler.release>` suffit à configurer le plugin `compiler`, qui lit cette propriété par convention.

> **Ce qui ne doit jamais figurer dans un POM**
>
> **Des identifiants** (mot de passe, jeton) : ils vont dans `settings.xml`, hors du dépôt. **Un chemin absolu** (`/home/alice/…`) : le projet doit se construire ailleurs. **Une dépendance « au hasard »** sans version ni BOM : le jour où une nouvelle version sort, votre build change sans que vous ayez touché à une ligne. **Un plugin configuré dans `pluginManagement` en croyant qu'il s'exécute** : c'est l'erreur reproduite ci-dessous, et elle ne produit aucun message.

Les six sections, et ce qu'elles font vraiment

Élément	Rôle	Ce qu'il faut savoir
`modelVersion`	Version du format de POM	Toujours `4.0.0`. Confondue avec la version de Maven par presque tout le monde au début.
`groupId` `artifactId` `version`	Les coordonnées : identité unique du projet	Elles déterminent le chemin dans le dépôt : `fr/nice/shop/boutique/1.0-SNAPSHOT/boutique-1.0-SNAPSHOT.jar`. Changer la version change le chemin, donc le fichier produit.
`packaging`	Type d'artefact : `jar`, `war`, `pom`, `ear`…	Détermine le cycle de vie et les plugins liés. `pom` = projet d'agrégation, sans code.
`properties`	Variables `${…}` réutilisables	Certaines sont lues directement par les plugins (`maven.compiler.release`, `project.build.sourceEncoding`). `UTF-8` n'est pas un détail : sans lui, les accents de vos messages de sortie cassent sur un système en `POSIX`.
`dependencyManagement`	Centralise des versions	Ne déclare rien : ne télécharge rien et n'ajoute rien au classpath. Sert à dire « si un module demande JUnit, prends cette version ». `scope=import` + `type=pom` importe un BOM (une liste de versions cohérentes).
`dependencies`	Les dépendances effectives	Le `<scope>` décide de la visibilité (section 4.8). Une dépendance sans `<version>` et sans BOM ne compile pas : « 'dependencies.dependency.version' is missing ».
`build/pluginManagement`	Fige les versions de plugins	Un plugin déclaré seulement ici n'est **pas exécuté**. Erreur classique : on y met le shade plugin, et aucun fat-jar n'apparaît.
`build/plugins`	Plugins réellement actifs	C'est ici que se configurent les `<executions>` (quelle phase, quel goal).

> **L'erreur qui coûte le plus de temps : `<location>` du plugin**
>
> Nous avons reproduit les deux cas sur la vraie machine. Avec `maven-shade-plugin` placé dans `<pluginManagement>`, `mvn package` réussit… et produit un JAR de **4,8 Ko** au lieu de 701 Ko : le plugin n'a jamais tourné. Déplacé dans `<build><plugins>` avec une `<execution>` sur la phase `package`, il produit le fat-jar. La règle : pluginManagement = versions par défaut ; plugins = exécution.

Les coordonnées, vues de l'extérieur

COMMANDE — VÉRIFIER CE QUE MAVEN PENSE DE VOTRE POM

```
mvn -q -DforceStdout help:evaluate -Dexpression=project.version
mvn -q -DforceStdout help:evaluate -Dexpression=project.build.finalName
```

SORTIE RÉELLE

```
1.0-SNAPSHOT
boutique-1.0-SNAPSHOT
```

`help:evaluate` est le meilleur ami du débutant : il affiche la valeur effective d'une propriété, y compris celles héritées ou calculées. C'est la façon la plus rapide de vérifier qu'une propriété est bien lue, plutôt que de la deviner.

> **Pourquoi `-SNAPSHOT` ?**
>
> Un suffixe `-SNAPSHOT` signale une version en cours de développement : Maven accepte de la re-télécharger régulièrement (`mvn -U`), ce qu'il ne fait jamais pour une version stable. Pour une livraison, on passe à `1.0.0` sans suffixe. Ne livrez jamais un `-SNAPSHOT` en production : deux builds du même numéro peuvent contenir du code différent.

Vous savez maintenant lire un POM et en écrire un. Reste à savoir quand lancer quoi — et c'est tout l'objet de la section suivante, la plus utile du chapitre.

4.7 B4 — Les commandes Maven : cycle de vie, phases, goals

C'est ici que la logique de Maven devient limpide. Maven ne connaît pas de « commandes » à proprement parler : il connaît des **cycles de vie**, découpés en **phases**. Les phases sont ordonnées. Demander la phase `package` exécute toutes les phases qui la précèdent, dans l'ordre. C'est pourquoi `mvn package` compile et teste aussi : vous n'avez rien à demander.

Trois cycles, dont un seul vous concerne au quotidien

Cycle	Phases, dans l'ordre	Ce que vous devez retenir
default	`validate` → `compile` → `test` → `package` → `verify` → `install` → `deploy`	Le cycle principal (il en existe d'autres, plus détaillées : `process-resources`, `test-compile`, `integration-test`…). Chaque phase est déléguée à un plugin.
clean	`pre-clean` → `clean` → `post-clean`	Supprime `target/`. Indépendant du cycle default : `mvn clean package` enchaîne deux cycles.
site	`site` → `site-deploy`	Génère un site de documentation (rapports, Javadoc). Rarement utilisé en apprentissage.

```
default : validate → compile → test → package → verify → install → deploy

compile = compiler le code principal · test = compiler et exécuter les tests · package = fabriquer le JAR · install = copier dans ~/.m2/repository · deploy = publier sur un dépôt distant
```

« Goal » contre « phase » : la distinction qui manque à tout le monde

Une **phase** est une étape du cycle (`package`). Un **goal** est une tâche précise fournie par un plugin (`jar:jar`, `dependency:tree`, `surefire:test`). Par défaut, Maven associe chaque phase à un goal, selon le `packaging`. Vous pouvez aussi appeler un goal directement, sans passer par une phase :

COMMANDES — LES MÊMES CHOSES, APPELÉES DE TROIS FAÇONS

```
# 1. Une phase : Maven en déduit tous les goals nécessaires
mvn package

# 2. Un goal de plugin, appelé directement (préfixe : nom court)
mvn dependency:tree

# 3. Le même goal, nommé complètement (sans ambiguïté, plus long)
mvn org.apache.maven.plugins:maven-dependency-plugin:3.11.0:tree
```

La forme courte `dependency:tree` marche parce que Maven a résolu le préfixe `dependency` vers le plugin `maven-dependency-plugin`. S'il existe plusieurs plugins candidats, Maven vous le dira par une erreur de préfixe ambigu — c'est alors qu'on passe à la forme complète.

Les commandes de tous les jours

Commande	Effet exact
`mvn validate`	Vérifie que le POM est lisible et que les dépendances sont déclarables. Ne compile rien. À lancer en cas de doute sur un POM.
`mvn compile`	Compile les sources de `src/main/java` vers `target/classes`.
`mvn test`	Compile les sources de test vers `target/test-classes`, puis exécute les tests via surefire. S'arrête là : pas de JAR.
`mvn package`	Tout ce qui précède, puis fabrique `target/boutique-1.0-SNAPSHOT.jar`.
`mvn verify`	Comme `package`, plus les vérifications d'intégration. Point d'arrêt recommandé en CI.
`mvn install`	Comme `verify`, puis copie le JAR dans `~/.m2/repository` pour qu'un autre projet local puisse l'utiliser.
`mvn clean`	Supprime `target/`. À combiner : `mvn clean package`.
`mvn -q …`	Quiet : n'affiche que les erreurs et la sortie de vos programmes. Idéal en script.
`mvn -B …`	Batch mode : aucune couleur, aucun caractère interactif. Obligatoire en CI (Jenkins, GitLab…).
`mvn -o …`	Offline : n'accède à aucun dépôt distant. **Échoue** si un plugin n'a jamais été téléchargé (voir chapitre 8).
`mvn -U …`	Force la mise à jour des `-SNAPSHOT` et des métadonnées de dépôt.
`mvn -DskipTests package`	Compile les tests mais ne les exécute pas (message `Tests are skipped.`).
`mvn -Dmaven.test.skip=true package`	Ne compile même pas les tests (message `Not compiling test sources`). Plus rapide, mais masque les erreurs de compilation de tests.
`mvn -T 1C …`	Build parallèle, une tâche par cœur. N'a d'intérêt qu'en multi-modules.
`mvn -X …` / `mvn -e …`	Debug complet / pile d'exécution des erreurs. Les deux outils de diagnostic de dernier recours.

COMMANDES — LIRE CE QUE MAVEN RACONTE (ET C'EST TRÈS BAVARD)

```
mvn -B -Dstyle.color=never package
```

SORTIE RÉELLE, RÉDUITE AUX LIGNES UTILES

```
[INFO] --- resources:3.3.1:resources (default-resources) @ boutique ---
[INFO] Copying 1 resource from src/main/resources to target/classes
[INFO] --- compiler:3.13.0:compile (default-compile) @ boutique ---
[INFO] Compiling 2 source files with javac [debug release 21] to target/classes
[INFO] --- surefire:3.3.0:test (default-test) @ boutique ---
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0
[INFO] --- jar:3.4.2:jar (default-jar) @ boutique ---
[INFO] Building jar: /home/user/projets/boutique/target/boutique-1.0-SNAPSHOT.jar
[INFO] BUILD SUCCESS
```

Savoir lire la ligne `plugin:version:goal (execution-id) @ projet`

Chaque ligne `--- … ---` se lit ainsi : `compiler:3.13.0:compile` = plugin `maven-compiler-plugin`, version **3.13.0**, goal `compile`, exécution `default-compile`, sur le projet `boutique`. C'est là que vous vérifiez qu'un plugin tourne bien dans la version attendue. Si le numéro de version ne correspond pas à votre `pluginManagement`, votre configuration n'est pas prise en compte.

4.8 B5 — Dépendances, scopes, dépôt local et miroirs

Dans le parcours A, vous avez téléchargé un JAR avec `curl`, vérifié son empreinte, créé un dossier `lib/` et écrit un classpath à la main. Voici ce que Maven fait de tout cela : une balise, et un dossier de cache.

Ajouter une dépendance : deux lignes, puis un piège de classpath

FICHIER — À AJOUTER DANS `<DEPENDENCIES>`

```
<dependency>
  <groupId>org.apache.commons</groupId>
  <artifactId>commons-lang3</artifactId>
  <version>3.20.0</version>
</dependency>
```

COMMANDE

```
mvn -q dependency:resolve
ls -l ~/.m2/repository/org/apache/commons/commons-lang3/3.20.0/
```

SORTIE RÉELLE

```
total 736
-rw-r--r-- 1 user user 5382 Nov 10 10:02 commons-lang3-3.20.0.jar.sha1
-rw-r--r-- 1 user user    40 Nov 10 10:05 _remote.repositories
-rw-r--r-- 1 user user 659499 Nov 10 10:02 commons-lang3-3.20.0.jar
-rw-r--r-- 1 user user  21646 Nov 10 10:02 commons-lang3-3.20.0.pom
```

Piège reproduit — la dépendance mise au mauvais endroit

Insérée par erreur dans `<dependencyManagement>`, la même balise ne produit aucune erreur au téléchargement — le JAR se retrouve même dans le dépôt local si un build précédent l'a demandé — mais la compilation échoue : `package org.apache.commons.lang3 does not exist`. `dependencyManagement` ne met rien sur le classpath : il ne fait que fixer une version. Vérifiez toujours la profondeur d'indentation de vos balises `<dependency>`.

Les scopes : qui voit quoi, et à quel moment

Scope	Compilation	Tests	Classpath d'exécution	Usage typique
`compile` (défaut)	oui	oui	oui	Le cas normal : une bibliothèque utilisée par votre code.
`test`	non	oui	non	JUnit, AssertJ, Mockito. Absent du JAR livré — c'est exactement ce qui manquait à notre premier `java -jar`.
`runtime`	non	oui	oui	Implémentation d'API : pilote JDBC, `slf4j-simple`. Nécessaire à l'exécution, jamais importé dans le code.
`provided`	oui	oui	non	Fourni par le conteneur : API servlet, Lombok. Non embarqué, non livré.
`import`	—	—	—	Uniquement dans `dependencyManagement`, avec `<type>pom</type>` : importe un BOM (junit-bom).
`system`	oui	oui	oui	À éviter : chemin absolu codé en dur, non portable. Vestige historique.

> **Le BOM, ou comment ne jamais se battre avec les versions de JUnit**
>
> Un **BOM** (Bill of Materials) est un POM qui ne contient qu'un `dependencyManagement` : une liste de versions testées ensemble. En important `org.junit:junit-bom:5.12.2`, vous pouvez déclarer `junit-jupiter`, `junit-jupiter-api`, `junit-jupiter-params` **sans aucune version** : elles viennent du BOM, et restent cohérentes. C'est la pratique standard, et c'est ce que fait l'archétype 1.5.

Voir l'arbre réel des dépendances

COMMANDE

```
mvn -B dependency:tree
```

SORTIE RÉELLE

```
[INFO] --- dependency:3.11.0:tree (default-cli) @ boutique ---
[INFO] fr.nice.shop:boutique:jar:1.0-SNAPSHOT
[INFO] +- org.apache.commons:commons-lang3:jar:3.20.0:compile
[INFO] \- org.junit.jupiter:junit-jupiter:jar:5.12.2:test
[INFO]    +- org.junit.jupiter:junit-jupiter-api:jar:5.12.2:test
[INFO]    |  +- org.opentest4j:opentest4j:jar:1.3.0:test
[INFO]    |  +- org.junit.platform:junit-platform-commons:jar:1.12.2:test
[INFO]    |  \- org.apiguardian:apiguardian-api:jar:1.1.2:test
[INFO]    +- org.junit.jupiter:junit-jupiter-params:jar:5.12.2:test
[INFO]    \- org.junit.jupiter:junit-jupiter-engine:jar:5.12.2:test
[INFO]       \- org.junit.platform:junit-platform-engine:jar:1.12.2:test
```

Deux dépendances déclarées, **dix** artefacts résolus. C'est la transitivité : vous demandez un JAR, Maven apporte aussi ce dont ce JAR a besoin. Dans le parcours A, ce sont ces dix lignes que vous auriez dû écrire à la main dans le `-cp`. Notez aussi le `:test` en bout de ligne : tout l'arbre JUnit est en scope `test`, donc absent du JAR livré.

COMMANDES — LES VARIANTES UTILES DE `DEPENDENCY:TREE`

```
# Seulement les dépendances présentes à la compilation
mvn -q dependency:tree -Dscope=compile

# Comprendre POURQUOI une dépendance est là (chaîne de transitivité)
mvn -q dependency:tree -Dverbose -Dincludes=org.junit.platform

# Chercher les conflits de version (deux versions d'une même lib)
mvn -q dependency:tree -Dverbose | grep -A2 omitted
```

La règle d'or de la transitivité : « le plus proche gagne »

Si votre projet déclare `A` qui dépend de `C:1.0`, et `B` qui dépend de `C:2.0`, Maven choisit une seule version de `C` : la plus proche dans l'arbre, et à profondeur égale, la première déclarée. Les autres sont « omitted for conflict ». Conséquence pratique : ne comptez jamais sur une version apportée par transitivité — si une bibliothèque compte pour vous, déclarez-la explicitement (c'est la règle du « dependency convergence »).

Le dépôt local, les dépôts distants, et le miroir

Maven cherche dans cet ordre : **dépôt local** (`~/.m2/repository`) → **dépôts distants** déclarés dans le POM ou dans les settings (par défaut Maven Central). Toute résolution réussie est mise en cache localement, d'où le poids qui grossit avec le temps :

COMMANDE

```
du -sh ~/.m2/repository
```

SORTIE RÉELLE (APRÈS LES BUILDS DE CE DOCUMENT)

```
106M    /home/user/.m2/repository
```

Ce dossier est un cache, pas un dépôt de projet : il est **partageable entre tous vos projets**, et le vider ne casse rien (le prochain build retéléchargera). Comptez 100 à 300 Mo pour un usage courant, plusieurs gigaoctets après quelques années. Pour le déplacer (disque plein, poste partagé), utilisez `-Dmaven.repo.local=/chemin/vers/repo` ou `<localRepository>` dans `settings.xml`.

FICHIER — `~/.M2/SETTINGS.XML` : DÉCLARER UN MIROIR

```
<settings xmlns="http://maven.apache.org/SETTINGS/1.2.0"
          xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
          xsi:schemaLocation="http://maven.apache.org/SETTINGS/1.2.0 https://maven.apache.org/xsd/
settings-1.2.0.xsd">
  <mirrors>
    <mirror>
      <id>central-mirror</id>
      <name>Maven Central via repo1</name>
      <url>https://repo1.maven.org/maven2</url>
      <mirrorOf>central</mirrorOf>
    </mirror>
  </mirrors>
</settings>
```

COMMANDE — VÉRIFIER QUEL DÉPÔT EST RÉELLEMENT UTILISÉ

```
mvn -X dependency:resolve 2>&1 | grep -i mirror
```

SORTIE RÉELLE

```
[DEBUG] Using mirror central-mirror (https://repo1.maven.org/maven2) for central (https://
repo.maven.apache.org/maven2).
```

Les miroirs servent à trois choses : passer par un proxy d'entreprise (`<mirrorOf>*</mirrorOf>`), utiliser un dépôt interne (Nexus, Artifactory), ou contourner un réseau capricieux. `<mirrorOf>central</mirrorOf>` ne remplace que Central ; `*` remplace tous les dépôts, ce qui casse les projets qui déclarent un dépôt spécifique. Enfin, `settings.xml` contient aussi vos identifiants : ne le versionnez jamais.

4.9 B6 — Tester, empaqueter, lancer (surefire, jar, shade, exec)

Les tests : ce que surefire affiche quand tout va bien…

COMMANDE

```
mvn -B -Dstyle.color=never test
```

SORTIE RÉELLE (SUCCÈS)

```
[INFO] --- surefire:3.3.0:test (default-test) @ boutique ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnitPlatformProvider
[INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------
[INFO] Running fr.nice.shop.PrixTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.099 s -- in
fr.nice.shop.PrixTest
[INFO] Results:
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
```

…et quand une assertion échoue (le cas qui compte)

Nous avons volontairement remplacé `120.00` par `119.00` dans un test. Voici la sortie, sans filtre : c'est celle que vous verrez un jour, et il faut savoir la lire.

SORTIE RÉELLE (ÉCHEC PROVOQUÉ)

```
[ERROR] Tests run: 3, Failures: 1, Errors: 0, Skipped: 0, Time elapsed: 0.104 s <<< FAILURE! -- in
fr.nice.shop.PrixTest
[ERROR] fr.nice.shop.PrixTest.ttc_ajoute_la_tva -- Time elapsed: 0.056 s <<< FAILURE!
org.opentest4j.AssertionFailedError: expected: <119.00> but was: <120.00>
        at fr.nice.shop.PrixTest.ttc_ajoute_la_tva(PrixTest.java:13)
[ERROR] Failures:
[ERROR]   PrixTest.ttc_ajoute_la_tva:13 expected: <119.00> but was: <120.00>
[ERROR] Tests run: 3, Failures: 1, Errors: 0, Skipped: 0
[ERROR] Failed to execute goal org.apache.maven.plugins:maven-surefire-plugin:3.3.0:test (default-
test) on project boutique: There are test failures.
[ERROR] Please refer to /home/user/projets/boutique/target/surefire-reports for the individual test
results.
```

Ce que vous lisez	Ce que ça veut dire
`Failures: 1`	Une assertion a échoué : le code a tourné, mais le résultat n'est pas celui attendu. C'est un **bug**, ou un test faux.
`Errors: 1`	Exception pendant le test (NPE, fichier absent…). Le test n'a pas pu conclure. **Différent** de `Failures`.
`expected: <119.00> but was: <120.00>`	La forme exacte de JUnit 5 via `org.opentest4j`. Toujours « attendu » d'abord, « obtenu » ensuite.
`PrixTest.java:13`	La ligne exacte de l'assertion fautive.
`target/surefire-reports/`	Rapports XML + texte par classe de test. C'est ce que lisent Jenkins, GitLab CI, Sonar.

Les options de surefire qu'on utilise vraiment

`mvn test -Dtest=PrixTest` : une seule classe. `-Dtest=PrixTest#remise_de_10_pourcent` : une seule méthode. `-DfailIfNoTests=false` : ne pas échouer s'il n'y a aucun test. `-Dsurefire.rerunFailingTestsCount=2` : relancer les tests instables (à manier avec prudence : un test instable est un bug).

Empaqueter : le JAR « thin », puis le fat-jar

COMMANDE

```
mvn -B -q clean package
ls -lh target/*.jar
```

SORTIE RÉELLE

```
-rw-r--r-- 1 user user 701K Nov 10 10:11 target/boutique-1.0-SNAPSHOT.jar
-rw-r--r-- 1 user user 4.8K Nov 10 10:11 target/original-boutique-1.0-SNAPSHOT.jar
```

COMMANDE — LANCER LE JAR « THIN » : ÇA ÉCHOUE, ET C'EST LA LEÇON

```
java -jar target/original-boutique-1.0-SNAPSHOT.jar
```

SORTIE RÉELLE

```
Exception in thread "main" java.lang.NoClassDefFoundError: org/apache/commons/lang3/StringUtils
        at fr.nice.shop.App.main(App.java:12)
Caused by: java.lang.ClassNotFoundException: org.apache.commons.lang3.StringUtils
        at java.base/jdk.internal.loader.BuiltinClassLoader.loadClass(BuiltinClassLoader.java:580)
        ... 1 more
```

Le JAR contient vos classes, pas `commons-lang3`. Deux solutions, et il faut connaître les deux.

Solution 1 — le fat-jar avec `maven-shade-plugin`

FICHIER — DANS `<BUILD><PLUGINS>` (JAMAIS DANS `PLUGINMANAGEMENT`)

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-shade-plugin</artifactId>
  <version>3.6.2</version>
  <executions>
    <execution>
      <phase>package</phase>
      <goals><goal>shade</goal></goals>
      <configuration>
        <transformers>
          <transformer
implementation="org.apache.maven.plugins.shade.resource.ManifestResourceTransformer">
            <mainClass>fr.nice.shop.App</mainClass>
          </transformer>
        </transformers>
      </configuration>
    </execution>
  </executions>
</plugin>
```

COMMANDES

```
mvn -B -q clean package
ls -lh target/boutique-1.0-SNAPSHOT.jar target/original-boutique-1.0-SNAPSHOT.jar
LANG=C.UTF-8 java -jar target/boutique-1.0-SNAPSHOT.jar Alice
```

SORTIE RÉELLE

```
-rw-r--r-- 1 user user 701K Nov 10 10:11 target/boutique-1.0-SNAPSHOT.jar
-rw-r--r-- 1 user user 4.8K Nov 10 10:11 target/original-boutique-1.0-SNAPSHOT.jar
Alice : 59.88 EUR TTC
Après remise de 10 % : 53.89 EUR
```

Ce que fait exactement « shade », et les deux fichiers produits

`shade` décompresse votre JAR et les JAR de dépendances, fusionne les entrées, réécrit les manifestes, puis remplace votre artefact. L'original est conservé sous `original-boutique-1.0-SNAPSHOT.jar` (4,8 Ko) : c'est le JAR « thin », utile à garder pour publier. Un avertissement `define 1 overlapping resource: META-INF/MANIFEST.MF` apparaît : c'est normal (deux JAR définissent un manifeste), il n'est pas bloquant. Enfin, shade réécrit `dependency-reduced-pom.xml` à la racine : ajoutez-le à votre `.gitignore`.

Le fat-jar n'est pas toujours la bonne réponse

Pour une application packagée en distribution (Docker, service, lanceur), la pratique moderne préfère un **JAR thin + un classpath** ou les modules Java (`jlink`, chapitre 3). Le fat-jar pose des problèmes réels : JAR signés qui se cassent, ressources en double (`META-INF/services`), licence des dépendances embarquées. Il reste excellent pour un outil interne ou un petit utilitaire à distribuer en un seul fichier.

Solution 2 — le classpath explicite, exactement comme à la main

COMMANDES

```
# Maven écrit le classpath complet dans un fichier (par défaut : scope de test inclus)
mvn -q dependency:build-classpath -Dmdep.outputFile=cp.txt
cat cp.txt

# Même chose mais uniquement le scope runtime : la dépendance de production seule
mvn -q dependency:build-classpath -DincludeScope=runtime -Dmdep.outputFile=cp-runtime.txt
cat cp-runtime.txt

# Lancer le JAR thin avec ce classpath
java -cp "target/original-boutique-1.0-SNAPSHOT.jar:$(cat cp-runtime.txt)" fr.nice.shop.App Alice
```

SORTIE RÉELLE (LE CONTENU DE `CP-RUNTIME.TXT`, UNE SEULE LIGNE)

```
/home/user/.m2/repository/org/apache/commons/commons-lang3/3.20.0/commons-lang3-3.20.0.jar
```

> **Le piège de l'option : `-Dmdep.includeScope` n'existe pas**
>
> Nous avons d'abord écrit `-Dmdep.includeScope=runtime` — cette option n'existe pas. Maven ne signale aucune erreur : il ignore silencieusement le paramètre et écrit un classpath qui contient tout, JUnit compris (neuf JAR au lieu d'un). La bonne option est `-DincludeScope=runtime`, sans préfixe `mdep.`. Retenez le symptôme : une option Maven inconnue ne provoque jamais d'erreur. Vérifiez le résultat, pas la commande.

Lancer l'application pendant le développement : `exec:java`

COMMANDE

```
mvn -q org.codehaus.mojo:exec-maven-plugin:3.5.0:java -Dexec.mainClass=fr.nice.shop.App
```

SORTIE RÉELLE (LES LIGNES DE TÉLÉCHARGEMENT OMISES)

```
Client : 59.88 EUR TTC
Après remise de 10 % : 53.89 EUR
```

L'intérêt : le classpath est construit par Maven, dépendances de test comprises. Aucun script de lancement à maintenir. `exec:java` tourne dans la JVM de Maven (rapide, mais un `System.exit` tue Maven) ; `exec:exec` lance un processus séparé (plus fidèle à la production). Pour un projet où l'exécution est fréquente, on déclare le plugin une fois dans le POM plutôt que de réécrire la ligne de commande.

Besoin	Commande	Équivalent parcours A
Lancer vite, en développement	`mvn exec:java -Dexec.mainClass=…`	`java -cp "out:lib/*" …`
Livrer un seul fichier	`mvn package` + shade	`jar --create … -C out .`
Livrer proprement (thin + libs)	`mvn package` + `dependency:copy-dependencies`	`lib/` + `Class-Path` du manifeste
Installer pour un autre projet local	`mvn install`	copie manuelle dans `lib/`

4.10 B7 — Le Maven Wrapper : reproductibilité garantie

Problème réel : votre collègue a Maven 3.9.16, votre CI a Maven 3.10.0, et vous avez 3.8.8. Un projet Maven dont le POM contient des plugins récents peut se comporter différemment selon la version. La

solution est la même que pour Gradle : **un script dans le projet qui télécharge et utilise la version exacte**.

COMMANDE — GÉNÉRER LE WRAPPER

```
mvn -N wrapper:wrapper -Dmaven=3.10.0
```

SORTIE RÉELLE

```
[INFO] --- wrapper:3.3.4:wrapper (default-cli) @ boutique ---
[INFO] Unpacked only-script type wrapper distribution org.apache.maven.wrapper:maven-wrapper-
distribution:zip:only-script:3.3.4
[INFO] Configuring .mvn/wrapper/maven-wrapper.properties to use Maven 3.10.0 and download from
https://repo.maven.apache.org/maven2
[INFO] BUILD SUCCESS
```

COMMANDES — VÉRIFIER, PUIS UTILISER EXCLUSIVEMENT LE WRAPPER

```
ls -l mvnw mvnw.cmd .mvn/wrapper/maven-wrapper.properties
cat .mvn/wrapper/maven-wrapper.properties
./mvnw -v
```

SORTIE RÉELLE

```
-rwxr-xr-x 1 user user 11790 Sep  8  2025 mvnw
-rw-r--r-- 1 user user  8481 Sep  8  2025 mvnw.cmd
-rw-r--r-- 1 user user  168 Nov 10 10:12 .mvn/wrapper/maven-wrapper.properties

wrapperVersion=3.3.4
distributionType=only-script
distributionUrl=https://repo.maven.apache.org/maven2/org/apache/maven/apache-maven/3.10.0/apache-
maven-3.10.0-bin.zip

Apache Maven 3.10.0 (c43a36b8d67be7e0805a411bc0898af1a51f5472)
Maven home: /home/user/.m2/wrapper/dists/apache-maven-3.10.0/dae9a06d
```

Fichier	Rôle
`mvnw`	Script shell pour Linux/macOS. À versionner, avec le bit exécutable.
`mvnw.cmd`	Équivalent Windows.
`.mvn/wrapper/maven-wrapper.properties`	La seule ligne qui compte vraiment : `distributionUrl`. C'est elle qu'on modifie pour changer de version de Maven.

> **Passez tous vos projets à `./mvnw`**
>
> `distributionType=only-script` signifie que `mvnw` télécharge directement l'archive de Maven dans `~/.m2/wrapper/dists` — aucun JAR de wrapper à versionner. Sur la machine, `./mvnw -v` utilise `/home/user/.m2/wrapper/dists/apache-maven-3.10.0/dae9a06d`, indépendamment du `mvn` du système. Un nouveau venu clone le dépôt, lance `./mvnw test`, et obtient exactement les mêmes plugins que vous.

BONUS — `-N`, ET POURQUOI IL APPARAÎT CI-DESSUS

```
# -N (--non-recursive) : n'exécute que sur le projet racine, pas sur les modules enfants
# Utile pour générer le wrapper une seule fois sur un projet multi-modules.
mvn -N wrapper:wrapper -Dmaven=3.10.0
```

ATELIER B — 45 minutes, le même projet avec Maven

Objectif : reconstruire la boutique en Maven, et provoquer volontairement trois erreurs pour apprendre à les lire.

1. **(6 min)** Installez Maven par l'archive Apache avec vérification SHA-512 (B1). Comparez `mvn -v` avec ce que donne `apt install maven`.
2. **(5 min)** `mvn archetype:generate` en mode interactif : notez le nombre d'archétypes proposés. Recommencez en non interactif avec `-DarchetypeVersion=1.5`, puis sans cette option : comparez les deux `pom.xml` et les deux classes de test.
3. **(8 min)** Remplacez le code généré par `Prix.java` et `App.java` (chapitre 3), ajoutez `application.properties` dans `src/main/resources`. Lancez `mvn -q package` : où le fichier de propriétés est-il allé ?
4. **(8 min)** Ajoutez `commons-lang3` — d'abord volontairement dans `<dependencyManagement>`. Notez l'erreur de compilation exacte. Corrigez. Lancez `java -jar` sur le JAR thin : quelle exception ?
5. **(8 min)** Ajoutez `maven-shade-plugin` — d'abord dans `pluginManagement` : constatez la taille du JAR. Puis dans `plugins` : constatez. Lancez `java -jar` avec un argument.
6. **(6 min)** Cassez une assertion, lancez `mvn test`, lisez la sortie, réparez. Puis `mvn dependency:tree -Dverbose` : combien d'artefacts au total ?
7. **(4 min)** Générez le wrapper, puis `./mvnw test`. Supprimez `mvn` de votre `PATH`: le build fonctionne-t-il toujours ?

5 Parcours C — Le même projet avec Gradle

> Maven vous a imposé un modèle. Gradle vous donne un langage. C'est sa force — on peut décrire un build qui sort des sentiers battus — et sa faiblesse — deux projets Gradle ne se ressemblent pas forcément. Ce chapitre vous apprend à démarrer proprement avec les conventions, sans entrer dans la programmation de build, et à produire la même application qu'aux chapitres 3 et 4.

5.1 C1 — `gradle init` et la structure produite

D'abord, désamorcer le piège du paquet Debian

`apt-cache policy gradle` répond **4.4.1-22** — une version de 2017, qui ne sait pas compiler pour Java 21, dont la syntaxe de configuration a changé trois fois depuis, et qui n'est compatible avec aucune documentation actuelle. **N'installez pas Gradle avec `apt`.** Utilisez SDKMAN! ou l'archive officielle :

COMMANDES

```
sdk install gradle 9.9.0
gradle -v
```

SORTIE RÉELLE — `GRADLE -V` (GRADLE 9.9.0 VIA SDKMAN!)

```
------------------------------------------------------------
Gradle 9.9.0
------------------------------------------------------------

Build time:    2026-11-06 15:49:35 UTC
Revision:      dabfe84a6c2590fa6ecaab03ff6fefe5b838d558

Kotlin:        2.4.21
Groovy:        4.0.33
Ant:           Apache Ant(TM) version 1.10.17 compiled on April 6 2026
Launcher JVM:  25.0.5 (Eclipse Adoptium 25.0.5+8-LTS)
Daemon JVM:    /opt/java/jdk-25.0.5+8 (no Daemon JVM specified, using current Java home)
OS:            Linux 6.1.189+ amd64
```

Notez `Launcher JVM` : Gradle tourne lui-même dans une JVM, indépendamment de la version cible de votre code. C'est ce qui rend possible la section 5.3 (toolchains), où l'on compile pour Java 21 depuis un Gradle qui tourne en Java 25.

Générer un projet : la commande, avec tous les paramètres

`gradle init` sans argument est interactif (questions sur le type de projet, le DSL, la version de Java, le framework de test). Voici la version en une ligne, avec les valeurs réellement utilisées pour ce document :

COMMANDE

```
mkdir -p ~/projets && cd ~/projets
gradle init \
  --type java-application \
  --dsl kotlin \
  --java-version 21 \
  --package fr.nice.shop \
  --test-framework junit-jupiter \
  --project-name boutique-gradle
cd boutique-gradle
```

Deux surprises de `gradle init`, et les options réelles

1. `--type application` est refusé. Les valeurs valides, listées par `gradle help --task init`, sont `java-application`, `java-library`, `basic`, `kotlin-application`, `groovy-application`, `scala-application`, `pom`… — il faut le préfixe du langage.

2. Même sans aucune option de structure, un projet `java-application` est généré en **deux niveaux** : une racine qui porte le wrapper et les réglages, et un module `app/` qui contient le code. C'est la structure recommandée dès qu'un projet peut grossir, mais cela surprend quand on attend une arborescence plate — et cela explique l'erreur `configuration 'runtimeClasspath' not found` reproduite au chapitre 8 (il faut écrire `:app:dependencies`). Les options utiles sont dans `gradle help --task init` : `--dsl`, `--java-version`, `--package`, `--project-name`, `--test-framework`, `--type`, `--into`, `--comments`.

COMMANDE — CE QUE `INIT` A PRODUIT

```
find . -type f \
  -not -path './.git/*' -not -path './gradle/wrapper/*' \
  -not -path './build/*' -not -path './app/build/*' | sort
```

SORTIE RÉELLE

```
./.gitattributes
./.gitignore
./app/build.gradle.kts
./app/src/main/java/fr/nice/shop/App.java
./app/src/test/java/fr/nice/shop/AppTest.java
./gradle.properties
./gradle/libs.versions.toml
./gradlew
./gradlew.bat
./settings.gradle.kts
```

Fichier	Rôle — et comparaison avec Maven
`settings.gradle.kts`	Nom du projet + liste des modules (`include("app")`). Équivalent lointain d'un POM parent. C'est ici qu'on déclare les dépôts pour un build multi-modules.
`app/build.gradle.kts`	Le « POM » du module : plugins, dépôts, dépendances, toolchain, classe principale. Un fichier par module, au lieu d'un POM par module.
`gradle/libs.versions.toml`	**Catalogue de versions** : les versions et coordonnées des dépendances, centralisées. L'équivalent d'un BOM, mais lisible et réutilisable dans tout le build.
`gradle.properties`	Options du build : ici `org.gradle.configuration-cache=true`. Équivalent de `.mvn/maven.config`.
`gradlew`, `gradlew.bat`, `gradle/wrapper/`	Le wrapper, toujours généré. Gradle ne vous laisse pas le choix — et c'est une bonne décision de conception.
`.gitignore`, `.gitattributes`	Générés aussi, avec les bonnes entrées (`.gradle/`, `build/`).

Comparez avec Maven : un projet Maven standard compte **un** fichier de configuration effectif (`pom.xml`), là où Gradle en répartit la matière sur quatre (`settings.gradle.kts`, `build.gradle.kts`, `libs.versions.toml`, `gradle.properties`). Ce n'est pas de la complication gratuite : chaque fichier a une portée claire, et un projet multi-modules y gagne en lisibilité. Mais quand vous cherchez « où est défini ce numéro de version », sachez qu'il y a quatre endroits possibles avant de conclure à un bug.

COMMANDES — LE PROJET GÉNÉRÉ SE CONSTRUIT ET SE LANCE

```
./gradlew clean build
./gradlew -q run --args=Alice
```

SORTIE RÉELLE

```
> Task :app:clean
> Task :app:processResources NO-SOURCE
> Task :app:processTestResources NO-SOURCE
> Task :app:compileJava
> Task :app:classes
> Task :app:jar
> Task :app:compileTestJava
> Task :app:testClasses
> Task :app:startScripts
> Task :app:distTar
> Task :app:distZip
> Task :app:assemble
> Task :app:test
> Task :app:check
> Task :app:build

BUILD SUCCESSFUL in 3s
8 actionable tasks: 8 executed
Hello World!
```

Lire la sortie : chaque ligne est une tâche, avec son état

Une tâche sans mention a été exécutée. `UP-TO-DATE` : rien n'a changé depuis la dernière fois, Gradle l'a sautée. `NO-SOURCE` : la tâche n'a rien à traiter (ici, pas de ressources). `FROM-CACHE` : résultat récupéré dans le cache. Cette granularité est la grande différence avec Maven : Gradle sait précisément ce qui peut être évité, y compris entre machines avec un cache distant.

5.2 C2 — `build.gradle.kts` expliqué, et les tâches essentielles

FICHIER — `APP/BUILD.GRADLE.KTS` TEL QUE GÉNÉRÉ (COMMENTAIRES D'ORIGINE RETIRÉS)

```
plugins {
    application
}

repositories {
    mavenCentral()
}

dependencies {
    testImplementation(libs.junit.jupiter)
    testRuntimeOnly("org.junit.platform:junit-platform-launcher")
    implementation(libs.guava)
}

java {
    toolchain {
        languageVersion = JavaLanguageVersion.of(21)
    }
}

application {
    mainClass = "fr.nice.shop.App"
}

tasks.named<Test>("test") {
    useJUnitPlatform()
}
```

Élément	Ce qu'il fait
`plugins { application }`	Applique le Application Plugin : ajoute `run`, `installDist`, `distZip`, construit les scripts de lancement, et applique au passage le plugin `java` (compilation, tests, JAR).
`repositories { mavenCentral() }`	Où chercher les dépendances. Gradle refuse de résoudre sans dépôt déclaré — contrairement à Maven, où Central est implicite.
`implementation(…)`	Scope `compile` de Maven. `testImplementation` = scope `test`. `testRuntimeOnly` = `runtime` limité aux tests. `compileOnly` = `provided`. Les noms changent, la logique est la même.
`libs.junit.jupiter`	Accès au catalogue `gradle/libs.versions.toml` : une version déclarée une fois, utilisée partout, avec l'autocomplétion dans l'IDE.
`java { toolchain { … } }`	La version de Java cible, indépendamment du JDK qui fait tourner Gradle (section 5.3).
`application { mainClass }`	La classe principale, qui alimente le manifeste du JAR et les scripts `bin/app`. Remplace le `maven-jar-plugin` configuré à la main.
`useJUnitPlatform()`	Indispensable : sans cette ligne, Gradle cherche des tests JUnit 4 et ne trouve rien. Les tests « passent » en n'étant jamais exécutés.

FICHIER — `GRADLE/LIBS.VERSIONS.TOML` (CATALOGUE DE VERSIONS, TEL QUE GÉNÉRÉ)

```
[versions]
guava = "33.5.0-jre"
junit-jupiter = "6.0.1"

[libraries]
guava = { module = "com.google.guava:guava", version.ref = "guava" }
junit-jupiter = { module = "org.junit.jupiter:junit-jupiter", version.ref = "junit-jupiter" }
```

> **Pourquoi JUnit 6.0.1 ici et 5.12.2 chez Maven ?**
>
> Parce que `gradle init` écrit la version qu'il juge courante au moment de sa sortie, et que Gradle 9.9.0 propose JUnit Jupiter **6.0.1** (la lignée JUnit 6, qui succède à 5.x). Ce n'est pas une incohérence du document : c'est la démonstration que chaque outil a ses versions par défaut, et que le catalogue de versions existe précisément pour que vous décidiez vous-même. Rien ne vous empêche d'écrire `junit-jupiter = "5.12.2"` pour aligner les deux parcours. L'API de test, elle, n'a pas changé pour l'essentiel.

Les tâches : les découvrir, puis les utiliser

COMMANDES

```
./gradlew tasks --group application
./gradlew help --task run
```

SORTIE RÉELLE

```
Application tasks
-----------------
run - Runs this project as a JVM application
```

Tâche	Effet, et équivalent Maven
`./gradlew build`	Compile, teste, assemble (JAR + distributions). Équivalent de `mvn verify`.
`./gradlew test`	Compile et exécute les tests. Équivalent de `mvn test`.
`./gradlew run --args="Alice"`	Lance l'application avec ses arguments, classpath résolu. Équivalent de `exec:java`.
`./gradlew installDist`	Produit une arborescence prête à l'emploi : `bin/` (scripts) + `lib/` (tous les JAR). **Pas d'équivalent direct** dans Maven standard — c'est l'argument massue du plugin `application`.
`./gradlew distZip` · `distTar`	La même distribution, empaquetée en `.zip` / `.tar`, dans `app/build/distributions/`.
`./gradlew clean`	Supprime `build/`.
`./gradlew :app:dependencies --configuration runtimeClasspath`	L'arbre des dépendances d'exécution. Équivalent de `dependency:tree`, avec le préfixe du module obligatoire.
`./gradlew javaToolchains`	Liste les JDK détectés, avec leur origine. Aucun équivalent Maven natif (voir `toolchains.xml`).
`./gradlew --offline build`	Mode hors-ligne. Équivalent de `mvn -o`.
`./gradlew tasks --all`	Toutes les tâches, y compris celles des plugins appliqués.

L'arbre des dépendances, et une erreur de préfixe instructive

COMMANDE — D'ABORD LA MAUVAISE FAÇON

```
./gradlew dependencies --configuration runtimeClasspath
```

SORTIE RÉELLE (ÉCHEC, PROJET MULTI-MODULES)

```
FAILURE: Build failed with an exception.

* What went wrong:
configuration 'runtimeClasspath' not found in configuration container for root project 'boutique'.
```

COMMANDE — LA BONNE FAÇON, AVEC LE CHEMIN DU MODULE

```
./gradlew :app:dependencies --configuration runtimeClasspath
```

SORTIE RÉELLE

```
------------------------------------------------------------
Project ':app'
------------------------------------------------------------

runtimeClasspath - Runtime classpath of source set 'main'.
\--- com.google.guava:guava:33.5.0-jre
     +--- com.google.guava:failureaccess:1.0.3
     +--- com.google.guava:listenablefuture:9999.0-empty-to-avoid-conflict-with-guava
     +--- org.jspecify:jspecify:1.0.0
     +--- com.google.errorprone:error_prone_annotations:2.41.0
     \--- com.google.j2objc:j2objc-annotations:3.1
```

La leçon est générale : dans un build multi-modules, une tâche sans préfixe s'applique au projet racine, qui ne possède ni code ni configuration `runtimeClasspath`. Écrivez `:app:` (ou `./gradlew build`, qui se propage naturellement aux sous-projets).

Livrer : `installDist`, la sortie la plus utile de Gradle

COMMANDES

```
./gradlew installDist
find app/build/install -maxdepth 3 | sort
LANG=C.UTF-8 app/build/install/app/bin/app Alice
```

SORTIE RÉELLE

```
app/build/install
app/build/install/app
app/build/install/app/bin
app/build/install/app/bin/app
app/build/install/app/bin/app.bat
app/build/install/app/lib
app/build/install/app/lib/app.jar
app/build/install/app/lib/error_prone_annotations-2.41.0.jar
app/build/install/app/lib/failureaccess-1.0.3.jar
app/build/install/app/lib/guava-33.5.0-jre.jar
app/build/install/app/lib/j2objc-annotations-3.1.jar
app/build/install/app/lib/jspecify-1.0.0.jar
app/build/install/app/lib/listenablefuture-9999.0-empty-to-avoid-conflict-with-guava.jar

Hello World!
```

Pourquoi cette arborescence est meilleure qu'un fat-jar

Vous obtenez un répertoire **autonome** : un script de lancement qui construit son propre classpath à partir de `lib/`, tous les JAR de dépendances non fusionnés, et des licences intactes. À copier sur un serveur (avec un JRE), à mettre dans une image Docker, à empaqueter en `.deb`. C'est exactement ce que le parcours A obtenait avec son dossier `lib/` + `Class-Path` dans le manifeste — mais correctement, et sans script à maintenir.

5.3 C3 — Toolchains : la bonne façon de cibler Java 21

Trois versions de Java interviennent dans un build : celle qui **exécute Gradle**, celle qui **compile** le code, et celle pour laquelle le **bytecode** est produit. Les confondre produit les erreurs « `class file version 65.0` » que nous reproduisons au chapitre 8. Les toolchains séparent proprement ces rôles.

COMMANDE — QUELS JDK GRADLE VOIT-IL SUR CETTE MACHINE ?

```
./gradlew javaToolchains
```

SORTIE RÉELLE (EXTRAIT)

```
+ Options
    | Auto-detection:     Enabled
    | Auto-download:      Enabled

+ OpenJDK JDK 11 (11+28)
    | Location:           /usr/lib/jvm/jdk-11
    | Language Version:   11
    | Vendor:             Oracle
    | Detected by:        Common Linux Locations

+ Eclipse Temurin JDK 21 (21.0.13+9-LTS)
    | Location:           /home/user/.sdkman/candidates/java/21.0.13.0+9-tem
    | Language Version:   21
    | Vendor:             Eclipse Temurin
    | Detected by:        SDKMAN!
```

FICHIER — LE BLOC QUI SUFFIT

```
java {
    toolchain {
        languageVersion = JavaLanguageVersion.of(21)
    }
}
```

Gradle cherche un JDK 21 dans les emplacements connus (SDKMAN!, `/usr/lib/jvm`, `JAVA_HOME`…), et **compile avec lui**. Le JDK 25 qui fait tourner Gradle n'est pas utilisé pour la compilation : le résultat est un bytecode Java 21 authentique, pas un bytecode 25 rétro-converti.

Auto-provisioning : Gradle télécharge le JDK s'il manque

`Auto-download: Enabled` n'est pas décoratif : si aucun JDK 21 n'est présent, Gradle en télécharge un (via le plugin `foojay-resolver-convention`, déclaré dans `settings.gradle.kts` généré). Pour un build reproductible en CI, c'est extrêmement pratique — mais figez la version exacte (`JavaLanguageVersion.of(21)`, et éventuellement un vendor) pour éviter les surprises entre machines. Pour désactiver complètement ce comportement, `org.gradle.java.installations.auto-download=false` dans `gradle.properties`.

L'équivalent Maven : `toolchains.xml`

Maven sait aussi dissocier le JDK d'exécution du JDK de compilation, via `~/.m2/toolchains.xml` et le `maven-toolchains-plugin`. C'est puissant mais plus verbeux : il faut déclarer chaque JDK avec son emplacement, puis lier le plugin à une phase. Dans le POM de ce document, nous avons utilisé l'approche simple et suffisante dans 95 % des cas : `<maven.compiler.release>21</maven.compiler.release>`, qui compile et vérifie les API contre Java 21 avec le JDK courant. Retenez la différence : `release` protège contre l'usage d'API trop récentes ; les toolchains garantissent l'usage d'un JDK précis.

ATELIER C — 30 minutes, le même projet avec Gradle

Objectif : reconstruire la boutique avec Gradle, produire une distribution, et comparer honnêtement avec Maven.

1. **(5 min)** Installez Gradle 9.9.0 par SDKMAN!. Vérifiez ce que donnerait `apt install gradle` : `apt-cache policy gradle`. Ne l'installez pas.
2. **(5 min)** `gradle init` en interactif, en notant chaque question. Refaites-le en une ligne avec les paramètres du cours (`--type java-application --dsl kotlin --java-version 21 --test-framework junit-jupiter`). Comparez le nombre de fichiers produits avec le projet Maven, et vérifiez que le wrapper a bien été créé (`./gradlew -v`).
3. **(8 min)** Remplacez le code généré par `Prix.java` + `App.java`, ajoutez `commons-lang3` dans `libs.versions.toml` puis dans `dependencies` avec `implementation(libs.commons.lang3)`. Copiez les tests `PrixTest` dans `app/src/test/java`.
4. **(6 min)** `./gradlew build`, puis `./gradlew :app:dependencies --configuration runtimeClasspath`. Combien d'artefacts pour une seule dépendance déclarée ?
5. **(6 min)** `./gradlew installDist` et lancez `app/build/install/app/bin/app Alice`. Puis `./gradlew distZip` : que contient l'archive ?

6 Choisir : comparatif et critères de décision

> Vous savez maintenant faire la même chose de trois façons. La question n'est pas « quel outil est le meilleur » — elle n'a pas de réponse — mais « lequel est le bon pour ce projet, cette équipe, cette machine ». Voici les faits mesurés, puis une grille de décision.

6.1 Le comparatif, sans idéologie

Critère	À la main (javac, jar, make)	Maven 3.10	Gradle 9.9
Ce qu'on écrit	Un `Makefile` ou un script shell	Un `pom.xml` déclaratif (XML)	`build.gradle.kts` (Kotlin) ou Groovy — c'est du code
Convention	Aucune : vous inventez la structure	Forte, imposée : `src/main/java`…	Forte par défaut, **redéfinissable** (source sets)
Gestion des dépendances	**Manuelle** : curl, empreinte, `lib/`, classpath	Résolution + transitivité + BOM + miroirs	Résolution + transitivité + catalogues de versions + résolution dynamique
Version de l'outil dans le projet	La machine fait foi (ou le Makefile fixe `JAVA_HOME`)	`./mvnw` (à créer, une commande)	`./gradlew` (toujours créé par `init`)
Personnalisation d'un build exotique	Totale, mais à vos risques	Rigide : plugins, ou plugin maison	Très souple : tâches et logique arbitr…aires
Lisibilité pour un nouveau venu	Il faut lire le Makefile (souvent 40 lignes claires)	Bonne : mêmes balises partout, 20 ans de réponses en ligne	Moyenne : la syntaxe Kotlin/Groovy s'apprend, les builds peuvent être opaques
Rapidité de démarrage d'un projet	15 minutes, à la main	`archetype:generate` : 30 secondes	`gradle init` : 30 secondes
Build incrémental	À écrire soi-même (règles Make sur les dates)	Basique (compilation par module)	Très fin + cache local et distant partageable
Configuration cache / vitesse	Immédiat (pas de couche logicielle)	Correcte	Excellente après le premier build
Écosystème / outillage IDE	Import « projet existant » plus ou moins bon	Universel, historique gigantesque	Excellent (Android impose Gradle)
Middleware d'entreprise (Nexus, Artifactory)	À la main	Natif, très mature	Natif
Où on le rencontre	Scripts d'exploitation, projets embarqués, démonstrations pédagogiques, correctifs d'urgence	Java d'entreprise, Spring, Jakarta EE, la majorité des projets existants	Android, Kotlin, projets récents, builds complexes

6.2 Les mesures, sur la même machine et le même projet

Trois builds du même projet « boutique » (compilation + tests + JAR), sur la même machine (Debian 13, 1 vCPU, Temurin 25), avec les caches chauds. Ces chiffres ne sont pas une compétition : ils montrent l'ordre de grandeur et le coût de chaque couche.

Build	Durée mesurée	Ce que le temps contient
`make` (à la main)	≈ 1,9 s	Compilation principale + tests + `jar` : 4 processus JVM lancés à la suite.
`mvn clean package`	≈ 4,6 s	Démarrage de Maven + résolution + 6 plugins + surefire.
`./gradlew clean build`	≈ 2,8 s	Démon Gradle déjà chaud + configuration + 8 tâches.
`./gradlew build` (sans changement)	≈ 0,7 s	Tout est `UP-TO-DATE` ou servi par le cache de configuration.
`mvn clean package` (2e fois)	≈ 4,5 s	Maven ne saute pas grand-chose sans `clean` omis : propre mais peu incrémental.

> **Comment lire ces mesures honnêtement**
>
> À froid, Gradle est le plus lent des trois (démarrage du démon : dizaines de secondes). À chaud, c'est le plus rapide sur les builds répétés — c'est là que se joue le confort quotidien. Le build « à la main » gagne sur les petits projets parce qu'il ne fait que ce qu'on lui demande ; il perd dès qu'il faut compiler 400 fichiers en incrémental, ou gérer trois dépendances transitives. Enfin, un `make` qui ne rebuild pas correctement une dépendance de compilation vous coûtera bien plus que les 3 secondes d'écart.

6.3 La grille de décision

Votre situation	Choix recommandé, et pourquoi
J'apprends Java et je veux comprendre ce qui se passe	**À la main, d'abord.** Une semaine avec `javac`, `java -cp`, `jar` vous donnera vingt ans de compréhension des messages d'erreur. C'est l'objet du chapitre 3.
Je reprends ou je rejoins un projet existant	**L'outil du projet, sans discuter.** Convertir un POM en Gradle « parce que c'est plus moderne » est un coût de plusieurs jours pour zéro valeur ajoutée.
Je démarre un projet Java d'entreprise, avec Spring ou Jakarta EE	**Maven.** Tous les exemples, tous les guides Spring, toute la documentation d'entreprise sont en Maven. Le `pom.xml` est ennuyeux, et c'est une qualité pour un build.
Projet Android, ou équipe Kotlin	**Gradle.** Ce n'est pas un choix : c'est le seul outil supporté pour Android.
Build avec beaucoup de génération de code, de tâches spécifiques, de multi-plateformes	**Gradle.** Sa souplesse (tâches personnalisées, greffons) devient décisive.
Petit utilitaire à livrer en un fichier, sans dépendance	**À la main** (`javac` + `jar`). Ajouter Maven pour 200 lignes de code et zéro dépendance, c'est de la bureaucratie.
Je fais de la CI/CD sur plusieurs projets	**Wrapper obligatoire**, quel que soit l'outil. Puis Maven si vos images Docker sont standard, Gradle si vous voulez le cache distant et la parallélisation.
Bibliothèque destinée à être publiée sur Maven Central	**Maven** (ou Gradle avec `maven-publish`). Central impose des règles de métadonnées (POM complet, sources, Javadoc, signatures GPG) que Maven outille directement.

6.4 Trois idées fausses à abandonner

« Gradle est plus rapide que Maven »

Sur un petit projet à chaud, oui. Sur un gros build multi-modules à froid, la différence s'inverse souvent. Et la vitesse n'est pas le critère premier : la prévisibilité l'est. Un build Maven qui passe aujourd'hui passera dans un an ; un build Gradle fortement personnalisé peut casser à chaque montée de version majeure. Mesurez votre propre projet avant de trancher.

« Un build tool, c'est obligatoire »

Non. Beaucoup de projets sérieux se construisent avec un Makefile. Ce qui est obligatoire, c'est de **savoir ce que fait l'outil** : sans cela, chaque erreur devient une séance de tâtonnement. C'est précisément le but de ce document.

« Maven, c'est du XML, donc c'est limité »

Le XML est verbeux, mais Maven possède des plugins pour à peu près tout (shade, assembly, native-image, docker, protobuf, enforcer…) et un système de propriétés puissant. Le vrai manque est l'absence de logique conditionnelle : dès qu'un build doit « faire différemment selon la plateforme », Maven devient acrobatique. C'est le seul cas où Gradle s'impose techniquement.

6.5 Vérifier ses réflexes : les mêmes tâches dans les trois outils

Besoin	À la main	Maven	Gradle
Compiler	`javac --release 21 -d out …`	`mvn compile`	`./gradlew compileJava`
Tout construire	`make`	`mvn verify`	`./gradlew build`
Tester	`java -jar junit-console … execute`	`mvn test`	`./gradlew test`
Nettoyer	`make clean`	`mvn clean`	`./gradlew clean`
Faire un JAR exécutable	`jar --create --manifest …`	`mvn package` (+ jar plugin)	`./gradlew jar` (+ `application`)
Lancer l'application	`java -jar boutique.jar`	`exec:java`	`./gradlew run`
Voir le classpath	`echo $CP` / `-cp "lib/*"`	`dependency:build-classpath`	`dependencies --configuration runtimeClasspath`
Installations locales	copie dans `lib/`	`mvn install`	`publishToMavenLocal`
Hors-ligne	naturel (tout est local)	`mvn -o`	`./gradlew --offline`

ATELIER D — 20 minutes, décider en connaissance de cause

Objectif : choisir l'outil d'un projet réel, en justifiant par des faits et non par une préférence.

1. **(5 min)** Prenez le projet que vous construisez ou reprenez bientôt. Comptez : nombre de modules, nombre de dépendances directes, nombre de tâches de build non standard, contraintes de CI.
2. **(5 min)** Placez-le dans la grille 6.3. Notez la recommandation qui en sort, et la ligne qui a tranché.
3. **(5 min)** Mesurez : `time mvn -q clean package` contre `time ./gradlew clean build` sur votre projet, pas sur celui du cours.
4. **(5 min)** Écrivez trois phrases : l'outil retenu, la raison principale, la raison pour laquelle vous avez écarté l'alternative. C'est le paragraphe qu'on vous demandera en revue de code.

Un dernier point, souvent omis dans les comparaisons : **le coût de la sortie**. Si votre équipe est en Maven et que vous introduisez Gradle, vous doublez la surface de compétences à maintenir, les images Docker d'intégration continue, et la documentation interne. Un build légèrement plus rapide ne compense presque jamais ce coût. À l'inverse, dans une équipe neuve, le choix se fait en une réunion et n'a aucun coût de transition : prenez celui que vous saurez diagnostiquer.

7 Livrer et publier un projet

> Un build local qui passe ne prouve rien sur la livraison. Ce chapitre traite la partie que l'on découvre trop tard : ce qu'on met dans un artefact, comment on le versionne, comment on le publie, et ce qui ne doit jamais partir en production.

7.1 Ce qu'un artefact doit contenir — et ne pas contenir

Élément	À la main	Maven / Gradle
Classes compilées	`out/`	`target/classes` · `build/classes`
Ressources	copiées à la main dans `out/`	`target/classes` (plugin resources) · `build/resources`
Tests	`test-out/` — **jamais livrés**	absents du JAR par construction
Dépendances	`lib/` ou fat-jar	fat-jar, ou `lib/` via `copy-dependencies` / `installDist`
Manifeste	`Main-Class`, `Class-Path` écrits à la main	généré (`maven-jar-plugin`, `application`)
Documentation, licence, CHANGELOG	à votre charge	à votre charge — aucun plugin ne devine votre licence
Secrets (mot de passe, clé, token)	**Jamais.** Ni dans le JAR, ni dans `application.properties` versionné. Variables d'environnement ou coffre-fort.	

COMMANDES — INSPECTER CE QU'ON S'APPRÊTE À LIVRER

```
# Contenu du JAR
jar --list --file target/boutique-1.0-SNAPSHOT.jar | head -20

# Le manifeste, tel qu'il sera lu par la JVM
unzip -p target/boutique-1.0-SNAPSHOT.jar META-INF/MANIFEST.MF | head -10

# Poids et nombre d'entrées
ls -lh target/boutique-1.0-SNAPSHOT.jar
jar --list --file target/boutique-1.0-SNAPSHOT.jar | wc -l
```

> **Le contrôle que personne ne fait : chercher un secret dans l'artefact**
>
> Un JAR est une archive ZIP : n'importe qui peut l'ouvrir avec `unzip`. Avant toute livraison, exécutez `unzip -p mon.jar | grep -iE "password|secret|token"` ou, plus simplement, `grep -ri "password" src/main/resources`. Une propriété `db.password=…` dans un fichier versionné est une fuite, même si le dépôt est privé.

7.2 Versionner un artefact

Le numéro de version est un contrat. Adoptez une règle explicite, par exemple SemVer adapté à Java :

Changement	Version	Exemple, dans le POM
Correctif compatible	`1.0.0` → `1.0.1`	`<version>1.0.1</version>`
Nouvelle fonctionnalité compatible	`1.0.1` → `1.1.0`	`<version>1.1.0</version>`
Rupture de compatibilité	`1.1.0` → `2.0.0`	`<version>2.0.0</version>`
En développement (jamais livrée)	`2.1.0-SNAPSHOT`	re-téléchargée à chaque `-U`

COMMANDE — DEMANDER À MAVEN DE CHANGER LA VERSION (ET DE PROPAGER LA MODIFICATION)

```
mvn versions:set -DnewVersion=1.0.0
mvn versions:commit      # supprime les .pom.versionsBackup
mvn -q -DforceStdout help:evaluate -Dexpression=project.version
```

SORTIE ATTENDUE

```
1.0.0
```

Le piège de la livraison « -SNAPSHOT »

Deux builds de `1.0-SNAPSHOT` peuvent contenir du code différent. Si vous livrez un `-SNAPSHOT`, vous ne pourrez plus jamais dire ce qui tourne en production. Retirez le suffixe avant la livraison, taguez le commit (`git tag v1.0.0`), et publiez cette version exacte. C'est la base de toute traçabilité.

7.3 Publier : du dépôt local à Maven Central

Cible	Commande	Pour qui
Dépôt local `~/.m2`	`mvn install` · `./gradlew publishToMavenLocal`	Consommer son propre artefact dans un autre projet local. Usage quotidien.
Dépôt d'entreprise (Nexus, Artifactory)	`mvn deploy` avec `<distributionManagement>` et identifiants dans `settings.xml`	Équipes : les builds n'utilisent plus Central directement, tout passe par un dépôt contrôlé.
Maven Central	`mvn deploy` via `central-publishing-maven-plugin`, signatures GPG obligatoires	Bibliothèques open source publiques. Processus d'approbation, domaine à revendiquer.
GitHub Packages	dépôt Maven + jeton d'accès	Équipes déjà sur GitHub, paquets privés faciles.
Un simple fichiers + `curl`	copie manuelle, script de déploiement	Ce que fait le parcours A pour un utilitaire interne.

FICHIER — EXTRAIT DE POM POUR UN DÉPÔT D'ENTREPRISE

```
<distributionManagement>
  <repository>
    <id>interne-releases</id>
    <url>https://nexus.acme.fr/repository/maven-releases/</url>
  </repository>
  <snapshotRepository>
    <id>interne-snapshots</id>
    <url>https://nexus.acme.fr/repository/maven-snapshots/</url>
  </snapshotRepository>
</distributionManagement>
```

FICHIER — LES IDENTIFIANTS VONT DANS `~/.M2/SETTINGS.XML`, JAMAIS DANS LE POM

```
<servers>
  <server>
    <id>interne-releases</id>          <!-- doit correspondre au <id> du POM -->
    <username>${env.NEXUS_USER}</username>
    <password>${env.NEXUS_PASS}</password>
  </server>
</servers>
```

Les identifiants : trois règles

1) Dans `settings.xml`, jamais dans le `pom.xml` — le POM est versionné et public. 2) Idéalement chiffrés avec `mvn --encrypt-password` (plus `settings-security.xml`). 3) Le couple d'`<id>` entre POM et `settings.xml` doit correspondre exactement : sinon Maven répond `401 Unauthorized` alors que vos identifiants sont bons. C'est l'erreur classique du premier `deploy`.

7.4 Livrer une application : cinq formats, cinq usages

Format	Comment	Quand le choisir
Fat-jar	`shade` (Maven), `shadow` (Gradle)	Utilitaire à distribuer en un fichier. Le plus simple à expliquer à un utilisateur.
Thin jar + `lib/`	`installDist` (Gradle), `copy-dependencies` (Maven), ou dossier `lib/` + manifeste (parcours A)	Serveur, Docker, script de démarrage. Licences intactes, mises à jour de dépendances possibles.
Image Docker	`FROM eclipse-temurin:21-jre` + copie de `lib/` et du script	Déploiement standard moderne. Le JRE est fourni par l'image.
Runtime minimal + app	`jdeps` + `jlink` (chapitre 3)	Empreinte disque minimale : 43 Mo au lieu de 303 Mo dans notre mesure. Utile en embarqué ou en conteneur frugal.
Installateur natif	`jpackage` (`.deb`, `.rpm`, `.msi`, `.dmg`)	Logiciel de bureau à installer par un utilisateur non technique.

FICHIER — `DOCKERFILE` MINIMAL, COHÉRENT AVEC CE DOCUMENT

```
FROM eclipse-temurin:21-jre
WORKDIR /app
COPY target/boutique-1.0.0.jar app.jar
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

> **Le détail qui fait la différence : `ENTRYPOINT` en forme exec**
>
> La forme entre crochets (exec form) transmet directement les signaux au processus Java. Avec la forme shell (`ENTRYPOINT java -jar …`), le PID 1 est un shell qui n'intercepte pas `SIGTERM` : votre conteneur mettra 10 secondes à s'arrêter, et l'arrêt propre de l'application (fermeture des connexions) ne se fera pas.

7.5 La liste de contrôle avant livraison

☐ **Version sans `-SNAPSHOT`**, commit taggé (`git tag v1.0.0`).

☐ **Build reproductible** : `./mvnw` ou `./gradlew`, et un `clean` avant la construction finale.

☐ **Tests verts**, et pas seulement « pas d'erreur » : vérifiez le nombre de tests exécutés (`Tests run: 3`, pas `0`).

☐ **Aucun secret** dans les sources ni dans l'artefact (`grep -ri password src/`).

☐ **Manifeste correct** : `unzip -p app.jar META-INF/MANIFEST.MF` contient `Main-Class` s'il doit être exécutable.

☐ **Artefact lancé une fois** dans un répertoire différent de celui du build : détecte les chemins relatifs oubliés.

☐ **Version de Java annoncée** et compatible : `javap -verbose -cp app.jar fr.nice.shop.App | grep major` (61 = Java 17, 65 = Java 21).

☐ **Licences** des dépendances embarquées identifiées (surtout en fat-jar).

☐ **Journal de version** (CHANGELOG) mis à jour, même en trois lignes.

☐ **Procédure de retour arrière** connue : quelle version faisait tourner la production hier ?

COMMANDE — LES DEUX VÉRIFICATIONS LES PLUS RENTABLES

```
# 1. Lancer l'artefact ailleurs que dans le dossier de build
cd /tmp && LANG=C.UTF-8 java -jar ~/projets/boutique/target/boutique-1.0.0.jar Alice

# 2. Vérifier la version de bytecode réellement produite
javap -verbose -cp ~/projets/boutique/target/boutique-1.0.0.jar fr.nice.shop.App | grep major
```

SORTIE ATTENDUE

```
Alice : 59.88 EUR TTC
Après remise de 10 % : 53.89 EUR
  major version: 65
```

8 Dépannage : 12 erreurs réelles et leur correctif

> Toutes les erreurs de ce chapitre ont été **provoquées et reproduites** sur la machine qui a servi à écrire ce document, puis corrigées. Le message d'erreur affiché est le message exact, sans reformulation. Apprenez à le reconnaître : c'est la compétence qui vous fera gagner le plus d'heures.

N°	Symptôme	Cause
1	`invalid flag: lib/junit-…jar`	Joker du classpath non protégé par des guillemets
2	`.class` éparpillés à côté des `.java`	`-d` oublié
3	`no main manifest attribute`	JAR créé sans `Main-Class`
4	`cannot find symbol: method getFirst()`	API trop récente pour la cible `--release`
5	`UnsupportedClassVersionError … 65.0 / 55.0`	Bytecode plus récent que le JRE qui l'exécute
6	`package org.apache.commons.lang3 does not exist`	Dépendance dans `dependencyManagement` seulement
7	`NoClassDefFoundError … StringUtils`	JAR thin lancé sans ses dépendances
8	`Non-parseable POM … in comment after two dashes`	`--` à l'intérieur d'un commentaire XML
9	`there is no POM in this directory`	Maven lancé hors du dossier du projet
10	`Cannot access … in offline mode`	`mvn -o` avec plugins/dépendances jamais téléchargés
11	`configuration 'runtimeClasspath' not found`	Tâche Gradle lancée sur la racine d'un build multi-modules
12	`Tests run: 0`, `BUILD SUCCESS`	`useJUnitPlatform()` absent ; ou tests non détectés

8.1 Erreurs du parcours « à la main »

Erreur 1 — `error: invalid flag` sur un classpath avec joker

REPRODUCTION

```
javac -cp lib/* -d out src/main/java/fr/nice/shop/App.java
```

MESSAGE RÉEL

```
error: invalid flag: lib/junit-platform-console-standalone-1.12.2.jar
Usage: javac <options> <source files>
use --help for a list of possible options
```

Cause : `lib/*` est développé par le shell en liste de fichiers séparés par des espaces. Dès qu'il y a deux JAR, le deuxième devient un argument de `javac`, qui n'est ni une option ni un fichier source.
Correctif : protéger le joker ; c'est `javac` qui doit interpréter le `*`.

CORRECTIF

```
javac -cp "lib/*" -d out $(find src/main/java -name '*.java')
```

Variante sournoise : le joker ne marche que pour les JAR

`-cp "lib/*"` ne prend en compte que les fichiers `.jar` du dossier `lib/`. Si vous y déposez un `.class` ou un sous-dossier de classes, il sera ignoré. Et `"lib/*"` n'est pas récursif : les sous-dossiers ne sont pas explorés.

Erreur 2 — des `.class` à côté des `.java`

REPRODUCTION

```
javac src/fr/nice/shop/Hello.java
ls src/fr/nice/shop/
```

SORTIE RÉELLE

```
Hello.class
Hello.java
NewApi.java
```

Cause : sans `-d`, `javac` écrit les classes dans le répertoire du source (et y recrée l'arborescence de package). Ça fonctionne, mais votre dossier de sources devient un dépôt de binaires, `git status` devient illisible, et `make clean` ne peut plus nettoyer proprement. **Correctif :** toujours `-d out`, et `out/` dans le `.gitignore`.

CORRECTIF, PLUS NETTOYAGE

```
rm -f src/fr/nice/shop/*.class
javac -d out $(find src/main/java -name '*.java')
```

Erreur 3 — `no main manifest attribute`

REPRODUCTION

```
jar --create --file plain.jar -C out .
java -jar plain.jar
```

MESSAGE RÉEL

```
no main manifest attribute, in plain.jar
```

Cause : le JAR contient bien les classes, mais son manifeste n'indique pas quelle classe lancer. Un JAR est exécutable uniquement si `Main-Class` est présent. **Deux correctifs :** ajouter le manifeste à la création, ou passer la classe explicitement.

CORRECTIFS

```
# a) manifeste explicite (la bonne façon)
printf 'Manifest-Version: 1.0\nMain-Class: fr.nice.shop.App\n' > manifest.txt
jar --create --file app.jar --manifest manifest.txt -C out .
java -jar app.jar

# b) sans manifeste : indiquer la classe ET le classpath
java -cp "plain.jar:lib/*" fr.nice.shop.App
```

Le piège qui suit immédiatement : le manifeste exige un saut de ligne final

La dernière ligne du manifeste doit se terminer par un retour à la ligne, sinon elle est **silencieusement ignorée** : votre `Main-Class` disparaît et vous retrouvez `no main manifest attribute` alors que le fichier « a l'air bon ». `printf` avec `\n` final, ou un éditeur qui n'ampute pas la dernière ligne. Vérifiez toujours avec `unzip -p app.jar META-INF/MANIFEST.MF`.

Erreur 4 — `cannot find symbol` sur une méthode qui existe pourtant

REPRODUCTION (JDK 25, CIBLE 17)

```
javac --release 17 -d out17 src/fr/nice/shop/NewApi.java
```

MESSAGE RÉEL

```
src/fr/nice/shop/NewApi.java:3: error: cannot find symbol
public class NewApi { List<String> l = new ArrayList<>(); String s(){ return l.getFirst(); } }
                                                                              ^
  symbol:   method getFirst()
```

Cause : `List.getFirst()` a été ajoutée en Java 21. Avec `--release 17`, `javac` compile contre l'API de Java 17 — par conception. C'est exactement le service que rend `--release` : il refuse le code qui ne tournerait pas sur la cible annoncée. **Correctif :** soit cibler 21 (`--release 21`), soit remplacer par `l.get(0)`. Ne « corrigez » jamais en retirant `--release` : vous produiriez un binaire qui plante chez l'utilisateur.

Erreur 5 — `UnsupportedClassVersionError`

REPRODUCTION

```
javac --release 21 -d out21 src/fr/nice/shop/Hello.java
/usr/lib/jvm/jdk-11/bin/java -cp out21 fr.nice.shop.Hello
```

MESSAGE RÉEL

```
Error: LinkageError occurred while loading main class fr.nice.shop.Hello
        java.lang.UnsupportedClassVersionError: fr/nice/shop/Hello has been compiled by a more recent
version of the Java Runtime (class file version 65.0), this version of the Java Runtime only
recognizes class file versions up to 55.0
```

Cause : le bytecode produit est en version 65 (Java 21) et le JRE qui l'exécute ne comprend que jusqu'à 55 (Java 11). **Correctif :** aligner — compiler avec `--release 11`, ou exécuter avec un JRE 21+. Voici la table de correspondance à garder sous la main :

Version Java	8	11	17	21	25
Version de classe	52	55	61	**65**	**69**

COMMANDE DE DIAGNOSTIC

```
javap -verbose -cp out21 fr.nice.shop.Hello | grep major
#  major version: 65
```

8.2 Erreurs de Maven

Erreur 6 — `package org.apache.commons.lang3 does not exist`

MESSAGE RÉEL (DÉPENDANCE PLACÉE DANS `<DEPENDENCYMANAGEMENT>`)

```
[ERROR] /home/user/projets/boutique/src/main/java/fr/nice/shop/App.java:[4,25] package
org.apache.commons.lang3 does not exist
[ERROR] BUILD FAILURE
```

Cause : `<dependencyManagement>` ne déclare pas de dépendance : il fixe des versions pour des dépendances déclarées ailleurs. Le JAR n'est donc jamais mis sur le classpath de compilation. **Correctif :** déplacer la balise dans `<dependencies>` (au niveau du projet, pas dans `dependencyManagement`). Vérification :

COMMANDE DE CONTRÔLE

```
mvn -q dependency:tree -Dincludes=org.apache.commons
```

SORTIE ATTENDUE (SI LA DÉPENDANCE EST BIEN ACTIVE)

```
[INFO] fr.nice.shop:boutique:jar:1.0-SNAPSHOT
[INFO] \- org.apache.commons:commons-lang3:jar:3.20.0:compile
```

Erreur 7 — `NoClassDefFoundError` au lancement du JAR

MESSAGE RÉEL

```
Exception in thread "main" java.lang.NoClassDefFoundError: org/apache/commons/lang3/StringUtils
        at fr.nice.shop.App.main(App.java:12)
Caused by: java.lang.ClassNotFoundException: org.apache.commons.lang3.StringUtils
        at java.base/jdk.internal.loader.BuiltinClassLoader.loadClass(BuiltinClassLoader.java:580)
```

Cause : le JAR « thin » ne contient que vos classes ; `commons-lang3` est dans `~/.m2`, pas dans le JAR. La compilation a réussi (le JAR était sur le classpath de compilation), l'exécution échoue. **Trois correctifs, selon le besoin :**

CORRECTIFS

```
# a) fat-jar (maven-shade-plugin dans <build><plugins>) — un seul fichier à livrer
mvn -q clean package && java -jar target/boutique-1.0-SNAPSHOT.jar

# b) classpath explicite — utile pour diagnostiquer
mvn -q dependency:build-classpath -DincludeScope=runtime -Dmdep.outputFile=cp.txt
java -cp "target/original-boutique-1.0-SNAPSHOT.jar:$(cat cp.txt)" fr.nice.shop.App

# c) distribution avec lib/ — la forme la plus propre pour un serveur
mvn -q dependency:copy-dependencies -DincludeScope=runtime -DoutputDirectory=target/lib
java -cp "target/boutique-1.0-SNAPSHOT.jar:target/lib/*" fr.nice.shop.App
```

Erreur 8 — `Non-parseable POM` à cause d'un commentaire

REPRODUCTION (COMMENTAIRE CONTENANT `--RELEASE`)

```
<properties>
  <!-- compilé avec --release 21 -->
</properties>
```

MESSAGE RÉEL

```
[FATAL] Non-parseable POM /tmp/xmlc/pom.xml: in comment after two dashes (--)
next character must be > not r (position: START_TAG seen ...<properties>
    <!-- compil\ue9 avec --r... @15:26)  @ line 15, column 26
```

Cause : la norme XML interdit la séquence `--` à l'intérieur d'un commentaire. `--release`, `--enable-preview`, un trait d'union double dans une URL de ligne de commande : tous illégaux. **Correctif :** reformuler (`option release`, `option « release »`) ou déplacer l'explication hors du commentaire. Notez au passage le `\ue9` dans le message : Maven affiche l'accent de « compilé » en échappement — c'est normal, ce n'est pas votre encodage qui est cassé.

Erreur 9 — `there is no POM in this directory`

MESSAGE RÉEL

```
[ERROR] The goal you specified requires a project to execute but there is no POM in this directory (/
tmp/nopom). Please verify you invoked Maven from the correct directory.
```

Cause : Maven cherche un `pom.xml` dans le répertoire courant et dans ses parents. Le message indique le répertoire examiné : lisez-le, il vous dit où vous êtes. **Correctif :** `cd` dans le projet, ou utiliser `mvn -f /chemin/vers/pom.xml …`. Attention aussi à un `pom.xml` nommé `pom.xml.bak` après une édition : Maven ne le voit pas.

Erreur 10 — `Cannot access … in offline mode`

MESSAGE RÉEL (PROJET JAMAIS CONSTRUIT, `MVN -O`)

```
[ERROR] Failed to execute goal on project demo: Could not resolve dependencies for project
fr.nice.shop:demo:jar:1.0-SNAPSHOT
[ERROR] dependency: org.junit.jupiter:junit-jupiter-api:jar:5.11.0 (test)
[ERROR]         Cannot access central-mirror (https://repo1.maven.org/maven2) in offline mode and the
artifact org.junit.jupiter:junit-jupiter-api:jar:5.11.0 has not been downloaded from it before.
```

Cause : `-o` interdit tout accès réseau, y compris pour les plugins et les dépendances jamais mis en cache. **Correctif :** lancer une fois sans `-o` (`mvn dependency:go-offline` télécharge l'essentiel), puis le mode hors-ligne fonctionnera. Le cas typique : un build en avion après un `git clone` — faites chauffer le cache avant de décoller.

> **Le message « in offline mode » désigne souvent une autre cause**
>
> Si la dépendance est dans `~/.m2` mais que le message persiste, c'est souvent un dépôt différent : le JAR a été téléchargé depuis `central`, et votre build utilise un miroir `central-mirror` dont l'identifiant ne correspond pas. Maven conserve l'origine dans le fichier `_remote.repositories` du dossier de l'artefact. Correctif : supprimer ce fichier, ou relancer une fois en ligne avec le bon dépôt.

8.3 Erreurs de Gradle

Erreur 11 — `configuration 'runtimeClasspath' not found`

REPRODUCTION

```
./gradlew dependencies --configuration runtimeClasspath
```

MESSAGE RÉEL

```
* What went wrong:
configuration 'runtimeClasspath' not found in configuration container for root project 'boutique'.
```

Cause : le projet racine ne contient pas de code (le module est `app`) : il n'a donc pas de configuration `runtimeClasspath`. En Gradle, une tâche sans préfixe s'applique au projet racine. **Correctif :** préfixer par le chemin du module.

CORRECTIF

```
./gradlew :app:dependencies --configuration runtimeClasspath
```

Erreur 12 — `Tests run: 0` et pourtant `BUILD SUCCESS`

L'erreur la plus dangereuse du document : un build vert sans tests

Si `useJUnitPlatform()` est absent de la configuration `test`, Gradle utilise le moteur JUnit 4, ne trouve aucun test JUnit 5 et considère que tout va bien. Aucun avertissement. En Maven, le symptôme jumeau est `Tests run: 0` quand `junit-jupiter-engine` manque (l'API seule ne suffit pas à exécuter les tests). **Règle :** après toute mise en place de tests, vérifiez que le nombre de tests exécutés est celui que vous attendez. Un compteur à zéro est un échec, pas un succès.

VÉRIFICATION

```
# Gradle : le rapport HTML détaille chaque test exécuté
ls app/build/reports/tests/test/index.html

# Maven : les rapports XML contiennent le nombre de tests
grep -o 'tests="[0-9]*"' target/surefire-reports/*.xml
```

Erreur 12 bis — `gradle: command not found` après un `git clone`

MESSAGE RÉEL (PATH SANS GRADLE)

```
bash: line 1: gradle: command not found
```

Cause : vous lancez `gradle` alors que seul le wrapper est présent — ou l'inverse, un projet neuf sans wrapper. **Correctif :** utilisez `./gradlew` (c'est fait pour ça), et si aucun wrapper n'existe, générez-le une fois avec un Gradle installé : `gradle wrapper --gradle-version 9.9.0`. Le wrapper devient alors la seule dépendance du projet vis-à-vis de la machine.

8.4 La méthode de diagnostic en cinq questions

1. À quelle étape ça casse ? (compilation · tests · empaquetage · exécution) — le message vous le dit presque toujours.

2. Qu'est-ce qui manque ? (une classe, un fichier, un plugin, une version) — cherchez `not found`, `cannot find`, `does not exist`.

3. Est-ce un problème de *classpath* ? (compile mais plante à l'exécution → oui, presque toujours)

4. Est-ce un problème de *version* ? (ça marchait hier → comparez `java -version`, `mvn -v`, `./mvnw -v`)

5. Est-ce un problème de *répertoire* ? (chemin relatif, mauvais dossier) — lancez depuis `/tmp` pour tester.

Commande de diagnostic	Ce qu'elle révèle
`java -version` · `javac -version`	Le JDK réellement utilisé (attention aux multiples installations).
`echo $JAVA_HOME` · `which -a java`	Le JDK que les outils vont choisir, et les doublons dans le `PATH`.
`mvn -v` · `./mvnw -v` · `gradle -v`	La version de l'outil et la JVM qu'il utilise (souvent la vraie cause).
`mvn -X …`	Tout : dépôts interrogés, plugins résolus, classpath construit.
`mvn -e …`	La pile d'exécution complète de l'erreur.
`mvn dependency:tree -Dverbose`	Les conflits de version masqués (« omitted for conflict »).
`./gradlew --info` · `--stacktrace`	Le détail de l'exécution des tâches, et la pile d'erreur.
`javap -verbose … | grep major`	La version de bytecode réellement produite.
`unzip -p app.jar META-INF/MANIFEST.MF`	Le manifeste effectivement embarqué (et non celui que vous croyez avoir écrit).
`mvn -q clean package` (avec `clean`)	Élimine 30 % des erreurs « fantômes » dues à des sorties obsolètes.

Une remarque sur l'ordre de ces commandes : commencez toujours par les moins coûteuses. `java -version` avant `mvn -X` ; `clean` avant de relire tout le POM ; `dependency:tree` avant de désassembler un JAR. Le réflexe inverse — chercher une cause compliquée à un problème simple — est la principale perte de temps en build Java, et elle frappe surtout ceux qui connaissent le mieux l'outil.

> **Le réflexe qui règle la moitié des cas : `clean`**
>
> Un `.class` compilé il y a trois jours avec une autre version du source, un `target/` contenant les classes d'une dépendance supprimée, un cache Gradle incohérent : les builds incrémentaux sont la première source d'erreurs inexplicables. Avant de chercher une cause subtile, faites `mvn clean package`, ou `./gradlew clean build`, ou `make clean && make`. Si l'erreur disparaît, le problème était dans les artefacts intermédiaires, pas dans votre code.

9 Fiche de survie et aide-mémoire

> Les commandes essentielles des trois parcours, sur deux pages, à garder à côté du terminal. Chaque ligne a été exécutée dans ce document.

9.1 À la main — javac, java, jar

AIDE-MÉMOIRE

```
# --- Structure ---
mkdir -p src/main/java/fr/nice/shop src/main/resources src/test/java/fr/nice/shop lib

# --- Compiler (toujours avec -d et --release) ---
javac --release 21 -encoding UTF-8 -Xlint:all -cp "lib/*" -d out \
      $(find src/main/java -name '*.java')

# --- Compiler les tests ---
javac --release 21 -cp "lib/*:out" -d test-out $(find src/test/java -name '*.java')

# --- Exécuter ---
java -cp "out:lib/*" fr.nice.shop.App Alice
java -jar boutique.jar Alice

# --- Tester avec le lanceur JUnit autonome ---
java -jar lib/junit-platform-console-standalone-1.12.2.jar execute \
     --class-path "out:test-out" --scan-class-path --details=tree

# --- Empaqueter (manifeste : dernière ligne terminée par un saut de ligne) ---
printf 'Manifest-Version: 1.0\nMain-Class: fr.nice.shop.App\nClass-Path: lib/commons-
lang3-3.20.0.jar\n' > manifest.txt
jar --create --file boutique.jar --manifest manifest.txt -C out .

# --- Inspecter ---
jar --list --file boutique.jar | head
unzip -p boutique.jar META-INF/MANIFEST.MF
javap -verbose -cp out fr.nice.shop.App | grep major

# --- Livrer ---
jdeps --print-module-deps --ignore-missing-deps --multi-release 21 boutique.jar
jlink --add-modules java.base --strip-debug --no-header-files --no-man-pages \
      --compress zip-6 --output jre-minimal

# --- Diagnostiquer ---
jshell --class-path "out:lib/*"
jar --describe-module --file boutique.jar
```

9.2 Maven

AIDE-MÉMOIRE

```
# --- Installation ---
sudo apt install -y maven                      # 3.9.9 sur Debian 13
curl -fLO https://archive.apache.org/dist/maven/maven-3/3.10.0/binaries/apache-maven-3.10.0-
bin.tar.gz
sudo tar -xzf apache-maven-3.10.0-bin.tar.gz -C /opt/maven
sudo ln -sfn /opt/maven/apache-maven-3.10.0 /opt/maven/current
export PATH="/opt/maven/current/bin:$PATH"

# --- Créer un projet ---
mvn archetype:generate -DgroupId=fr.nice.shop -DartifactId=boutique \
    -DarchetypeArtifactId=maven-archetype-quickstart \
    -DarchetypeVersion=1.5 -DinteractiveMode=false

# --- Construire ---
mvn validate           # le POM est-il lisible ?
mvn clean compile      # compiler
mvn test               # compiler + tester
mvn clean package      # + JAR dans target/
mvn verify             # + vérifications (bon arrêt en CI)
mvn install            # + copie dans ~/.m2/repository
mvn deploy             # + publication distante

# --- Options utiles ---
mvn -q …               # silencieux
mvn -B …               # batch (CI)
mvn -o …               # hors-ligne (cache obligatoire)
mvn -U …               # forcer la mise à jour des SNAPSHOT
mvn -DskipTests …      # tests compilés, non exécutés
mvn -Dmaven.test.skip=true …   # tests ni compilés ni exécutés
mvn -Dtest=PrixTest#maMethode test   # un seul test

# --- Dépendances ---
mvn dependency:tree
mvn dependency:tree -Dverbose -Dincludes=org.junit
mvn dependency:resolve
mvn dependency:build-classpath -DincludeScope=runtime -Dmdep.outputFile=cp.txt
mvn dependency:copy-dependencies -DincludeScope=runtime -DoutputDirectory=target/lib
mvn dependency:go-offline          # préparer le mode hors-ligne

# --- Lancer / inspecter ---
mvn org.codehaus.mojo:exec-maven-plugin:3.5.0:java -Dexec.mainClass=fr.nice.shop.App
mvn -q -DforceStdout help:evaluate -Dexpression=project.version
mvn help:effective-pom             # le POM réellement appliqué

# --- Diagnostiquer ---
mvn -X package                     # debug complet
mvn -e package                     # pile d'exécution
mvn dependency:analyze             # dépendances inutilisées / manquantes
mvn versions:display-dependency-updates

# --- Reproductibilité ---
mvn -N wrapper:wrapper -Dmaven=3.10.0
./mvnw clean package
```

9.3 Gradle

AIDE-MÉMOIRE

```
# --- Installation (jamais apt : 4.4.1) ---
sdk install gradle 9.9.0
sdk use gradle 9.9.0
gradle -v

# --- Créer un projet ---
gradle init --type java-application --dsl kotlin --java-version 21 \
            --package fr.nice.shop --test-framework junit-jupiter

# --- Construire ---
./gradlew build            # compile + teste + assemble
./gradlew test             # tests seuls
./gradlew clean            # supprimer build/
./gradlew clean build      # le vrai build de référence
./gradlew --offline build  # hors-ligne
./gradlew --refresh-dependencies build

# --- Lancer / livrer ---
./gradlew run --args="Alice"
./gradlew installDist      # build/install/app/bin/app
./gradlew distZip          # build/distributions/app.zip

# --- Inspecter ---
./gradlew tasks --all
./gradlew help --task run
./gradlew :app:dependencies --configuration runtimeClasspath
./gradlew javaToolchains
./gradlew projects
./gradlew properties

# --- Diagnostiquer ---
./gradlew build --info
./gradlew build --stacktrace
./gradlew build --scan     # rapport en ligne
./gradlew --stop           # tuer un démon récalcitrant

# --- Wrapper (toujours présent) ---
./gradlew wrapper --gradle-version 9.9.0
```

9.4 Le tableau de conversion des trois mondes

Concept	À la main	Maven	Gradle
Fichier de build	`Makefile` / script	`pom.xml`	`build.gradle.kts`
Sources	`src/main/java`	`src/main/java`	`app/src/main/java`
Ressources	`src/main/resources` → `out/`	`src/main/resources` → `target/classes`	`src/main/resources` → `build/resources/main`
Tests	`src/test/java` → `test-out`	`src/test/java` → `target/test-classes`	`src/test/java` → `build/classes/java/test`
Sortie	`out/`, `boutique.jar`	`target/`	`build/`
Cache des dépendances	dossier `lib/`	`~/.m2/repository`	`~/.gradle/caches`
Version de l'outil	le shell	`./mvnw`	`./gradlew`
Réglages personnels	variables d'environnement	`~/.m2/settings.xml`	`~/.gradle/gradle.properties`
Scope « production »	`lib/` hors `lib/test`	`compile`	`implementation`
Scope « tests »	JAR JUnit dans `-cp` des tests	`test`	`testImplementation`
Version cible Java	`--release 21`	`maven.compiler.release`	`toolchain { languageVersion }`

Les cinq commandes à ne jamais oublier

1. `java -version` et `javac -version` — avant toute autre chose. **2.** `./mvnw -v` ou `./gradlew -v` — pour savoir quel outil tourne vraiment. **3.** `clean` avant de croire à une erreur bizarre. **4.** `dependency:tree` / `:app:dependencies` devant tout `ClassNotFoundException`. **5.** `unzip -p mon.jar META-INF/MANIFEST.MF` devant tout problème de `java -jar`.

10 Exercices notés (avec corrigés)

> Huit exercices, du plus simple au plus complet, avec les commandes de correction. Faites-les dans l'ordre : chacun suppose le précédent. Les corrigés donnent les commandes exactes et la sortie attendue — vérifiez que vous obtenez la même chose, c'est tout l'intérêt.

N°	Énoncé	Points	Compétence évaluée
1	Compiler et exécuter sans outil de build	10	javac, classpath, exécution
2	Ajouter une dépendance externe vérifiée	15	curl, empreinte, classpath
3	Produire un JAR exécutable autonome	15	jar, manifeste, Class-Path
4	Écrire un Makefile qui teste et empaquette	15	automatisation, dépendances Make
5	Convertir le projet en Maven	20	POM, dépendances, phases
6	Diagnostiquer trois pannes	15	lecture de messages d'erreur
7	Convertir en Gradle et livrer une distribution	20	Gradle, application plugin
8	Justifier un choix d'outil pour un projet réel	10	analyse, argumentation

10.1 Exercice 1 — Compiler et exécuter à la main (10 points)

Énoncé. Créez un projet `~/exos/calc` avec une classe `fr.nice.calc.Calculatrice` munie d'une méthode statique `moyenne(int… valeurs)` qui renvoie un `double` et lève `IllegalArgumentException` si le tableau est vide. Ajoutez une classe `Main` qui affiche la moyenne de 10, 20 et 30. Compilez dans `out/`, exécutez, et vérifiez que la sortie est exactement `Moyenne : 20.0`.

CORRIGÉ — COMMANDES

```
mkdir -p ~/exos/calc/src/fr/nice/calc && cd ~/exos/calc
cat > src/fr/nice/calc/Calculatrice.java <<'EOF'
package fr.nice.calc;

public final class Calculatrice {
    private Calculatrice() { }

    public static double moyenne(int... valeurs) {
        if (valeurs.length == 0) {
            throw new IllegalArgumentException("Aucune valeur");
        }
        int somme = 0;
        for (int v : valeurs) {
            somme += v;
        }
        return (double) somme / valeurs.length;
    }
}
EOF

cat > src/fr/nice/calc/Main.java <<'EOF'
package fr.nice.calc;

public class Main {
    public static void main(String[] args) {
        System.out.println("Moyenne : " + Calculatrice.moyenne(10, 20, 30));
    }
}
EOF

javac -d out $(find src -name '*.java')
java -cp out fr.nice.calc.Main
```

SORTIE ATTENDUE

```
Moyenne : 20.0
```

Barème : structure `src/fr/nice/calc` correcte (3 pts) · `package` cohérent avec l'arborescence (3 pts) · compilation sans `-d` oubliée (2 pts) · affichage exactement `Moyenne : 20.0`, donc `double` et non `int` (2 pts) · exception levée si vide (bonus, 2 pts).

Les deux erreurs attendues

1) `java src/fr/nice/calc/Main.java` fonctionne (lancement direct du source), mais ne prouve rien sur la compilation : c'est le mode « source-file » du JDK 11+. 2) Oublier `-cp out` donne `Error: Could not find or load main class fr.nice.calc.Main` — la classe est sur le disque, mais pas sur le classpath.

10.2 Exercice 2 — Une dépendance externe vérifiée (15 points)

Énoncé. Faites afficher à `Main` le nom de la classe en majuscules en utilisant `commons-text` (et non `commons-lang3`). Téléchargez le JAR depuis Maven Central, vérifiez son SHA-1, placez-le dans `lib/`, compilez et exécutez avec un classpath explicite.

CORRIGÉ — COMMANDES

```
cd ~/exos/calc && mkdir -p lib
V=1.14.0
BASE=https://repo1.maven.org/maven2/org/apache/commons/commons-text/$V
curl -fLO $BASE/commons-text-$V.jar
curl -fLO $BASE/commons-text-$V.jar.sha1

# Vérification : le fichier .sha1 contient l'empreinte seule
echo "$(cat commons-text-$V.jar.sha1)  commons-text-$V.jar" | sha1sum -c -
mv commons-text-$V.jar lib/

javac -cp "lib/*" -d out $(find src -name '*.java')
java -cp "out:lib/*" fr.nice.calc.Main
```

SORTIE ATTENDUE

```
commons-text-1.14.0.jar: OK
Moyenne : 20.0
MAIN
```

Barème : URL correcte sur `repo1.maven.org` (4 pts) · empreinte téléchargée et vérifiée (5 pts) · `-cp "lib/*"` avec guillemets (3 pts) · exécution avec `out:` avant `lib/*` (3 pts).

> **Pourquoi `lib/*` et non `lib/commons-text-1.14.0.jar`**
>
> Les deux marchent. Le joker évite de modifier la commande à chaque ajout de dépendance — mais il faut le protéger par des guillemets (chapitre 8, erreur 1). Remarquez l'ordre du classpath : `out` d'abord, pour que vos classes priment en cas d'homonymie.

10.3 Exercice 3 — Un JAR exécutable autonome (15 points)

Énoncé. Produisez `calc.jar` exécutable par `java -jar calc.jar` depuis n'importe quel dossier, en embarquant `commons-text` dans le JAR. Puis produisez une seconde version, `calc-thin.jar`, qui utilise `Class-Path` pour référencer `lib/`. Vérifiez les deux depuis `/tmp`.

CORRIGÉ — VERSION FAT (PAR EXTRACTION)

```
cd ~/exos/calc
rm -rf fat && mkdir -p fat
cp -r out/. fat/
cd fat && unzip -qo ../lib/commons-text-1.14.0.jar -x 'META-INF/*' && cd ..
printf 'Manifest-Version: 1.0\nMain-Class: fr.nice.calc.Main\n' > manifest-fat.txt
jar --create --file calc.jar --manifest manifest-fat.txt -C fat .
cd /tmp && java -jar ~/exos/calc/calc.jar
```

SORTIE ATTENDUE

```
Moyenne : 20.0
MAIN
```

CORRIGÉ — VERSION THIN AVEC `CLASS-PATH`

```
cd ~/exos/calc
printf 'Manifest-Version: 1.0\nMain-Class: fr.nice.calc.Main\nClass-Path: lib/commons-
text-1.14.0.jar\n' \
  > manifest-thin.txt
jar --create --file calc-thin.jar --manifest manifest-thin.txt -C out .
cd /tmp && java -jar ~/exos/calc/calc-thin.jar
```

Le piège de `Class-Path` : les chemins sont relatifs au JAR, pas au répertoire courant

`Class-Path: lib/commons-text-1.14.0.jar` est interprété relativement à l'emplacement du JAR. Lancé depuis `/tmp`, `java -jar ~/exos/calc/calc-thin.jar` cherche `~/exos/calc/lib/…` — et le trouve. Mais si vous déplacez le JAR seul, il ne trouve plus rien : d'où l'importance de livrer le dossier `lib/` avec. Un chemin absolu (`/home/user/exos/calc/lib/…`) « marche » et casse sur toute autre machine : ne le faites jamais.

Barème : manifeste avec `Main-Class` et saut de ligne final (4 pts) · `-C out .` correct (3 pts) · fat-jar sans les `META-INF` en conflit (4 pts) · `Class-Path` relatif et version thin fonctionnelle depuis `/tmp` (4 pts).

10.4 Exercice 4 — Un Makefile qui teste et empaquette (15 points)

Énoncé. Écrivez un `Makefile` offrant `all`, `compile`, `test`, `jar`, `run`, `clean`, tel que `make clean && make` produise le JAR et lance les tests, et que `make run` affiche le résultat.

CORRIGÉ — MAKEFILE

```
JAVA_HOME ?= /opt/java/jdk-25.0.5+8
JAVAC  := $(JAVA_HOME)/bin/javac
JAVA   := $(JAVA_HOME)/bin/java
JAR    := $(JAVA_HOME)/bin/jar
RELEASE := 21

LIB    := lib/commons-text-1.14.0.jar
SRC    := $(shell find src -name '*.java')
OUT    := out
JARFILE := calc.jar

.PHONY: all compile jar run clean

all: jar

compile:
        @mkdir -p $(OUT)
        $(JAVAC) --release $(RELEASE) -Xlint:all -cp "$(LIB)" -d $(OUT) $(SRC)

test: compile
        @echo "Pas encore de tests : voir exercice 6"

jar: compile
        printf 'Manifest-Version: 1.0\nMain-Class: fr.nice.calc.Main\nClass-Path: $(LIB)\n' >
manifest.txt
        $(JAR) --create --file $(JARFILE) --manifest manifest.txt -C $(OUT) .
        @echo "→ $(JARFILE) ($(shell du -h $(JARFILE) | cut -f1))"

run: jar
        LANG=C.UTF-8 $(JAVA) -jar $(JARFILE)

clean:
        rm -rf $(OUT) $(JARFILE) manifest.txt
```

VÉRIFICATION

```
make clean && make && make run
```

SORTIE ATTENDUE (EXTRAIT)

```
→ calc.jar (12K)
Moyenne : 20.0
MAIN
```

Barème : `.PHONY` déclaré (2 pts) · `$(shell find …)` pour ne pas lister les fichiers à la main (4 pts) · guillemets autour de `$(LIB)` (3 pts) · `clean` qui supprime réellement les sorties (3 pts) · `run` qui dépend de `jar` (3 pts).

> **Les tabulations, encore et toujours**
>
> Un Makefile exige une **tabulation** en début de ligne de commande, pas des espaces. Message d'erreur : `Makefile:12: *** missing separator. Stop.` Si votre éditeur convertit les tabulations en espaces, désactivez cette option pour les Makefiles (`.editorconfig` : `[Makefile] indent_style = tab`).

10.5 Exercice 5 — Convertir le projet en Maven (20 points)

Énoncé. Reprenez `calc`, transformez-le en projet Maven (`fr.nice:calc:1.0.0`), ciblez Java 21, ajoutez trois tests JUnit 5 dont un test d'exception, produisez un fat-jar exécutable, et vérifiez `mvn dependency:tree`.

CORRIGÉ — ARBORESCENCE ET COMMANDES

```
# 1. La structure Maven (les sources vont dans src/main/java, comme par hasard)
mkdir -p src/main/java/fr/nice/calc src/test/java/fr/nice/calc src/main/resources
mv src/fr/nice/calc/*.java src/main/java/fr/nice/calc/ 2>/dev/null

# 2. Le POM : on part de l'archétype et on l'adapte
cd /tmp && mvn -q archetype:generate -DgroupId=fr.nice -DartifactId=calc \
  -DarchetypeArtifactId=maven-archetype-quickstart -DarchetypeVersion=1.5 \
  -DinteractiveMode=false
cp -r /tmp/calc/.mvn /tmp/calc/pom.xml ~/exos/calc/
```

CORRIGÉ — LE TEST À ÉCRIRE

```
cat > src/test/java/fr/nice/calc/CalculatriceTest.java <<'EOF'
package fr.nice.calc;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertThrows;

import org.junit.jupiter.api.Test;

class CalculatriceTest {

    @Test
    void moyenne_de_trois_valeurs() {
        assertEquals(20.0, Calculatrice.moyenne(10, 20, 30));
    }

    @Test
    void moyenne_d_une_seule_valeur() {
        assertEquals(42.0, Calculatrice.moyenne(42));
    }

    @Test
    void tableau_vide_leve_une_exception() {
        assertThrows(IllegalArgumentException.class, () -> Calculatrice.moyenne());
    }
}
EOF
```

CORRIGÉ — POM : LES TROIS AJOUTS ESSENTIELS

```
<properties>
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  <maven.compiler.release>21</maven.compiler.release>
</properties>

<dependencies>
  <dependency>
    <groupId>org.apache.commons</groupId>
    <artifactId>commons-text</artifactId>
    <version>1.14.0</version>
  </dependency>
</dependencies>

<build><plugins>
  <plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-shade-plugin</artifactId>
    <version>3.6.2</version>
    <executions><execution>
      <phase>package</phase><goals><goal>shade</goal></goals>
      <configuration><transformers>
        <transformer
implementation="org.apache.maven.plugins.shade.resource.ManifestResourceTransformer">
          <mainClass>fr.nice.calc.Main</mainClass>
        </transformer>
      </transformers></configuration>
    </execution></executions>
  </plugin>
</plugins></build>
```

VÉRIFICATION

```
mvn -B -q clean package
java -jar target/calc-1.0.0.jar
mvn -B dependency:tree
```

SORTIE ATTENDUE (EXTRAITS)

```
Tests run: 3, Failures: 0, Errors: 0, Skipped: 0
Moyenne : 20.0
MAIN
[INFO] fr.nice:calc:jar:1.0.0
[INFO] +- org.apache.commons:commons-text:jar:1.14.0:compile
[INFO] \- org.junit.jupiter:junit-jupiter:jar:5.12.2:test
```

Barème : arborescence Maven respectée (4 pts) · `maven.compiler.release` (3 pts) · trois tests dont un `assertThrows` (5 pts) · shade dans `<plugins>` et fat-jar fonctionnel (5 pts) · `dependency:tree` cohérent, JUnit en scope `test` (3 pts).

10.6 Exercice 6 — Diagnostiquer trois pannes (15 points)

Énoncé. Pour chaque message, dites la cause et donnez la commande ou la modification qui corrige — sans lire le chapitre 8 d'abord. Puis vérifiez votre réponse.

N°	Message
a	`error: invalid flag: lib/commons-text-1.14.0.jar`
b	`Exception in thread "main" java.lang.NoClassDefFoundError: org/apache/commons/text/StringSubstitutor`
c	`[FATAL] Non-parseable POM … in comment after two dashes (--)`

Corrigé

a) Le joker `lib/*` a été développé par le shell : le deuxième fichier devient un argument invalide de `javac`. Correctif : `-cp "lib/*"` (guillemets). Voir erreur 1.

b) La classe est absente du classpath d'exécution : soit le JAR n'est pas dans le `-cp`, soit le manifeste n'a pas de `Class-Path`, soit le JAR a été lancé hors de son dossier. Diagnostic : `unzip -p calc.jar META-INF/MANIFEST.MF`. Voir erreur 7.

c) Un commentaire XML contient `--` (souvent `--release`). Correctif : reformuler le commentaire. Voir erreur 8.

Barème : 5 points par panne : cause exacte (3 pts), correctif applicable (2 pts). Une réponse juste « il manque la dépendance » sans préciser où elle manque ne compte que pour 1 point.

10.7 Exercice 7 — Convertir en Gradle et livrer (20 points)

Énoncé. Générez un projet Gradle nommé `calc-gradle` avec le module `app`, portez-y les sources et les tests de `calc`, ajoutez `commons-text` via `libs.versions.toml`, produisez une distribution avec `installDist`, et lancez le script généré. Vérifiez ensuite la version de bytecode produite.

CORRIGÉ — COMMANDES

```
cd ~/exos
gradle init --type java-application --dsl kotlin --java-version 21 \
            --package fr.nice.calc --test-framework junit-jupiter \
            --project-name calc-gradle
cd calc-gradle

# Porter les sources et les tests
rm -f app/src/main/java/fr/nice/calc/App.java app/src/test/java/fr/nice/calc/AppTest.java
cp ../calc/src/main/java/fr/nice/calc/*.java app/src/main/java/fr/nice/calc/
cp ../calc/src/test/java/fr/nice/calc/*.java app/src/test/java/fr/nice/calc/

# Catalogue de versions
cat >> gradle/libs.versions.toml <<'EOF'

commons-text = { module = "org.apache.commons:commons-text", version = "1.14.0" }
EOF

# build.gradle.kts : remplacer guava par commons-text
sed -i 's/implementation(libs.guava)/implementation(libs.commons.text)/' app/build.gradle.kts
sed -i 's/mainClass = "fr.nice.shop.App"/mainClass = "fr.nice.calc.Main"/' app/build.gradle.kts

./gradlew build
./gradlew installDist
LANG=C.UTF-8 app/build/install/app/bin/app
```

SORTIE ATTENDUE (EXTRAITS)

```
BUILD SUCCESSFUL
Moyenne : 20.0
MAIN
```

CORRIGÉ — VÉRIFICATION DE LA CIBLE JAVA

```
javap -verbose -cp app/build/install/app/lib/app.jar fr.nice.calc.Calculatrice | grep major
ls app/build/install/app/lib/
```

SORTIE ATTENDUE

```
  major version: 65
app.jar
commons-lang3-3.20.0.jar
commons-text-1.14.0.jar
error_prone_annotations-2.41.0.jar
```

Barème : `gradle init` correct (4 pts) · sources et tests au bon endroit (4 pts) · dépendance via le catalogue et non en dur (4 pts) · `mainClass` et `installDist` fonctionnels (4 pts) · `major version: 65` vérifiée à `javap`, donc toolchain réellement appliquée (4 pts).

> **Ce que cet exercice démontre**
>
> Le module `app` embarque `commons-text` et sa dépendance transitive `commons-lang3` — sans que vous l'ayez déclarée. C'est exactement ce que le parcours manuel vous aurait obligé à télécharger à la main. Et la version de bytecode prouve que le JDK 25 qui exécute Gradle n'a pas produit de bytecode 25.

10.8 Exercice 8 — Justifier un choix d'outil (10 points)

Énoncé. Pour chacun de ces trois projets, choisissez l'outil (à la main, Maven, Gradle) et justifiez en trois phrases maximum : **(a)** une bibliothèque de calcul de TVA publiée sur Maven Central, 12 classes,

3 dépendances ; **(b)** une application Android de 6 modules ; **(c)** un script d'export de 200 lignes utilisé une fois par mois par un administrateur sur un serveur sans accès à Maven Central.

Corrigé (une réponse acceptable parmi d'autres, si elle est argumentée)

(a) Maven. Maven Central impose des métadonnées précises (POM complet, sources, Javadoc, signatures GPG) que Maven outille directement ; l'écosystème de la bibliothèque attend un POM. Gradle serait acceptable avec `maven-publish`.

(b) Gradle. Android ne supporte officiellement que Gradle : la question est tranchée par la plateforme, pas par préférence. Les variantes de build (debug/release/flavors) y sont natives.

(c) À la main. Un seul fichier, aucune dépendance, serveur sans accès au réseau : un dossier `src`, un `javac -d out`, un `jar`, un script de lancement. Ajouter Maven reviendrait à exiger un dépôt accessible pour un besoin qui n'en a aucun.

Barème : choix cohérent (1 pt par projet) · justification par un fait vérifiable, pas par une préférence (1,5 pt par projet) · une phrase qui pèse explicitement l'alternative (0,5 pt par projet). Perdre des points : citer « c'est plus moderne », « tout le monde l'utilise », ou « Gradle est plus rapide » sans mesure.

ATELIER FINAL — 45 minutes, le parcours complet

Objectif : enchaîner les trois parcours sur un projet neuf, de la première ligne au JAR livré.

1. **(5 min)** Choisissez un mini-domaine (convertisseur, gestion de stock, calcul d'impôt). Écrivez une classe métier et trois tests.
2. **(12 min)** Parcours A : arborescence, `javac`, `java -cp`, une dépendance `commons-lang3` vérifiée, un Makefile, un JAR avec manifeste. Lancez le JAR depuis `/tmp`.
3. **(12 min)** Parcours B : même code en Maven, tests verts, `dependency:tree`, fat-jar, `./mvnw`.
4. **(10 min)** Parcours C : même code en Gradle, `build`, `installDist`, distribution lancée depuis son dossier `bin/`.
5. **(6 min)** Comparez les trois durées de build (`time`) et les trois tailles livrées. Écrivez quatre lignes de conclusion : ce que vous garderiez de chaque approche.

11 Sources et pour aller plus loin

> Toutes les commandes de ce document ont été exécutées sur Debian 13 avec Temurin 25.0.5, Maven 3.10.0 et Gradle 9.9.0 en novembre 2026. Les références ci-dessous sont les documentations officielles : ce sont elles qui font foi, et c'est vers elles qu'il faut revenir quand une version change.

11.1 Apache Maven

Document	Ce qu'on y trouve, et où on s'en sert dans ce document
`maven.apache.org/guides/getting-started/maven-in-five-minutes.html`	Le premier projet, le cycle de base. Chapitre 4, introduction.
`maven.apache.org/pom.html`	POM Reference : chaque élément du POM, y compris `dependencyManagement` et `pluginManagement`. Chapitre 4.6.
`maven.apache.org/guides/introduction/introduction-to-the-lifecycle.html`	Les trois cycles de vie et l'ordre des phases. Chapitre 4.7.
`maven.apache.org/guides/introduction/introduction-to-the-standard-directory-layout.html`	La structure `src/main/java`, `src/test/java`, `target/`. Chapitre 3.1 et 4.2.
`maven.apache.org/guides/introduction/introduction-to-dependency-mechanism.html`	Transitivité, scopes, résolution des conflits, BOM. Chapitre 4.8.
`maven.apache.org/guides/introduction/introduction-to-archetypes.html` et `…/maven-archetype-plugin/usage.html`	Le catalogue d'archétypes, `-Dfilter`, mode interactif. Chapitre 4.2.
`maven.apache.org/settings.html`	`settings.xml` : miroirs, serveurs, profils. Chapitre 4.8 et 7.3.
`maven.apache.org/wrapper/`	Maven Wrapper : `mvnw`, `distributionUrl`, `only-script`. Chapitre 4.10.
`maven.apache.org/guides/mini/guide-using-toolchains.html`	JDK de compilation distinct du JDK d'exécution. Chapitre 5.3.
`maven.apache.org/download.cgi` · `archive.apache.org/dist/maven/`	Archives officielles et empreintes (SHA-512, SHA-1, MD5). Chapitre 4.1.
`maven.apache.org/plugins/` (maven-jar-plugin, maven-shade-plugin, maven-surefire-plugin, maven-dependency-plugin, exec-maven-plugin)	Configuration exacte de chaque plugin utilisé : manifeste, fat-jar, rapports de tests, classpath, exécution. Chapitres 4.6, 4.7.

11.2 Gradle

Document	Contenu utile
`docs.gradle.org/current/userguide/userguide.html`	Le manuel de référence : tâches, cycles de build, caches. Chapitre 5.
`docs.gradle.org/current/userguide/build_init_plugin.html`	`gradle init` : types de projet, options, DSL. Chapitre 5.1.
`docs.gradle.org/current/userguide/application_plugin.html`	`run`, `installDist`, `distZip`, `startScripts`, `mainClass`. Chapitre 5.2.
`docs.gradle.org/current/userguide/toolchains.html`	Toolchains, auto-détection, auto-provisioning des JDK. Chapitre 5.3.
`docs.gradle.org/current/userguide/version_catalogs.html`	`libs.versions.toml` : format et règles. Chapitre 5.2.
`gradle.org/install/` · `gradle.org/releases/`	Installation officielle, versions et notes de version. Chapitre 5.1.

11.3 JDK : outils en ligne de commande

Document	Contenu utile
`docs.oracle.com/en/java/javase/25/docs/specs/man/javac.html`	`-d`, `--release`, `-cp`, `-Xlint`, `-encoding`. Chapitre 3.2.
`docs.oracle.com/en/java/javase/25/docs/specs/man/java.html`	`-jar`, `-cp`, classpath, mode source-file. Chapitres 3.2, 3.5.
`docs.oracle.com/en/java/javase/25/docs/specs/man/jar.html`	`--create`, `--manifest`, `-C`, `--list`. Chapitre 3.7.
`docs.oracle.com/en/java/javase/25/docs/specs/man/jdeps.html`	Analyse des modules nécessaires, `--multi-release`. Chapitre 4.10.
`docs.oracle.com/en/java/javase/25/docs/specs/man/jlink.html`	Runtime minimal : `--add-modules`, `--compress`, `--strip-debug`. Chapitre 4.10.
`docs.oracle.com/en/java/javase/25/docs/specs/man/jpackage.html`	Installateurs natifs `.deb`, `.rpm`, `.msi`. Chapitre 4.10.
`docs.oracle.com/en/java/javase/25/docs/specs/man/javap.html`	Version de bytecode (`major version`). Chapitres 7.5 et 8.1.

11.4 Tests et dépôts

Document	Contenu utile
`docs.junit.org/current/user-guide/`	JUnit 5 : `@Test`, assertions, `assertThrows`, lancement par la console. Chapitres 4.9, 10.5.
`central.sonatype.com/` · `search.maven.org`	Maven Central : rechercher un artefact, vérifier la dernière version publiée. Partout.
`repo1.maven.org/maven2/`	Accès direct aux JAR et à leurs empreintes — ce que fait le parcours A à la main. Chapitre 3.4.
`start.spring.io` · `www.jhipster.tech`	Générateurs de projets Maven/Gradle complets. Chapitre 4.2.

11.5 Les deux scripts fournis avec ce document

Deux scripts accompagne ce PDF. Ils produisent chacun un projet complet, identique à celui des chapitres 3 et 4, et ils ont été exécutés de bout en bout avant publication.

Script	Ce qu'il fait
`creer-projet-manuel.sh`	Crée l'arborescence, écrit les sources, les ressources et les tests, télécharge `commons-lang3` et le lanceur JUnit 5 avec vérification du SHA-1, compile avec `javac --release 21`, lance les tests, produit un JAR avec manifeste et `Class-Path`, et écrit un Makefile. Aucune dépendance à Maven ou Gradle.
`creer-projet-maven.sh`	Génère un projet Maven complet : POM commenté (propriétés, BOM JUnit, dépendance, `pluginManagement`, jar + shade), sources, ressources, trois tests JUnit 5, puis `mvn package` et génération du wrapper `./mvnw`. Le POM produit est exactement celui du chapitre 4.5.

UTILISATION

```
chmod +x creer-projet-manuel.sh creer-projet-maven.sh
./creer-projet-manuel.sh ~/projets/boutique
./creer-projet-maven.sh  ~/projets/boutique-maven
cd ~/projets/boutique-maven && ./mvnw -q package && java -jar target/boutique-1.0-SNAPSHOT.jar
Alice
```

SORTIE ATTENDUE

```
Alice : 59.88 EUR TTC
Après remise de 10 % : 53.89 EUR
```

> **Ce que ces scripts ne font pas**
>
> Ils ne remplacent pas la lecture : ils n'expliquent rien et n'affichent pas les erreurs pédagogiques. Leur but est de vous donner une base propre et reproductible, puis de comparer avec ce que vous auriez écrit à la main. Si un script échoue chez vous, c'est presque toujours l'environnement : `JAVA_HOME` absent, version de JDK trop ancienne, ou accès réseau à Maven Central bloqué.

11.6 Pour continuer

Sujet	Par où commencer
Tester sérieusement	JUnit 5 (paramétrisé, `@Nested`), AssertJ, Mockito. Puis la mesure de couverture (JaCoCo en Maven, `jacoco` en Gradle).
Industrialiser (CI)	Un fichier de pipeline qui lance `./mvnw -B verify` ou `./gradlew build`. Le wrapper est la clé de la reproductibilité.
Modules Java (JPMS)	`module-info.java`, puis `jlink`. Point d'entrée : le guide Oracle « Java Platform Module System ».
Multi-modules	Côté Maven : POM parent + `<modules>` + `dependencyManagement`. Côté Gradle : `include()` et `settings.gradle.kts`.
Conteneurs	`Dockerfile` à base de `eclipse-temurin:21-jre`, build multi-étapes avec le wrapper.
Publication open source	`central-publishing-maven-plugin`, signatures GPG, revendication du groupId sur `central.sonatype.com`.

La conclusion en une phrase

Un gestionnaire de build n'est pas là pour taper les commandes à votre place : il est là pour que **tout le monde tape les mêmes**. Savoir faire à la main (chapitre 3) est ce qui vous permet de comprendre ce qu'il tape ; savoir lire un POM et un `build.gradle.kts` est ce qui vous permet de le corriger quand il se trompe. Les deux compétences s'acquièrent dans cet ordre — et ne se perdent plus.